

Lenguaje de Programación JAVA

<https://docs.oracle.com/javase/specs/index.html>

© JMA 2001. All rights reserved

Contenidos

- Introducción a la plataforma
 - Características
 - Versiones y ediciones. Licenciamiento.
 - Arquitectura
 - Entorno de ejecución
 - Entorno de desarrollo
 - Estructura de programas y código
- Utilidades
 - jshell
 - javadoc
- Introducción al lenguaje
 - Sintaxis, identificadores y notaciones
 - Tipos primitivos, literales, casting e inferencia
 - Variables, referencias, arrays y constantes
 - Operadores, expresiones e instrucciones
- Clases
 - Declaración
 - Atributos y métodos
 - Encapsulación y modificadores
 - Constructores, inicializadores y finalizadores
 - Ciclo de vida y gestión de memoria
 - Herencia y Abstracción
 - Polimorfismo, sobrecarga y reemplazo
 - Clases internas, selladas, record
- Interfaces
 - Declaración
 - Implementación
 - Métodos por defecto, estáticos y privados
 - Patrón delegate e interfaces funcionales
 - Expresiones lambda y programación funcional
- Genéricos
 - Tipos Genéricos o parametrizados
 - Métodos Genéricos
 - Borrado de tipos
 - Restricciones sobre genéricos
 - Inferencia de tipos
- Enumerados
 - Declaración
 - Constructores, atributos y métodos
- Gestión de excepciones.
 - Errores en tiempo de ejecución y de compilación
 - Excepciones
 - Tratamiento de las excepciones
 - Captura de múltiples tipos de excepción
 - Crear, lanzar, relanzar y propagar excepciones
 - Try-con-recursos
 - Aserciones
 - Tratamiento de nulos, Optional
 - Registro y trazas
- Reflexión
 - Ventajas e Introducción
 - Objetos de clase
 - Obtención y acceso de Miembros
 - Restringir el acceso a miembros
- Anotaciones
 - Metadatos
 - Anotaciones predefinidas
 - Crear y consumir anotaciones
- Paquetes y Módulos
 - Paquetes
 - Importaciones y referenciado
 - Administrar archivos
 - Modularidad, tipos, JDK, grafos
 - Creación de módulos
- Manipulación de datos
 - Abrir, Exportar y Requerir
 - Servicios: proveer y usar
 - Seguridad integrada: políticas, permisos, firmar código
- Colecciones
 - Objetos, clonación y comparaciones
 - java.Lang, java.util,
 - Numéricos, autoboxing y unboxing
 - Fechas
 - Cadenas: generación, extracción y formateo.
 - Arrays
 - Expresiones regulares
- Streams
 - Definición y obtención de flujos
 - Filtrado
 - Mapeado
 - Reducción
 - Paralelismo

© JMA 2001. All rights reserved

INTRODUCCIÓN

© JMA 2001. All rights reserved

Características

- Sun lo describe como:
 - “simple, orientado a objetos, distribuido, interpretado, robusto, seguro, de arquitectura neutra, portable, de altas prestaciones, multitarea y dinámico”.
- Toma su sintaxis del C++.
- Recogía las tendencias actuales en desarrollo de software.
- Soporta:
 - Encapsulado
 - Herencia simple
 - Polimorfismo
 - Sobrecarga de métodos
 - Tratamiento de excepciones
- Java es tanto un lenguaje de programación como una plataforma de ejecución (JVM). Originalmente estaban asociados, actualmente se puede programar en Java para otras plataformas y las JVM se puede programar en otros lenguajes diferentes al Java (Groovy, Kotlin, Scala, ...)

© JMA 2001. All rights reserved

Historia

- Historia:
 - Lenguaje desarrollado por ingenieros de Sun Microsystems en 1991 destinado a electrodomésticos.
 - Netscape incorpora en 1995 a su navegador la versión de Java 1.0 (12 packages).
- Versiones de JAVA
 - Java JDK 1.0 (1996), java 1.1 (1997)
 - Java J2SE 1.2 (1999) (Java 2 Standard Edition)
 - Java J2SE 1.3 (2000)
 - Java J2SE 1.4 (2002)
 - Java 1.5 J2SE (2004)
 - Java SE 6 (2006) (Se cambia el nombre y se elimina 1.X)
 - Java SE 7 (2011)
 - Java SE 8 (2014)
 - Java SE 9 (2017), Java SE 10 (2018), Java SE 11 LTS (2018), Java SE 17 LTS (2021), Java SE 21 LTS (2023), Java SE 25 LTS (2025), ...[semestral(marzo/septiembre)+LTS]

© JMA 2001. All rights reserved

Versionado

- A partir de la publicación de Java 9 se cambia el modelo de publicación de nuevas versiones optando por uno basada en calendario en vez de por características a incluir.
- Se ha optado por producir una nueva versión cada seis meses (marzo y septiembre) con el OpenJDK y licencia GPL para satisfacer las necesidades de los desarrolladores y una versión con soporte de largo plazo (LTS) cada tres años (septiembre) para satisfacer las necesidades de tiempo de soporte prolongado de las empresas, Oracle JDK.
- Se elimina el soporte gratuito de versiones OpenJDK por parte de Oracle salvo para la última versión (estrictamente), la salida de una versión cancela el soporte de las anteriores.
- Se incluyen las Preview Features
- Se elimina la distinción entre Java Development Kit (JDK) y Java Runtime Environment (JRE), queda la JDK y desaparece el JRE.

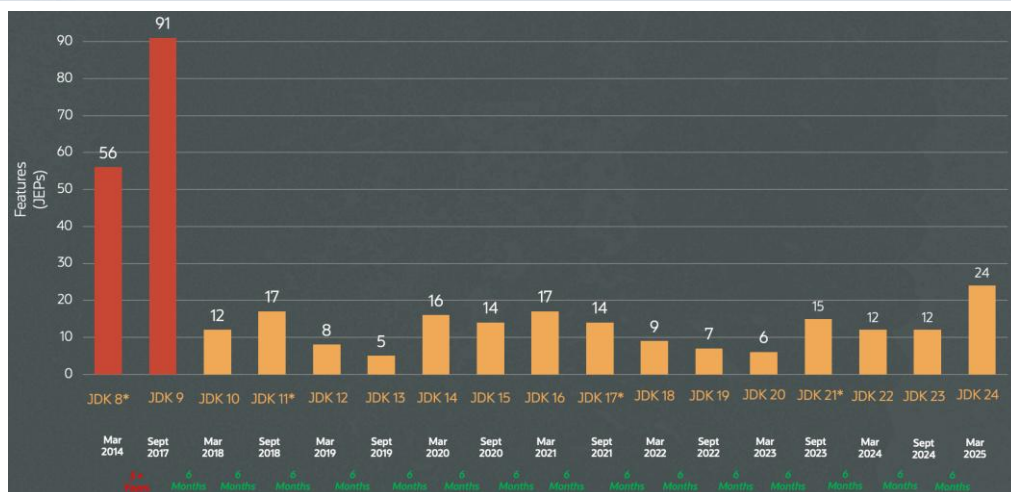
© JMA 2001. All rights reserved

Preview Features

- Una función en vista previa es una nueva función cuyo diseño, especificación e implementación están completos, pero que no es permanente, lo que significa que la función puede existir en una forma diferente o no existir en las futuras versiones de JDK.
- Esto permite que la mayor audiencia posible de desarrolladores pruebe la función en el mundo real y proporcione comentarios.
- Para activarlas es necesario:
 - `javac --enable-preview --release 12 MyApp.java`
- Las funciones de vista previa no deben utilizarse en contextos de aplicaciones para explotación.

© JMA 2001. All rights reserved

Aceleración de la innovación



© JMA 2001. All rights reserved

Licencia de código binario

- Binary Code License for Java SE (“BCL”)?
 - Puedes usarlo, pero no puedes modificarlo
 - Aceptas no demandar a Oracle si algo sale mal
 - Puedes redistribuirlo / publicarlo (para que puedas vender productos con Java integrado), pero si lo haces, aceptas indemnizar a Oracle; así que, si alguien te demanda, no puedes arrastrar a Oracle contigo.
- Oracle JDK (licencias de Oracle)
 - BCL Public (proyectos de código abierto)
 - BCL Non-comercial (uso personal o educativo)
 - Comercial License
- Oracle OpenJDK (características)
 - GPL: Sin soporte, con actualizaciones exclusivamente para la versión actual (incluidas las LTS)
- Alternativas a JRE (binarios no disponibles a partir de la versión 11)
 - Producciones propias: jlink (OpenJDK)
 - Proveedores OpenJDK

© JMA 2001. All rights reserved

OpenJDK Providers

- Oracle JDK - <https://www.oracle.com/technetwork/java/javase/downloads>
- Oracle OpenJDK - <http://jdk.java.net>
- Eclipse Temurin - <https://adoptium.net/es/temurin/releases/>
- RedHat - <https://developers.redhat.com/products/openjdk/overview>
- Amazon – Corretto - <https://aws.amazon.com/corretto>
- AdoptOpenJDK - <https://adoptopenjdk.net>
- Azul Zulu - <https://www.azul.com/downloads/zulu/>
- BellSoft Liberica - <https://bell-sw.com/java.html>
- IBM - <https://www.ibm.com/developerworks/java/jdk>
- jClarity - <https://www.jclarity.com/adoptopenjdk-support/>
- OpenJDK Upstream - <https://adoptopenjdk.net/upstream.html>
- ojdkbuild - <https://github.com/ojdkbuild/ojdkbuild>
- SapMachine - <https://sap.github.io/SapMachine>

© JMA 2001. All rights reserved

Plataformas de Java

- Java no es solo un lenguaje de programación, sino una plataforma completa que permite desarrollar y ejecutar aplicaciones en una amplia gama de dispositivos y sistemas operativos. Esta portabilidad se logra gracias a la Máquina Virtual de Java (JVM), que interpreta el bytecode Java en el sistema operativo subyacente.
- La plataforma Java se divide históricamente en varias "ediciones" o plataformas, cada una diseñada para satisfacer diferentes necesidades de desarrollo:
 - Standard Edition (J2SE, Java SE)
 - Enterprise Edition (J2EE, JEE, Java EE) ahora Jakarta EE
 - Micro Edition (J2ME, Java ME, Java SE Embedded)
 - JavaFX (rich client platform), Java DB (Apache Derby), Java Card (smart cards), Java TV (Java ME to smart TV).

© JMA 2001. All rights reserved

Java Platform, Standard Edition (Java SE)

- **Descripción:**

Es la base de la plataforma Java. Proporciona el JDK (Java Development Kit), que incluye el compilador de Java, el JRE (Java Runtime Environment) para ejecutar aplicaciones, y la JVM. Contiene las API fundamentales del lenguaje, la biblioteca de clases estándar (colecciones, E/S, redes, concurrencia, etc.), y las herramientas para desarrollar aplicaciones de escritorio (con frameworks como Swing o JavaFX, que ahora es un módulo separado) y aplicaciones de servidor de propósito general.
- **Uso principal:**

Aplicaciones de escritorio, aplicaciones de línea de comandos, desarrollo de componentes y librerías, y la base para otras ediciones de Java. Es el punto de partida para cualquier desarrollador Java.

© JMA 2001. All rights reserved

Java Platform, Micro Edition (Java ME, Java SE Embedded)

- **Descripción:**

Diseñada para dispositivos con recursos limitados, como teléfonos móviles antiguos (antes de los SmartPhones), PDAs, decodificadores de TV, impresoras y dispositivos de IoT (Internet of Things). Proporciona un subconjunto de las API de Java SE y una máquina virtual más ligera (KVM o CVM) optimizada para memoria y consumo de energía. Se basa en "configuraciones" y "perfiles" para adaptar la plataforma a las capacidades específicas del dispositivo.

- **Uso principal:**

Desarrollo para dispositivos embebidos, IoT, y la base de muchas aplicaciones móviles antes del auge de los smartphones. Aunque su popularidad ha disminuido con Android (que usa Java, pero con su propio SDK y JVM - Dalvik/ART), sigue siendo relevante en ciertos nichos.

© JMA 2001. All rights reserved

Java Platform, Enterprise Edition (Java EE) / Jakarta EE

- **Descripción:**

Construida sobre Java SE, Java EE (hasta la versión 8 ahora conocida como Jakarta EE desde que Oracle la donó en 2018 a la Eclipse Foundation) está diseñada para el desarrollo de aplicaciones empresariales a gran escala, distribuidas, escalables y seguras. Proporciona un conjunto extenso de APIs y especificaciones para servicios web (RESTful y SOAP), componentes de negocio (Enterprise JavaBeans - EJB), persistencia de datos (JPA), mensajería (JMS), seguridad, transacciones y más. Las aplicaciones Java EE se ejecutan en servidores de aplicaciones (como Tomcat, WildFly, GlassFish, WebLogic, WebSphere, ...).

- **Uso principal:**

Aplicaciones web empresariales, microservicios, APIs REST, servicios de integración, sistemas transaccionales y aplicaciones distribuidas.

© JMA 2001. All rights reserved

Aplicaciones empresariales

- La plataforma Java EE está diseñada para ayudar a los desarrolladores a crear aplicaciones de red a gran escala, multicapa, escalables, fiables y seguras. Un nombre abreviado para estas aplicaciones es "aplicaciones empresariales", llamadas así porque están diseñadas para resolver los problemas que enfrentan las grandes empresas. Sin embargo, las aplicaciones empresariales no solo son útiles para grandes corporaciones, agencias y gobiernos. Los beneficios de una aplicación empresarial son útiles, incluso esenciales, para desarrolladores individuales y pequeñas organizaciones en un mundo cada vez más interconectado.
- Las características que hacen potentes las aplicaciones empresariales, como la seguridad y la fiabilidad, suelen hacerlas complejas. La plataforma Java EE reduce la complejidad del desarrollo de aplicaciones empresariales al proporcionar un modelo de desarrollo, una API y un entorno de ejecución que permiten a los desarrolladores centrarse en la funcionalidad.
- En una aplicación multicapa, su funcionalidad se divide en áreas funcionales aisladas, denominadas niveles. Normalmente, las aplicaciones multicapa constan de un nivel de cliente, un nivel intermedio (negocio) y un nivel de datos (a menudo denominado nivel de sistemas de información empresarial). El desarrollo de aplicaciones Java EE se concentra en el nivel intermedio para hacer que la gestión de aplicaciones empresariales sea más fácil, más sólida y más segura.

© JMA 2001. All rights reserved

Nivel de cliente

- El nivel de cliente consiste en un programa cliente que realiza solicitudes al nivel intermedio. Este se divide en un nivel web y un nivel empresarial, que gestionan las solicitudes de los clientes y procesan los datos de la aplicación, almacenándolos en un almacén de datos permanente en el nivel de datos.
- La capa web consta de componentes que gestionan la interacción entre los clientes y la capa empresarial. Sus principales tareas son:
 - Generar contenido dinámicamente en varios formatos para el cliente.
 - Recopilar información de los usuarios de la interfaz del cliente y devolver los resultados apropiados de los componentes en el nivel empresarial
 - Controlar el flujo de pantallas o páginas en el cliente
 - Mantener el estado de los datos de la sesión de un usuario
 - Realizar cierta lógica básica y mantener algunos datos temporalmente en beans administrados

© JMA 2001. All rights reserved

Nivel de cliente

- Servlets
 - Clases del lenguaje de programación Java que procesan solicitudes dinámicamente y construyen respuestas, generalmente páginas HTML
- Tecnología JavaServer Faces
 - Un marco de componentes de interfaz de usuario para aplicaciones web que le permite incluir componentes de UI (como campos y botones) en una página XHTML, llamada página Facelets; convertir y validar datos de componentes de UI; guardar datos de componentes de UI en almacenes de datos del lado del servidor; y mantener el estado del componente
- Lenguaje de expresión
 - Un conjunto de etiquetas estándar utilizadas en las páginas Facelets para hacer referencia a componentes de Java EE
- Contextos e inyección de dependencias para Java EE
 - Un conjunto de servicios contextuales que facilitan a los desarrolladores el uso de beans empresariales junto con la tecnología JavaServer Faces en aplicaciones web

© JMA 2001. All rights reserved

Capa de negocio

- La capa de negocio consta de componentes que proporcionan la lógica de negocio para una aplicación. La lógica de negocio es el código que proporciona funcionalidad a un dominio empresarial específico, como el sector financiero o un sitio de comercio electrónico. En una aplicación empresarial bien diseñada, la funcionalidad principal reside en los componentes de la capa de negocio.
- Las siguientes tecnologías Java EE se encuentran entre las que se utilizan en el nivel empresarial en aplicaciones Java EE:
 - Componentes de Enterprise JavaBeans (enterprise bean)
 - Servicios web RESTful JAX-RS
 - Entidades de la API de persistencia de Java

© JMA 2001. All rights reserved

Nivel de sistemas de información empresarial (EIS)

- El nivel de sistemas de información empresarial (EIS) consta de servidores de bases de datos, sistemas de planificación de recursos empresariales (ERP) y otras fuentes de datos heredadas, como mainframes. Estos recursos suelen estar ubicados en una máquina independiente del servidor Java EE y los componentes del nivel empresarial acceden a ellos.
- Las siguientes tecnologías Java EE se utilizan para acceder al nivel EIS en aplicaciones Java EE:
 - La API de conectividad de base de datos de Java (JDBC)
 - La API de persistencia de Java
 - La arquitectura del conector Java EE
 - La API de transacciones de Java (JTA)

© JMA 2001. All rights reserved

Servidores de aplicaciones

- Un servidor de aplicaciones es un software que proporciona un entorno para ejecutar aplicaciones empresariales, especialmente aquellas desarrolladas con tecnologías como Java EE (ahora Jakarta EE). Su propósito principal es gestionar y ejecutar componentes del lado del servidor (como servlets, EJBs, servicios web, etc.).
- En esencia, un servidor de aplicaciones se encarga de:
 - Generación de contenido dinámico
 - Ejecución de la lógica de negocio
 - Conectividad con bases de datos y otros sistemas (mensajería)
 - Gestión de transacciones
 - Gestión de sesiones
 - Seguridad
 - Agrupación de recursos (Connection Pooling)
 - Alta disponibilidad y escalabilidad
 - Soporte a múltiples protocolos

© JMA 2001. All rights reserved

Servidores y contenedores Java EE

- Un servidor Java EE es una aplicación de servidor que implementa las API de la plataforma Java EE y proporciona servicios estándar de Java EE, se denominan servidores de aplicaciones porque permiten servir datos de aplicaciones a los clientes, de forma similar a como los servidores web sirven páginas web a los navegadores. Los servidores Java EE alojan varios tipos de componentes de aplicación que corresponden a los niveles de una aplicación multicapa. El servidor Java EE proporciona servicios a estos componentes en forma de contenedor.
- Los contenedores Java EE son la interfaz entre el componente y la funcionalidad de nivel inferior que la plataforma proporciona para soportarlo. La funcionalidad del contenedor la define la plataforma y varía según el tipo de componente. Sin embargo, el servidor permite que los diferentes tipos de componentes colaboren para proporcionar funcionalidad en una aplicación empresarial.
- Todos los servidores de aplicaciones incluyen un contenedor web, pero no todos los contenedores web son servidores de aplicaciones.
- Los servidores de aplicaciones son cruciales para el desarrollo y despliegue de aplicaciones empresariales basadas en las especificaciones de Java EE.

© JMA 2001. All rights reserved

Servidores y contenedores Java EE

- El **contenedor web** es la interfaz entre los componentes web y el servidor web. Un componente web puede ser un servlet o una página Facelets de JavaServer Faces. El contenedor gestiona el ciclo de vida del componente, envía solicitudes a los componentes de la aplicación y proporciona interfaces para los datos de contexto, como la información sobre la solicitud actual.
- El **contenedor EJB** es la interfaz entre los enterprise beans, que proporcionan la lógica de negocio en una aplicación Java EE, y el servidor Java EE. El contenedor EJB se ejecuta en el servidor Java EE y gestiona la ejecución de los enterprise beans de una aplicación.
- El **contenedor de cliente de aplicación** es la interfaz entre los clientes de aplicación Java EE (aplicaciones Java SE especiales que utilizan componentes de servidor Java EE) y el servidor Java EE. El contenedor de cliente de aplicación se ejecuta en el equipo cliente y actúa como enlace entre la aplicación cliente y los componentes de servidor Java EE que utiliza el cliente.

© JMA 2001. All rights reserved

Comparación general

Característica	Contenedor Web	Servidor de Aplicaciones
Propósito principal	Ejecutar aplicaciones web (Servlets, JSP)	Ejecutar aplicaciones empresariales completas
Alcance	Capa de presentación (HTTP)	Capa de presentación y lógica de negocio
APIs soportadas	Servlet, JSP, WebSocket	Java EE/Jakarta EE completo: EJB, JPA, JMS, JAX-RS
Transacciones distribuidas	✗ No	✓ Sí
Mensajería (JMS)	✗ No	✓ Sí
Persistencia JPA	✗ No (solo si se integra manualmente)	✓ Integrado
Escalabilidad y clustering	Limitada	Avanzada
Ejemplos	Apache Tomcat , Jetty, Undertow	GlassFish, Payara, WildFly , WebLogic, WebSphere

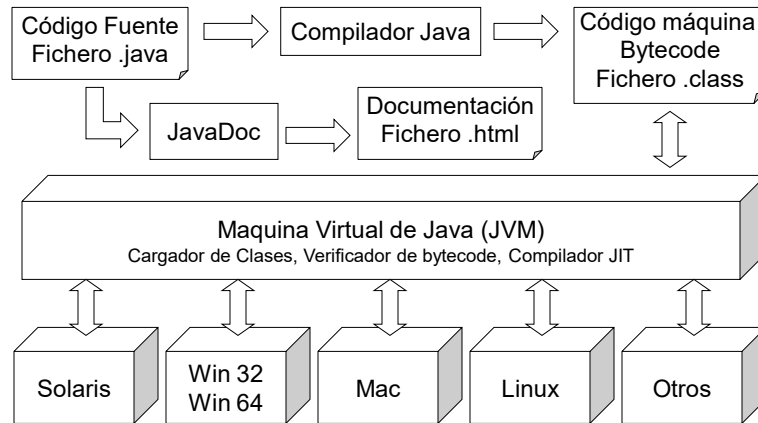
© JMA 2001. All rights reserved

Servidores de Aplicaciones Java (Java EE / Jakarta EE)

Nombre	Tipo	Notas
Apache Tomcat	Contenedor web (no full EE)	Soporta solo Servlet/JSP. Ligero, muy popular.
WildFly (antes JBoss)	Full Jakarta EE	Soporte completo de Java EE. Patrocinado por Red Hat.
Payara Server	Full Jakarta EE	Derivado de GlassFish, robusto y moderno.
GlassFish	Referencia oficial	Base de Jakarta EE. Completo, pero menos usado en producción.
WebLogic	Comercial (Oracle)	Usado en grandes empresas, muy robusto.
WebSphere	Comercial (IBM)	Potente, orientado a entornos corporativos complejos.

© JMA 2001. All rights reserved

Arquitectura Java



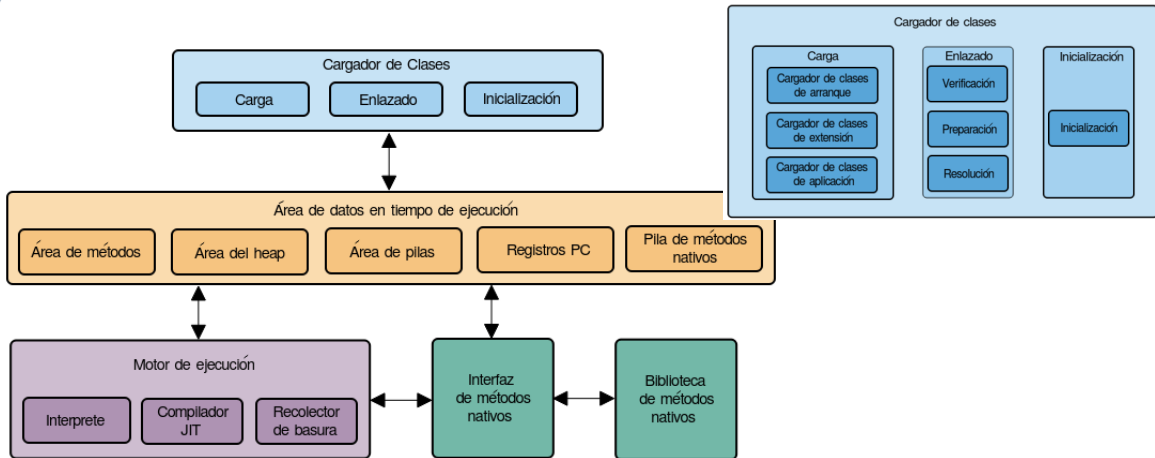
© JMA 2001. All rights reserved

La Máquina Virtual Java (JVM)

- La Máquina Virtual Java (JVM) es un entorno de ejecución que permite a las aplicaciones escritas en Java ejecutarse en cualquier plataforma, independientemente del sistema operativo o hardware original, garantizando así la portabilidad del código. Funciona interpretando y ejecutando el bytecode (código intermedio o código máquina virtual) generado por el compilador Java, actuando como un intermediario entre el código de la aplicación y el sistema físico.
- Los componentes y funciones principales de la JVM son:
 - Gestión de memoria: Incluye un gestor de asignación de memoria y un recolector de basura que se encarga de liberar automáticamente la memoria que ya no está en uso.
 - Cargador de clases (Class Loader): Se encarga de cargar los archivos .class que contienen el bytecode en la memoria.
 - Verificador de bytecode (Bytecode Verifier): Asegura que el bytecode sea válido y seguro antes de su ejecución.
 - Intérprete de bytecode: Lee y ejecuta las instrucciones del bytecode de forma secuencial.
 - Compilador Just-In-Time (JIT): Compila y optimiza el bytecode a código nativo específico de la máquina física de ejecución, mejorando el rendimiento de la aplicación.
- Cómo funciona (ciclo de vida de un programa Java):
 - Compilación: El código fuente de Java (.java) es compilado con javac a un archivo de bytecode (.class) o empaquetado (.jar).
 - Carga: El Class Loader localiza y carga el archivo .class en la memoria RAM.
 - Verificación: El verificador de bytecode se asegura de que el código sea correcto y seguro.
 - Inicialización: La JVM prepara el código, asigna valores a las variables y comienza la ejecución.
 - Ejecución: El intérprete o el compilador JIT ejecutan el bytecode, traducéndolo a instrucciones que la máquina física puede entender.

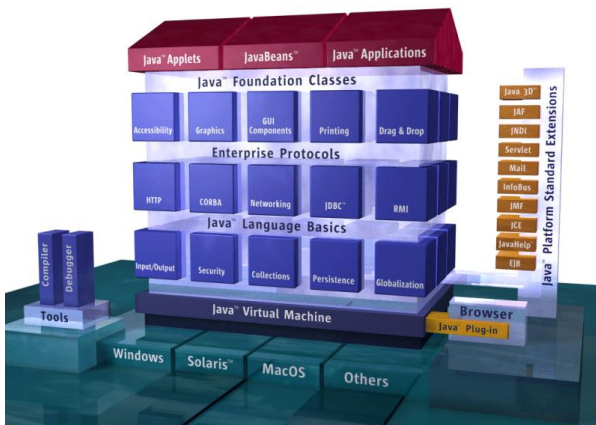
© JMA 2001. All rights reserved

Arquitectura de la JVM



© JMA 2001. All rights reserved

Entorno de desarrollo



© JMA 2001. All rights reserved

- La JVM viene acompañada por una extensa biblioteca de clases y frameworks estándar.
- Instalación:
 - JRE (Java Runtime Environment)
 - JDK (Java Development Kit)

A partir de JDK 9 también se eliminó la distinción entre Java Development Kit (JDK) y Java Runtime Environment (JRE).
- Variables de entorno:
 - CLASSPATH
 - PATH
- Comandos (incluidos en el JDK):
 - java, javac, jar, jdeps, jshell, javadoc, ...
- IDEs:
 - Eclipse, NetBeans, IntelliJ, JBuilder, ...

Estructura de un programa

- El Java es un lenguaje que permite la creación de nuevos tipos: clases, interfaces y enumeraciones.
- Un fichero .java solo puede contener un único tipo (no anidado)
 tipo.java → tipo.class
- Los ficheros se almacenan en una jerarquía de directorios y subdirectorios. Cada uno de ellos se denomina paquete.
- Un directorio, individualmente o con sus subdirectorios, se puede empaquetar en un fichero comprimido .jar.
 dir → dir.jar
- Conjuntos de paquetes se agrupan en módulos. Cada módulo establece cuales de sus paquetes se pueden utilizar externamente. Así mismo, establece sus dependencias, de que otros módulos depende porque contienen los paquetes con los tipos necesarios para su código.
- Cada fichero .java debe indicar el origen de los tipos utilizados mediante su nombre cualificado (que incluye el paquete) o mediante importaciones.

© JMA 2001. All rights reserved

Estructura del código

```
[package <nombre_paquete>;]  
[<Importación de paquetes>]  
  
public class <NombreAplicación> [<herencia>] [<interface>] {  
...  
    public static void main(String arg[]) {  
        ...  
    }  
...  
}
```

© JMA 2001. All rights reserved

Archivos fuente compactos (v25)

- Los archivos fuente compactos son archivos de código fuente con atributos y métodos que no están encerrados en una declaración de clase. El archivo fuente compacto declara implícitamente una clase con el nombre del archivo, cuyos miembros son los atributos y métodos no encerrados.
- Los archivos fuente compacto deben estar en el paquete por defecto e importan automáticamente las clases e interfaces públicas de nivel superior de los paquetes exportados por el módulo java.base.
- Los archivos fuente compacto mejoran el protocolo mediante el cual se inician los programas Java permitiendo métodos de instancia main que no son static, no necesitan ser public y no necesitan tener un parámetro String[].

App.java

```
String greeting = "Hello, World!";  
void main() {  
    saluda(greeting);  
}  
void saluda(String msg) {  
    IO.println(msg);  
}
```

© JMA 2001. All rights reserved

Conceptos O-O

- Objetos: Clases e instancias
- Miembros: Campos (atributos) y métodos
- Constructores y destructores
- Encapsulación, reutilización y herencia
- Polimorfismo e interfaces
- Sobrecarga, reemplazo y sombreado
- Clases abstractas
- Miembros de clase e instancia

© JMA 2001. All rights reserved

Novedades Java 8

- Gestión de excepciones.
 - Captura de múltiples tipos de excepción
 - Try-con-recursos
 - Tratamiento de nulos, Optional
- Interfaces
 - Métodos por defecto y estáticos en
 - Reglas de herencia
- Programación funcional
 - Patrón delegate e interfaces funcionales
 - Expresiones lambda
 - Tipos predefinidos de Lambda
- Streams
 - Definición y obtención de flujos
 - Filtrado
 - Mapeado
 - Reducción
 - Paralelismo
- Trabajar con fechas: java.time

© JMA 2001. All rights reserved

Novedades Java 11

- Nuevo sistema de licenciamiento (BCL)
- jshell (REPL: Read-Eval-Print Loop) (v9)
- Métodos privados en interfaces (v9)
- Nuevos métodos en las colecciones, Streams, Optional (v9)
- Nuevos métodos de cadenas (Strings): `isBlank()`, `lines()`, `repeat()`, `strip()`, `stripLeading()` y `stripTrailing()`.
- Paquetes y Módulos (v9)
- Inferencia de tipos (v10)
- Reactive Streams (Programación Reactiva) (v11)
- HTTP Client API (v11)
- java puede ejecutar single-file source programs (sin compilación explícita) (JEP 330).
- Eliminación/remoción de varias APIs obsoletas (por ejemplo, Java EE/ CORBA modules removidos del JDK).

© JMA 2001. All rights reserved

Novedades Java 17

- Expresiones switch (v12)
- Helpful NullPointerExceptions (mejoras diagnostico): - XX:+ShowCodeDetailsInExceptionMessages (v14)
- Bloques de texto (v15)
- Creación de clases ocultas (para la reflexión) desde bytecode (v15)
- Coincidencia de patrones para instanceof (v16)
- Clases record (v16)
- Clases selladas (v17)
- Generadores de números pseudoaleatorios mejorados (v17)
- **Eliminación de Applet API:** Esta API ha sido declarada obsoleta y se elimina de Java.

© JMA 2001. All rights reserved

Novedades Java 21

- **jwebserver:** herramienta de línea de comandos para iniciar un servidor web estático mínimo.
- **Hilos virtuales** (Proyecto Loom)
- Colecciones secuenciales
- Coincidencia de patrones para desestructurar record
- Nuevas funciones:
 - Math.clamp(long value, int min, int max)
 - Character.isEmoji
 - String.indexOf(s, ini, fin);
 - StringBuilder y StringBuffer con .repeat(s, 3)
 - Aleatorizar una lista: Collections.shuffle
- **HttpClient** implementa autocerrar:
 - close(): cierra el cliente esperando a que todas las conexiones se hayan completado.
 - shutdown(): inicia un apagado suave del cliente cuya llamada termina de inmediato, pero por detrás espera a que se cierren todas las conexiones.
 - shutdownNow(): inicia el apagado en el momento, intentando interrumpir las conexiones actuales, aunque no hayan terminado, pero no espera tampoco a que se cierre el cliente para seguir la ejecución.
 - awaitTermination(Duration duration): espera a que se apague el cliente durante como máximo el tiempo indicado. Si al terminar la duración indicada se consiguió cerrar devuelve true, o false en caso contrario.
 - isTerminated(): devuelve true si el cliente está apagado.

© JMA 2001. All rights reserved

Novedades Java 25

- Pattern matching con deconstrucción de records en switch.
- Uso del patrón `_` para ignorar valores no utilizados. (v22)
- Constructores flexibles, instrucciones antes de `this()` o `super()`
- Archivos fuente compactos y métodos `main` de instancia sin clase obligatoria, más cercanos a otros lenguajes modernos.
- Module Import Declarations (JEP 511) para simplificar imports.
- Stream **Gatherers**: Operaciones intermedias personalizadas en los Stream (v24)
- Mejoras en Javadoc:
 - Comentarios de la documentación en la variante CommonMark de **Markdown** (v23)
 - Mejorada de la navegación estructural en la documentación de la API (v23)
 - Nueva página de resumen para especificaciones externas (v24)
 - `@hidden` oculta un tipo o un miembro en la documentación (v25)

© JMA 2001. All rights reserved

EL DOCUMENTADOR: JAVADOC

© JMA 2001. All rights reserved

Documentar clases

- El Java permite introducir comentarios que documenten las clases y aporta herramientas que convierten dichos comentarios en páginas HTML con la documentación de la clase.
- Deben documentarse los siguientes elementos: las clases, los interfaces, los atributos y los métodos.
- Se documentan los elementos precediéndolos por un comentario de tipo:

```
/**
 * Descripción del elemento
 * @etiquetas
 */
```
- Las descripciones de los elementos y de las etiquetas pueden ser formateadas con contenido HTML, etiquetas en línea y etiquetas de bloque. También se pueden usar secuencias de escape para representar caracteres que de otro modo serían inconvenientes o difíciles de representar.

© JMA 2001. All rights reserved

Comentarios en Markdown (v23)

- Se admite el uso de la variante CommonMark de Markdown en los comentarios de la documentación.
- Los comentarios de la documentación que contienen Markdown se componen de una serie de líneas consecutivas, cada una comenzando con un espacio en blanco opcional seguido de `///`. El espacio en blanco horizontal al principio y al final de cada línea de código Markdown puede ser significativo.

```
/// Esto es un ejemplo con __CommonMark__ de *Markdown* en Javadoc
///
/// ...
/// /** Hello World! */
/// public class HelloWorld {
///     public static void main(String... args) {
///         System.out.println("Hello World!"); // the traditional example
///     }
/// }
///
/// ...
///
/// @since 11
class App {
```

- Un comentario de documentación Markdown puede contener contenido Markdown (incluyendo HTML cuando sea necesario), etiquetas en línea y etiquetas de bloque.

© JMA 2001. All rights reserved

Etiquetas de javadoc

- **@author** Nombres de los autores
 - Solo clases.
- **@exception** NombreExcepción Explicación
 - Solo métodos. Indica cada excepción que genera y se explica por qué se produce y como tratarla.
- **@param** NombreParámetro Descripción
 - Solo métodos. Indica cada parámetro y para que se emplea.
- **@see** NombreClase#Miembro
 - Aparece bajo la etiqueta “Véase también” (See Also) los enlaces relacionados. El #Miembro es opcional.
- **@since** Descripción
 - Solo clases. Indica a partir de que versión del Java es operativa.
- **@return** Descripción
 - Solo métodos. Describe el tipo y valor de retorno del método.
- **@version** Número
 - Solo clases. Número de la versión de la clase.
- **{@snippet ...}** (v18)
 - Para encerrar un fragmento de texto, como código fuente o cualquier otra forma de texto estructurado.

© JMA 2001. All rights reserved

Standard Tags

- | | | |
|-------------------------------------|--------------------------------------|---|
| • <u>author</u> | • <u>literal</u> | • <u>spec</u> |
| • <u>code</u> | • <u>param</u> | • <u>summary</u> |
| • <u>deprecated</u> | • <u>provides</u> | • <u>systemProperty</u> |
| • <u>docRoot</u> | • <u>return</u> | • <u>throws</u> |
| • <u>exception</u> | • <u>see</u> | • <u>uses</u> |
| • <u>hidden</u> | • <u>serial</u> | • <u>value</u> |
| • <u>index</u> | • <u>serialData</u> | • <u>version</u> |
| • <u>inheritDoc</u> | • <u>serialField</u> | |
| • <u>link</u> | • <u>since</u> | |
| • <u>linkplain</u> | • <u>snippet</u> | |

© JMA 2001. All rights reserved

JSHELL (v9)

© JMA 2001. All rights reserved

Introducción

- La herramienta Java Shell (JShell) es una herramienta interactiva para aprender el lenguaje de programación Java y crear prototipos de código Java. JShell es un ciclo de lectura-evaluación-impresión (REPL), que evalúa declaraciones, instrucciones y expresiones a medida que se introducen y muestra inmediatamente los resultados. La herramienta se ejecuta desde la línea de comandos.
- El desarrollo de programas Java normalmente implica el siguiente proceso:
 - Escribir el programa completo.
 - Compilarlo y corregir los errores.
 - Ejecutar el programa.
 - Averiguar qué tiene de malo.
 - Editarlo.
 - Repetir el proceso.
- JShell, no reemplaza un IDE, ayuda a probar código y explorar opciones fácilmente a medida que se desarrolla un programa, probando declaraciones individuales, diferentes variaciones de un método o experimentar con API desconocidas dentro de la sesión de JShell.

© JMA 2001. All rights reserved

Fragmentos y comandos

- JShell acepta declaraciones de Java, definiciones de variables, métodos y clases, importaciones y expresiones, denominados fragmentos.
- Los fragmentos de código Java se introducen en JShell y se evalúan inmediatamente: muestra resultados, comentarios sobre los resultados, las acciones realizadas y los errores que ocurrieron.

```
jshell> String saluda(String nombre) {  
    ...>   return "Hola " + nombre;  
    ...> }  
| created method saluda(String)
```



```
jshell> saluda("Mundo");  
$2 ==> "Hola Mundo"
```
- Los comandos se distinguen de los fragmentos por una barra inclinada (/) y permite obtener información acerca de las variables actuales (/vars), métodos (/methods), tipos (/types), fragmentos (/list), ficheros (/open /save), ...
- Para la ayuda (/help /?), reiniciar el entorno (/reset) y salir (/exit)

© JMA 2001. All rights reserved

JWEBSERVER (v18)

© JMA 2001. All rights reserved

Introducción

- La herramienta jwebserver proporciona un servidor HTTP mínimo, diseñado para usarse para creación de prototipos, pruebas y depuración. Sirve una única jerarquía de directorios y solo sirve archivos estáticos. Sólo se admite HTTP/1.1, no tiene soporte para HTTPS ni HTTP/2.
- Solo se atienden solicitudes HEAD y GET idempotentes. Cualquier otra solicitud recibe una respuesta "501 - Not Implemented" o "405 - Not Allowed". Las solicitudes GET se asignan al directorio que se sirve, de la siguiente manera:
 - Si el recurso solicitado es un archivo, se sirve su contenido.
 - Si el recurso solicitado es un directorio que contiene un archivo de índice (index.html), se entrega el contenido del archivo de índice.
 - De lo contrario, se enumeran los nombres de todos los archivos y subdirectorios del directorio. Los enlaces simbólicos y los archivos ocultos no se enumeran ni se publican.
- Los tipos MIME se configuran automáticamente mediante la tabla incorporada. Por ejemplo, los archivos .html se entregan como text/html y los archivos .java se entregan como text/plain.

© JMA 2001. All rights reserved

Comando

- El siguiente comando inicia el servidor web simple:
\$ jwebserver
- De forma predeterminada, el servidor se ejecuta en primer plano, sobre el directorio actual y se vincula a la dirección de bucle invertido y al puerto 8000. Esto se puede cambiar con las opciones -b y -p. Por ejemplo, para vincular el servidor web simple a todas las interfaces en el puerto 8080 y el directorio /cursos:
\$ jwebserver -p 8080 -b 0.0.0.0 -d c:\cursos
- Una vez iniciado con éxito, el servidor se ejecuta hasta que se detiene. Para detener el servidor, se presiona Ctrl + C o se cierra la consola de comandos. En plataformas Unix, se puede detener enviándole una señal SIGINT.
- jwebserver se encuentra en el módulo jdk.httpserver y, alternativamente, se puede iniciar con java -m jdk.httpserver. Se basa en la implementación del servidor web en el paquete com.sun.net.httpserver. La clase com.sun.net.httpserver.SimpleFileServer proporciona una forma programática de recuperar el servidor y sus componentes para su reutilización y extensión.

© JMA 2001. All rights reserved

LENGUAJE

© JMA 2001. All rights reserved

Sintaxis

- Sensible a mayúsculas y minúsculas.
 - Sentencias separadas por punto y coma (;).
 - Formato libre en las sentencias, una sentencia puede utilizar varias líneas y una línea puede incluir varias sentencias.
 - Los espacios en blanco se compactan a uno solo.
 - Bloques marcados por llaves: { <sentencias> }
 - Comentarios:
 - // hasta el final de la línea
 - /* ... */ una o varias líneas
 - /** ... */ una o varias líneas de documentación
 - 59 palabras reservadas (todas en minúsculas).
-

© JMA 2001. All rights reserved

Separadores

- () para contener listas de parámetros en los métodos, expresiones de control de flujo, tipos de conversión y establecer precedencias en la expresiones.
- { } para definir bloques de código y agrupar elementos de tablas inicializadas automáticamente.
- [] para declarar y referencias tablas.
- ; separar instrucciones.
- , separar identificadores en su declaración y concatenar sentencias dentro del for.
- . Separar paquetes y clases, y clases/instancias de atributos/métodos.

© JMA 2001. All rights reserved

Identificadores

- Compuesto de letras, números, _, \$.
- Juego de caracteres UNICODE.
- NO pueden comenzar con números.
- NO pueden ser palabras reservadas ni el nombre de un valor booleano (true / false).
- NO debe tener el mismo nombre que otro elemento del mismo ámbito.
- Sin limite en el número de caracteres.
- El identificador _ no esta permitido. (v9)
 - Patrón _ para variables y parámetros sin nombre, obligatorios pero no utilizados. (v22)

© JMA 2001. All rights reserved

Notación recomendada

- Utilizar nombres descriptivos: sustantivos para variables y objetos, y verbos para los métodos.
- Uso de mayúsculas y minúsculas:
 - Métodos, variables e instancia (CamelCase):
 - nombre, nombreCompuesto
 - Clases e Interfaces (PascalCase):
 - Nombre, NombreCompuesto
 - Constantes:
 - NOMBRE, NOMBRE_COMPUESTO
- Uso de mnemotécnicos:
 - btnBoton, txtTexto, frmFormulario, ...

© JMA 2001. All rights reserved

Valores constantes y Literales

- Enteros
 - Base 10 dddd...
 - Base 8 Oddd
 - Base 16 0xdd
 - Binario 0b1001 (v7)
- Punto Flotante
 - estándar 3.14159
 - científica 6.022E23
- Separadores (v7)
 - Para facilitar la lectura de grandes numero, el _ como separador de dígitos (siempre entre dos dígitos)
1001_1100_0011_1010
- Booleanos
 - true / false
- Caracteres
 - Unicode - 16 bits
 - 'a', '\n', '\\' ...
 - octal: \ddd
 - hexadecimal: \xdddd
 - Unicode Emoji (v21)
<https://www.unicode.org/Public/15.0.0/ucd/emoji/emoji-data.txt>
- Cadenas
 - entrecomillas: "...."
 - Bloques de texto: """"
.....
"""" (v15)
- Sin referencia:
 - null

© JMA 2001. All rights reserved

Variables

- Lenguaje fuertemente tipado.
- Dos tipos según contenido:
 - Primitivos: Contienen un único valor de tipo base.
 - Referencias: Contienen las referencias a objetos o tablas.
- Dos tipos según su ámbito:
 - Miembros: Atributos (field) de una clase.
 - Locales: Definidos dentro de un bloque.

© JMA 2001. All rights reserved

Tipos Primitivos

- Enteros:
 - byte 8 bits -128 .. 127
 - short 16 bits -32.768 .. 32.765
 - int 32 bits -2.147.483.648 .. 2.147.483.647
 - long 64 bits -1019 .. 1019 (sufijo L para las constantes)
- Reales:
 - float 32 bits entre 6 y 7 decimales equivalentes
 - double 64 bits unos 15 decimales equivalentes
- boolean 1 byte true o false
 - tipo devuelto en operaciones de comparación
 - requerido en operaciones de control
- char 2 byte 1 carácter UNICODE
 - pueden operarse como enteros
- void
 - indica ausencia de tipo

© JMA 2001. All rights reserved

Declaración de variables

- Se declaran como:
[final] <tipo> <nombre> [= <valor inicial>];
- Pueden declararse varias a la vez del mismo tipo separando los nombres por comas.
- Una variable final solo puede asignarse una vez, habitualmente en la inicialización.
- A una variable local se le debe dar explícitamente un valor antes de ser utilizada, ya sea por inicialización o asignación.
- Se pueden declarar en cualquier punto, pero siempre antes de utilizarlas. Se recomienda definir las al principio del bloque.
- Las definidas en bloques interiores solapan a las de bloques superiores con el mismo nombre.
- Alcance: desde donde se definen hasta el final del bloque.

© JMA 2001. All rights reserved

Referencias

- Permiten direccionar instancias de los objetos.
- Al declarar una variable de tipo referencia, NO se crea una instancia del objeto.
- Contiene null hasta que se le asigna un objeto.
- Se declaran:
<Nombre Clase> <nombre> [= <valor inicial>];
<Nombre Clase> <nombre> [= new <Nombre Clase> (<lista de parámetros>)];

© JMA 2001. All rights reserved

Inferencia de tipos (v10)

- La inferencia de tipos permite al compilador obtener un tipo estático basándose en un valor inicial sin que sea necesario expresarlo de forma explícita.
- La inferencia de tipos para variables locales se declaran como:
var <nombre> = <valor inicial>;
- Cuando se infiere de un tipo de retorno hace innecesaria la importación explícita del tipo.
- El uso de var no está permitido en retornos, parámetros, propiedades, variables sin inicializar o inicializadas a null.
- Se permite en los parámetros de una expresión lambda asignada a un interfaz funcional. (v11)

© JMA 2001. All rights reserved

Variables sin nombre (v22)

- Se puede usar la palabra clave de subrayado (_) como nombre de una variable local, excepción o parámetro lambda en una declaración cuando no se necesita el valor de la declaración. Esto se denomina variable sin nombre, que representa una variable que se declara, pero no tiene un nombre utilizable.

```
try (var _ = new FileReader(path)) {  
    for (int _ : lines)  
        total++;  
    lines.find(_ -> total++)  
} catch (IOException _) {  
    System.err.println(path + " isn't valid");  
}
```
- Las variables sin nombre son útiles cuando la sintaxis obliga a declarar una variable que luego no se va a utilizar y así evitar las advertencias de variable declarada sin utilizar.

© JMA 2001. All rights reserved

Casting

- El Java es un lenguaje fuertemente tipado, por lo que hace comprobación de tipos antes de realizar cualquier operación.
- La conversión de un tipo de menor precisión a uno de mayor es automática.
- Se denomina casting a la operación que fuerza la conversión de un tipo a otro.
- Se realiza:
 - (`<nuevo tipo>`) `<expresión>`
- La conversión de tipos de mayor precisión a uno de menor implica la pérdida de las posiciones altas.

© JMA 2001. All rights reserved

Tablas (I)

- Las tablas son un tipo especial de objetos en Java.
- Los elementos pueden ser de cualquier tipo, pero todos los elementos de una tabla deben ser del mismo tipo.
- Se declaran (se crea una referencia):
 - `<Tipo> <nombreTabla> [] [= <Iniciación>];`
 - `<Tipo> [] <nombreTabla> [= <Iniciación>];`
- Se dimensionan:
 - `<nombreTabla> = new <Tipo> [<Dimensión>];`
 - `<nombreTabla> = {<lista de elementos>};`
 - La dimensión puede ser una expresión.
 - Los elementos de la lista separados por comas. El número de los mismos fijan la dimensión.

© JMA 2001. All rights reserved

Tablas (II)

- Los elementos se inicializan al valor por defecto de su tipo.
- Se referencian:
 - `<nombreTabla> [<Indice>]`
 - Rango de los índices: $0 \dots \text{<Dimensión>} - 1$
 - No se puede acceder a elementos fuera del rango.
- La dimensión de la tabla se obtiene dinámicamente con la propiedad:
 - `<nombreTabla>.length`
- La asignación de una tabla a otra implica un cambio de referencia no una copia.

© JMA 2001. All rights reserved

Tablas Multidimensionales

- Tablas bidimensionales:
 - Se pueden crear directamente:
`<Tipo> <nombreTabla> [] [] = new <Tipo> [<Dimensión 1>] [<Dimensión 2>];`
 - o por dimensión:
`<Tipo> <nombreTabla> [] [] = new <Tipo> [<Dimensión 1>] [];`
`<nombreTabla> [<Indice>] = new <Tipo> [<Dimensión 2>]`
 - Las segunda dimensión puede no coincidir en los diferentes elementos.
- Una tabla bidimensional se considera como una tabla de referencias a otras tablas, por lo que se pueden implementar tablas multidimensionales.
- Se recomienda no superar las cuatro dimensiones por la dificultad de su manejo conceptual.

© JMA 2001. All rights reserved

Operadores (I)

- Asignación
 - Simple: <Destino> = <Expresión>
- Aritméticos
 - Suma (+)
 - si el primer operando es una cadena se concatena.
 - Resta (-)
 - Producto (*)
 - División (/)
 - La división de enteros es un entero.
 - Resto (%)
 - Acumulativa:
 - += , -= , *= , /= , %=
- Operadores relacionales
 - > : mayor
 - >= : mayor o igual
 - < : menor
 - <= : menor o igual
 - == : igual
 - != : distinto
- Operadores lógicos (los operandos son booleanos)
 - && : AND Optima
 - || : OR Optima
 - ! : NOT
 - & : AND Completo
 - | : OR Completo

© JMA 2001. All rights reserved

Operadores (II)

- Binarios
 - >> : Desplazamiento Derecha
 - << : Desplazamiento Izquierda
 - >>> : Desplazamiento Derecha con signo
 - & : AND binario
 - | : OR binario
 - ^ : XOR binario
 - ~ : complemento
 - Acumulativos:
 - >>=
 - <<=
 - >>>=
 - &=
 - |=
 - ^=
 - ~=
- Incremento/decremento
 - ++<Variable>
 - Se incrementa en 1
 - Se consulta.
 - --< Variable >
 - Se decrementa en 1
 - Se consulta.
 - < Variable >++
 - Se consulta
 - Se incrementa en 1.
 - < Variable >--
 - Se consulta
 - Se decrementa en 1.

© JMA 2001. All rights reserved

Operadores (III)

- Operador condicional:
 - <Condición> ? <Expresión true> : <Expresión false>
- Operador instanceof:
 - informa si el operando pertenece a una determinada clase.
 - Coincidencia de patrones para if y case (v15):
 - referencia instanceof Clase nueva_referencia
- Operador new:
 - crea una instancia de la clase
 - ... new <NombreClase>([<Lista de Parámetros>])

© JMA 2001. All rights reserved

Prioridad de Operadores

- | | | |
|-----|--------------------|--|
| 1. | postfijos | [] . (params) expr++ expr-- |
| 2. | operadores unarios | ++expr --expr +expr -expr ~ ! |
| 3. | creación o cast | new (type)expr |
| 4. | multiplicativo | * / % |
| 5. | aditivo | + - |
| 6. | desplazamiento | << >> >>> |
| 7. | relacional | < > <= >= instanceof |
| 8. | igualdad | == != |
| 9. | bitwise | & |
| 10. | bitwise | ^ |
| 11. | bitwise | |
| 12. | lógico | && |
| 13. | lógico | |
| 14. | condicional | ? : |
| 15. | asignación | = += -= *= /= %= &= ^= = <<= >>= >>>= |

© JMA 2001. All rights reserved

Bifurcación simple

```
if(<Condición>)  
<Instrucción>; o { <Bloque de instrucciones> }  
[  
else  
<Instrucción>; o { <Bloque de instrucciones> }  
]
```

- Sin limite en los anidamientos.
- La parte else es opcional. Para evitar confusión es correcto crear bloques cuando existan anidamientos.

© JMA 2001. All rights reserved

Bifurcación Múltiple

```
switch(<Expresión>) {  
case <Valor 1> :  
    <Grupo de instrucciones>  
    break;  
case <Valor 2>, <Valor 3>: (v12)  
    <Grupo de instrucciones>  
    break;  
// ...  
default:  
    <Grupo de instrucciones>  
}
```

- <Expresión> debe ser char, byte, short, int, Character, Byte, Short o Integer (a partir de la v5 permite enumerados, de la v7 String y de la v17 tipos).
- Cada case solo puede expresar una única constante o, a partir de la v12, una lista separada por comas).

© JMA 2001. All rights reserved

Bifurcación Múltiple (v12)

- Nueva sintaxis alternativa (y recomendada):

```
switch(<Expresión>) {  
  case <Valores 1> -> <Instrucción>;  
  case <Valores 2> -> { <Bloque de instrucciones> }  
  // ...  
  default ->  
    <Instrucción>; o { <Bloque de instrucciones> }  
}
```

- Usa etiquetas *case ->* y *default ->*, para indicar que solo se ejecutará el código a la derecha si coincide con el caso.
- El código a la derecha -> puede ser una expresión, un bloque o una instrucción throw.
- NO REQUIERE BREAK;
- Cada case puede expresar un valor o una lista separada por comas.
- No se puede combinar la notación antigua con : y la nueva con ->

© JMA 2001. All rights reserved

Expresión switch (v12)

- Las expresiones switch se evalúan con un solo valor, siempre deben devolver valor (default salvo en enumeraciones) y se pueden usar en declaraciones. El valor de retorno se devuelve con la instrucción yield.

```
var rstl = switch(<Expresión>) {  
  case <Valor 1> [, ... ] :  
    <Grupo de instrucciones>  
    yield <Valor de la expresión>;  
  case <Valor 2> :  
    <Grupo de instrucciones>  
    yield <Valor de la expresión>;  
  // ...  
  default:  
    <Grupo de instrucciones>  
    yield <Valor por defecto>;  
}
```

- Si el cálculo del valor de retorno no necesita un bloque porque es el resultado de una expresión, se puede contraer a:
case MONDAY, FRIDAY, SUNDAY -> 6; // case MONDAY, FRIDAY, SUNDAY : yield 6;

© JMA 2001. All rights reserved

Coincidencia de patrones (v17, v21)

- La coincidencia de patrones implica probar si un objeto tiene una estructura particular y luego extraer a variables datos de ese objeto si hay una coincidencia.
if (shape instanceof Rectangle r) { (v17)
- La coincidencia de patrones permite los objetos en las expresiones de evaluación de los switches: (v21)
switch(obj) {
 case null -> // equivale a case Object obj ->
 case String s -> ...
 case Double d -> ...
- Un patrón protegido permite refinar un patrón con una expresión booleana. Un valor coincide con un patrón protegido si coincide con el tipo de patrón y la expresión booleana de la cláusula when se evalúa como verdadera.
case String s when s.length() > 10:

© JMA 2001. All rights reserved

Bucles

- Bucle Mientras
while(<Condición>)
 ; o <Instrucción>; o {<Bloque de instrucciones>}
 – Se ejecuta de 0 a n veces, mientras se cumpla la condición.
- Bucle Repetir
do {
 <Bloque de instrucciones>
} while(<Condición>;
 – Se ejecuta de 1 a n veces, mientras se cumpla la condición.

© JMA 2001. All rights reserved

Bucle Para

```
for(<Inicio>; <Permanencia>; <Iteración>)  
; o <Instrucción>; o {<Bloque de instrucciones>}
```

- Apartados:
 - <Inicio> : Se ejecutan antes de comenzar.
 - Lista de expresiones separadas por comas.
 - <Permanencia> : Expresión condicional, se ejecuta mientras se cumpla la condición.
 - <Iteración> : Se ejecutan en cada iteración.
 - Lista de expresiones separadas por comas.

© JMA 2001. All rights reserved

Bucle Para en iteradores

```
for(<Tipo> varDest : <Colección>)  
<Instrucción>; o {<Bloque de instrucciones>}
```

- Equivale a:

```
for (Iterator i = <Colección>.iterator(); i.hasNext(); ) {  
    <Tipo> varDest = (<Tipo>) i.next();  
    ...  
}
```
- El tipo de <Colección> debe ser un subtipo de Iterable o un tipo de matriz

© JMA 2001. All rights reserved

Control de Flujo

- **<NombreEtiqueta> :**
 - Define un punto de reentrada.
- **break [<NombreEtiqueta>;]**
 - Sale de la instrucción actual.
 - En caso de aparecer la etiqueta, salta a ella.
- **continue [<NombreEtiqueta>;]**
 - Fuerza la evaluación de la condición.
 - En caso de aparecer la etiqueta, salta a ella.
- **return [<Expresión>;]**
 - Termina un método y, opcionalmente, devuelve un valor.
- **yield <Expresión>; (v12)**
 - Termina una expresión case del operador switch y devuelve un valor.

© JMA 2001. All rights reserved

Condicional de Error

```
try {  
    <Bloque de instrucciones vigiladas>  
}  
catch(<TipoExcepción> <Referencia>) {  
    <Instrucciones de tratamiento de la excepción>  
}  
// Otros tratamientos de excepción ...  
finally {  
    <Sentencias que se ejecutan siempre>  
}  
  
throw <Excepción>
```

– Los catch y el finally son opcionales pero debe aparecer al menos un tratamiento de error aunque no haga nada.

© JMA 2001. All rights reserved

synchronized

synchronized(<Expresión>
<Instrucción>; o {<Bloque de instrucciones>}

- Una declaración synchronized adquiere un bloqueo de exclusión mutua en nombre del subproceso en ejecución, ejecuta un bloqueo y libera el bloqueo al terminar el bloque.
- No prosigue hasta que la acción de bloqueo se haya completado con éxito (espera). Ningún otro subproceso puede adquirir el bloqueo hasta que sea liberado, debe esperar.
- Solo un hilo a la vez puede mantener un bloqueo en un monitor. Cualquier otro hilo que intente bloquear ese monitor se bloquea hasta que pueda obtener un bloqueo en ese monitor.
- La sincronización es el mecanismo básico para la seguridad de subprocesos.

© JMA 2001. All rights reserved

TIPOS

© JMA 2001. All rights reserved

Definición de Clases

```
[<modificador>] class <Nombre> [<herencia>] [<interface>] {  
    <Cuerpo de la clase>  
}
```

- **Modificadores:**
 - **abstract** : clase abstracta no instanciable.
 - **public**: clase publica instanciable e importable, debe coincidir con el nombre del fichero (el fichero solo puede tener una clase publica a primer nivel).
 - **final**: clase sellada que no admite herencia.
 - **strictfp**: tratamiento estricto de coma flotante para double y float (compatibilidad heredada con IEEE-754 para la precisión)
 - Sin modificador es una clase interna del paquete.
- **Para clases internas:**
 - **static**: clase interna no miembro.
 - **protected**: clase miembro accesible por sus herederos.
 - **private**: clase miembro no accesible fuera de la clase.

© JMA 2001. All rights reserved

Miembros

- **Atributos**
 - Se declaran como las variables
 - Si no se asigna un valor inicial automáticamente se inicializan a 0 (cero) las numéricas, false las lógicas, "" o '\0' los caracteres y a null las referencias.
[<acceso>] [<modificador>] <tipo> <nombre> [= <valor inicial>];
- **Métodos**

```
[<acceso>] [<modificador>] <tipo> <nombre> ([<ListaDeParámetros>]) [<excepciones>] {  
    <Cuerpo del Método>  
}
```
- **Se invocan (mensaje):**

```
<nombreInstancia>.<nombre>  
<nombreInstancia>.<nombre> ([<Lista de Argumentos >])
```

© JMA 2001. All rights reserved

Paso de argumentos

- La lista de parámetros se define:
 <nombreMetodo>(<Tipo> <Nombre>[, <Tipo> <Nombre>, ...])
 - No es obligatorio definir parámetros, en cuyo caso aparecerán los paréntesis vacíos.
- La argumentos se separan por comas, y deben ser compatibles con los tipos definidos por los parámetros. Si el método no recibe parámetros aparecerán los paréntesis vacíos.
- Todos los argumentos se pasan por valor:
 - Los tipos primitivos no permiten modificar su valor.
 - Las referencias no pueden cambiar la instancia que referencian.
 - Se pueden utilizar métodos que cambien el contenido de las instancias.

© JMA 2001. All rights reserved

Varargs (v5)

- Lista de argumentos variables:
 - Solo un parámetro.
 - Deben ser el último de la lista.
 - Puede no recibir ningún argumento.

```
void Tradicional(int fijo1, int fijo2, int[] lista) {  
    //...  
}  
void Nueva(int fijo1, int fijo2, int... lista) {  
    //...  
    for(int i : lista)  
        //...  
}  
Tradicional(1, 2, new int[] { 3, 4, 5 });  
Nueva(1, 2, 3, 4, 5);
```

© JMA 2001. All rights reserved

Sobrecarga

- Se denomina sobrecarga a la existencia de varias versiones del mismo método dentro de la misma clase.
- Las versiones del método se diferencian por el número y/o tipo de parámetros. El valor de retorno no influye.
- El sistema determina qué método se está invocando por la lista de argumentos y sus tipos:
 1. Coinciden exactamente
 2. Promoviendo los tipos actuales a tipos superiores
- En muchos casos es necesario forzar la conversión de tipos mediante un casting.

© JMA 2001. All rights reserved

Control de acceso

- `private`
 - solo accesible desde la clase donde está definido.
- `protected`
 - accesible desde la clase donde está definido, **el resto de las clases del paquete** y por sus herederos en otros paquetes.
- `public`
 - accesible por todos.
- sin definir
 - accesible desde el resto del paquete.

© JMA 2001. All rights reserved

Modificador: static

- Indica que el atributo o el método es de clase.
- Una única copia del atributo para todas las instancias de la clase. Se crean cuando se carga la clase.
 - `<NombreClase>.<atributo>`
 - (desde fuera de la clase)
- Los métodos son comunes para la clase, solo pueden acceder a otros miembros de clase. Se invocan:
 - `<NombreClase>.<nombreMetodo> (<Param>)`

© JMA 2001. All rights reserved

Modificador: final

- Indica que el atributo es una constante o un atributo de solo lectura y no se puede modificar.
 - `[<acceso>] final <tipo> <NOMBRE> = <valor>;`
 - El valor puede ser el resultado de evaluar una expresión.
 - Se pueden inicializar en el constructor.
`public final int readOnly;`
 - Pueden ser de clase (constante):
`public static final int CONSTANTE = 10;`
 - Se invocan:
 - `<NombreClase>.CONSTANTE`
 - `<NombreInstancia>.readOnly`
- Los métodos finales no pueden ser redefinidos por las subclases.

© JMA 2001. All rights reserved

Modificador: transient

[<acceso>] transient <tipo> <NOMBRE> [= <valor>];

- Se aplica solamente a atributos.
- Especifica que un atributo es transitorio y no interviene en los procesos de serialización.
- Por defecto todos los atributos son persistentes.

© JMA 2001. All rights reserved

Modificador: volatile

[<acceso>] volatile <tipo> <NOMBRE> [= <valor>];

- Se aplica solamente a atributos.
- Especifica que un atributo es modificado de forma asíncrona (normalmente por procesos concurrentes).
- Obliga a la máquina a consultar su valor inmediatamente antes de ser usado.

© JMA 2001. All rights reserved

Modificador: synchronized

```
<modificador> synchronized <tipo> <nombre> ([<Lista de  
Parámetros>]) {  
    <Cuerpo del Método>  
}
```

- Se aplica solamente a métodos y en programación multihilo.
- Impide que más de un hilo pueda ejecutar el método a la vez.
- Evita los problemas de sobreescritura de atributos.

© JMA 2001. All rights reserved

Modificador: native

```
<modificador> native <tipo> <nombre> ([<Lista de Parámetros>]) ;
```

- Se aplica solamente a métodos.
- Indica que la implementación del método se ha realizado en código nativo y de forma externa.
- Los métodos nativos necesitan ser cargados en el inicializador de la clase. Para cargar un método se utiliza:
 - `System.load("...");`
 - `System.loadLibrary("...");`
- Solo se utilizarán métodos nativos cuando sea estrictamente necesario y la seguridad lo permita.

© JMA 2001. All rights reserved

Herencia

```
<Modificador> class <Nombre> extends <SuperClase> {  
    <Cuerpo de la clase>  
}
```

- La clase (subclase) hereda todos los atributos y métodos de su superclase y, opcionalmente, redefinirlos.
- No acepta herencia múltiple.
- No se puede heredar de una clase final.
- En caso de no definir herencia explícita, se asume que la clase hereda de la metaclassa Object.

© JMA 2001. All rights reserved

Clases Abstractas

```
<Modificador> abstract class <Nombre> [<herencia>] [<interface>] {  
    <Cuerpo de la clase>  
}
```

- Se definen para sintetizar conocimiento.
- Se utilizan en la herencia.
- No pueden ser instanciadas.
- Pueden definir métodos abstractos pero no son obligatorios.
 - <modificador> abstract <tipo> <nombre> ([<Lista de Parámetros>]) ;
 - No se implementa el método.
 - Solo pueden aparecer en clases abstractas.
 - Obliga a sus subclases (no abstractas) a implementar el método.
 - Permiten definir el interfaz de sus herederos.

© JMA 2001. All rights reserved

Sobreescritura de Métodos

- Una subclase puede redefinir los métodos heredados.
- El tipo de retorno, el nombre y la lista de parámetros debe ser iguales a los del método heredado. En caso contrario se denomina sobrecarga, y debe seguir las limitaciones de la misma.
- No puede lanzar excepciones no definidas para el método de la superclase, aunque puede restringir dicha lista.
- Los modificadores no pueden restringir el acceso definido en la superclase, pero si pueden ampliarlo.
- No se pueden sobrescribir métodos finales.
- Los métodos sobreescritos solapan a los métodos originales. Para acceder a los métodos originales es necesario hacer referencia a la superclase.

© JMA 2001. All rights reserved

Constructores

- Dirigen la creación e inicialización de las instancias de la clase.
- Son métodos que se denominan de la misma forma que la clase y no devuelven ningún tipo. Aceptan sobrecarga, pueden existir varias versiones con diferentes parámetros, dado que se invocan con el operador new en el proceso de instanciación.
- El compilador proporciona automáticamente un constructor predeterminado sin parámetros para cualquier clase sin constructores. Este constructor predeterminado se pierde al definir el primer constructor explícito y, si se desea recuperar, es necesario definirlo explícitamente.
- Un constructor puede invocar otro constructor de la misma clase (this()) o de su superclase (super()): obligatoriamente como primera instrucción. A partir de la **v25, se pueden agregar instrucciones** que no hagan referencia a la instancia que se está creando antes de una invocación explícita del constructor.
- Los constructores no se heredan como tales.
- Si se hereda de una clase, se invocara implícitamente a su constructor sin parámetros si no se invoca otro explícitamente, pero si no dispone del constructor sin parámetros es obligatorio tener constructores que empiecen invocando uno de los constructores con parámetros de la superclase.
- Aceptan modificadores de acceso. Si se define como private, solo podrá ser invocado por otro constructor de la misma clase o desde un método de clase.

© JMA 2001. All rights reserved

Inicializadores

- Dirigen la creación e inicialización de la clase (no de las instancias).
- Inicializan los atributos de clase, cargan los métodos nativos, permiten gestionar excepciones.
- Se invocan una sola vez para toda la clase.
- Se definen como métodos de clase sin nombre y sin tipo de retorno:

```
static {  
    <Cuerpo del inicializador>  
}
```

© JMA 2001. All rights reserved

Finalizadores

- Dirigen la destrucción de las instancias.
- Son invocados automáticamente por el recolector de la basura cuando va a destruir la instancia.
- Cierra, detiene y libera los recursos que utiliza.
- No es necesario que libere memoria, salvo la reservada por métodos nativos.
- Es un método de objeto, sin tipo de retorno, sin argumentos y se llama siempre `finalize`:

```
finalize() {  
    <Cuerpo del finalizador>  
    [super.finalize()]  
}
```

- *Obsoletos (v9): Sustituir por `AutoCloseable` o utilizar el `Cleaner API` (`java.lang.ref`)*

© JMA 2001. All rights reserved

Referencias especiales

- **this**
 - Referencia a la instancia con la que se está trabajando.
 - Solo es necesaria cuando en la implementación de los métodos de la clase es necesario hacer una referencia a la instancia del propio objeto.
 - Pueden usarse como pseudo nombre de los constructores de la clase (solo dentro del constructor)
- **super**
 - Cualificador de ámbito a la superclase.
 - Se emplea para invocar a los métodos originales redefinidos por la subclases y los atributos suplantados en las subclases.
 - Pueden usarse como pseudo nombre de los constructores de la superclase (solo dentro del constructor)

© JMA 2001. All rights reserved

Vida de un objeto

- Al crear la primera instancia de la clase o al utilizar el primer método o variable static se localiza la clase y se carga en memoria.
- Se ejecutan los inicializadores static (sólo una vez).
- Cada vez que se quiere crear una nueva instancia:
 - se comienza reservando la memoria necesaria
 - se da valor por defecto a las variables miembro de los tipos primitivos
 - se ejecutan los constructores
- Destrucción de un objeto
 - El sistema destruirá todas las instancias que no tengan referencias.
 - Para quitar una referencia a una instancia se asigna a la referencia:
 - otra instancia
 - el valor null
 - El objeto no se destruye inmediatamente, periódicamente el recolector de basura revisara la memoria para eliminar dichos objetos.

© JMA 2001. All rights reserved

Administrador de Memoria

- El sistema de forma automática ejecuta el recolector de basura (garbage collector) que elimina los objetos que no tienen referencias.
- Se puede sugerir que se ejecute:
`System.gc( );`
- o que ejecute todos los finalizadores pendientes:
`System.runFinalization( );`

© JMA 2001. All rights reserved

Clases Internas

- El lenguaje de programación Java permite definir una clase dentro de otra clase o de un método. Esta clase se llama clase anidada o interna.
- Las clases anidadas permiten:
 - Agrupar lógicamente las clases que solo se usan en un lugar: si una clase es útil solo para otra clase, entonces es lógico incrustarla en esa clase y mantener las dos juntas. Anidar tales "clases auxiliares" hace que su paquete sea más ágil.
 - Aumentar la encapsulación: las clases anidadas pueden acceder a los atributos y métodos de su clase contenedora o variables locales del método contenedor.
 - Producir un código más legible y fácil de mantener: Anidar clases pequeñas dentro de clases de nivel superior coloca el código más cerca de donde se usa y evita la proliferación de múltiples ficheros.

© JMA 2001. All rights reserved

Clases Internas

- Clases Internas static
 - Se declaran dentro de otra clase o interface a máximo nivel.
 - Pueden utilizar los miembros de clases de la clase contenedora.
 - Son accesibles como clases del paquete: contenedor.clase.
- Clases Internas miembro
 - Se declaran dentro de otra clase a máximo nivel.
 - Pueden utilizar los miembros de la clase contenedora.
 - No pueden definir ningún miembro estático por sí mismas, antes de (v16).
 - Solo son accesibles a través de la propia clase contenedora.
- Clases Internas locales
 - Se declaran dentro de un bloque.
 - No pueden definir ningún miembro estático por sí mismas.
 - Solo son accesibles el propio bloque donde están definidas.
- Clases anónimas

© JMA 2001. All rights reserved

Clases anónimas

- Una clase anónima es una expresión que define e instancia una clase: un solo uso.
- Su sintaxis es como la invocación de un constructor:
 - El operador new
 - El nombre de una interfaz para implementar o una clase para extender.
 - Paréntesis que contienen los argumentos del constructor de la clase que extiende o vacíos si implementa una interfaz o no se requieren argumentos.
 - El cuerpo de la clase como en el resto de las clases salvo que no pueden tener miembros de clase (salvo constantes) ni constructores.

© JMA 2001. All rights reserved

JavaBean

- Los JavaBeans son un convenio para crear componentes de software reutilizables que se puedan manipular visualmente en una herramienta de construcción (IDE).
- Los componentes cuentan con propiedades (personalizan el aspecto y el comportamiento), eventos (notifica lo que está ocurriendo) y métodos.
- Un JavaBean:
 - Debe tener un constructor sin argumentos.
 - Sus atributos deben ser privados.
 - Sus propiedades tienen nombre y deben ser accesibles mediante métodos prefijados por get (is en los booleanos), función que devuelve su valor, y set, procedimiento que establece su valor.
 - Sus eventos tienen nombre con el sufijo Listener y deben tener los métodos prefijados por add y remove para establecer o retirar los delegados que actúan como controladores de eventos.
 - Debe ser serializables (implementar java.io.Serializable).

© JMA 2001. All rights reserved

Interfaces

```
[public] interface <Nombre> [extends <ListaDeSuperInterfaces>] {  
    [<Declaración de constantes>]  
    [<Tipos anidados>]  
    <Métodos del Interfaz>  
    <Métodos default> (v8)  
    <Métodos static> (v8)  
    <Métodos private> (v9)  
    <Métodos private static> (v9)  
}
```

- Aceptan herencia múltiple, la lista debe ir separada por comas.
- Por defecto el acceso al interface es de paquete, con el modificador public permite acceso a todos, en cuyo caso el nombre del interface debe coincidir con el nombre del fichero.
- Se pueden crear referencias del tipo del interfaz pero no instancias.
- Los métodos del interface son siempre public y abstract de modo implícito.
- Los métodos NO se implementan en el cuerpo del interface, solo se declaran.

© JMA 2001. All rights reserved

Implementación de Interfaces

```
[<modificador>] class <Nombre> implements <ListaDeInterfaces> {  
    <Cuerpo de la clase>  
}
```

- Una clase puede implementar varios interfaces, en cuyo caso los nombres irán separados por comas.
- Las subclase heredan los interfaces de las superclases.
- Las clases están obligadas a implementar todos los métodos de sus interfaces, con excepción de las clases abstractas que podrán implementarlo parcialmente, siendo responsabilidad de sus subclases completarlos.

© JMA 2001. All rights reserved

Métodos por defecto y estáticos en interfaces (v8)

- Ahora se puede añadir una implementación por defecto a los métodos de los interfaces:

```
public interface Persona {  
    String getNombre();  
    String getApellidos();  
    default String toStringDatos() { return getNombre() + " " + getApellidos(); }  
}
```

- Definiendo métodos estáticos en las interfaces se evitar tener que crear clases de utilidad.

```
public interface Calc {  
    int add(int a, int b);  
    static int multiply(int a, int b) { return a * b; }  
}
```

- Se pueden crear métodos privados para refactorizar el código. (v9)

© JMA 2001. All rights reserved

Tipos Genéricos o parametrizados

- Java Generics es un mecanismo introducido en Java 1.5 para “especializar” las clases “genéricas”, similares a las plantillas en lenguaje C++
- Una clase es genérica declara una o más variables de tipos que se conocen como parámetros de tipo de la clase. Gracias a los genéricos podemos postergar la resolución de tipos variables al momento de la instanciación.
- Los genéricos se compilan a tipos internos específicos, en tiempo de ejecución la información de tipo especificada en la creación de instancias no está disponible (borrado de tipos).
- Los parámetros de tipo pueden sustituir a los tipos reales en la declaración de atributos, parámetros, tipos de retorno y variables. No se pueden emplear en los miembros de clase.
- Los parámetros de tipo se declaran con paréntesis angulares a continuación del nombre de la clase.

© JMA 2001. All rights reserved

Tipos Genéricos o parametrizados

- Por convención, los nombres de los parámetros de tipo son letras mayúsculas únicas:
 - T – Type
 - E – Elemento (Usado extensivamente en las colecciones en java)
 - K – Key
 - V – Value
 - N – Number
 - S, U, V etc. – 2nd, 3rd, 4th types
- Los parámetros de tipo pueden ser resueltos a cualquier tipo referencia, no a tipos valor (pero si sus tipos envolventes).
- Se puede limitar la resolución de los parámetros de tipo a tipos que hereden o implemente determinadas clases o interfaces, por lo que se podrá acceder a sus miembros:
 - <T extends MySuper>
 - <T extends MyInterface>
 - <T extends MySuper & MyInterface & ...>

© JMA 2001. All rights reserved

Tipos Genéricos o parametrizados

```
Modificador class Nombre<T, ...> ... {  
    // T stands for "Type"  
    private T t;  
  
    public void procedure(T t) {  
        ...  
    }  
    public T fuction() {  
        ...  
        return t;  
    }  
    ...  
}
```

© JMA 2001. All rights reserved

Tipos Genéricos o parametrizados

- Los parámetros de tipo se resuelven por los tipos reales cuando se declaran atributos, parámetros, tipos de retorno y variables.
 MiGenerico<String> var1 = new MiGenerico("uno");
 List<MiGenerico<Integer>> lista;
- Los argumentos de los parámetros de tipo pueden ser comodines que son útiles en situaciones en las que solo se requiere un conocimiento parcial del parámetro de tipo.
- El comodín se representa por el símbolo ? y pueden tener límites explícitos:
 - <? extends MySuper> debe ser del tipo o uno de sus herederos
 - <? super MyClass> debe ser del tipo o uno de sus antecesoros List<? extends Entity> lista;

© JMA 2001. All rights reserved

Métodos Genéricos

```
public <U> void method(U u) { ... }
```

- El tipo U se infiere del parámetro pasado.

- Si no puede inferir el tipo del parámetro hay que pasarlo específicamente:

```
public <U> U method(Class tipo) { ... }
```

- Los constructores también aceptan genéricos:

```
public <T> MyClass(T t) {  
    // ...  
}
```

- Se puede limitar con:

```
public <U extends MySuper> void method(U u) { ... }
```

© JMA 2001. All rights reserved

Borrado de tipos

- Para implementar genéricos, el compilador de Java aplica el borrado de tipos:
 - Borra todos los parámetros de tipo y reemplaza cada uno con su primer límite si el parámetro de tipo está delimitado o por Object si el parámetro de tipo es ilimitado.
 - También borra los parámetros de tipo en los argumentos de métodos genéricos.
 - Puede crear un método sintético, llamado método puente, como parte del proceso de borrado de tipos para solucionar problemas de sobrecarga y preservar el polimorfismo en tipos genéricos extendidos.
 - Inserta type casts si es necesario para preservar la seguridad del tipo.
- El borrado de tipos asegura que no se creen nuevas clases para tipos parametrizados; en consecuencia, los genéricos no incurrir en gastos generales de ejecución. El bytecode producido, por lo tanto, solo contiene clases, interfaces y métodos ordinarios.

© JMA 2001. All rights reserved

Restricciones sobre genéricos

- No se pueden crear instancias de tipos genéricos con tipos primitivos.
- No se pueden crear matrices de tipos parametrizados.
- No se pueden consultar el tipo de una referencia con instanceof con tipos parametrizados concretos, solo con comodines.
- No se puede utilizar el operador new en parámetros de tipo para crear una instancia de su tipo.
- No se pueden utilizar los parámetros de tipo en los miembros de clase.
- Los tipos parametrizados no pueden ser Throwable, o herederos, por lo que no se pueden usar para crear, atrapar o lanzar excepciones.
- No permiten resolver la sobrecarga de métodos.
- Presentan múltiples limitaciones con la reflexión.

© JMA 2001. All rights reserved

Inferencia de tipos genéricos (v7)

- Inferencia de tipos para la creación de instancia genérico:
 - Se puede sustituir los argumentos de tipo necesarios para invocar el constructor de una clase genérica con los paréntesis angulados de tipo vacíos (<>), informalmente llamados operador diamante, porque el compilador puede inferir los argumentos de tipo del contexto.
- La expresión:

```
Map<String, List<String>> myMap = new HashMap<String,  
List<String>>();
```
- Equivale a:

```
Map<String, List<String>> myMap = new HashMap<>();
```

© JMA 2001. All rights reserved

Interfaces funcionales (v8)

- Cada interfaz funcional tiene un único método abstracto (SAM), llamado método funcional, al cual se hace coincidir o adaptar el los parámetros de la expresión lambda y el tipo de retorno.
- Puede contener métodos adicionales si son default, static o private (v9).
- Para definir una interfaz funcional se puede usar la anotación `@FunctionalInterface` y pueden representarse con una expresión lambda.
- Algunos ejemplos de interfaces funcionales son `Runnable`, `ActionListener`, `Comparator` y `Callable`.

```
@FunctionalInterface
public interface ICalculadoraLambda {
    public int operacion (int x,int y);
}
```

© JMA 2001. All rights reserved

Expresiones lambda (v8)

- Importadas de la programación dinámica del paradigma funcional.
- Facilitan la implementación del patrón Delegate. Es el compilador el encargado de la creación de las clases anónimas.

```
public interface ICompararLambda {
    public boolean compara(String a, String b);
}

String extrae(String[] lista, ICompararLambda operar) { ... }

String[] lst = {"hola", "adios"};
String s = extrae(lst, new ICompararLambda() {
    @Override
    public boolean compara(String a, String b) {
        return a.compareTo(b) > 0;
    }
});
s = extrae(lst, (a, b) -> a.compareTo(b) > 0);
```

© JMA 2001. All rights reserved

Expresiones lambda (v8)

- Sintaxis:
 - p -> exp
 - () -> exp
 - p -> { ...; return rslt; }
 - (p1, p2) -> { ... }
- Los nombres de los parámetros son los que se utilizarán en la expresión o implementación del método.
- Si solo tiene un parámetro los paréntesis son opcionales.
- Si no tiene parámetros los paréntesis vacíos son obligatorios.
- El cuerpo puede ser una expresión, en cuyo caso la {} son opcionales y, en caso de ser una función, tiene un return implícito del valor.
- Si el cuerpo está compuesto de instrucciones deben ir entre {} como bloque, terminadas en ; y, en caso de ser una función, tiene que hacerse un return explícito del resultado.

© JMA 2001. All rights reserved

Referencias de método (v8)

- Se pueden utilizar métodos existentes como expresiones lambda.
- Requieren el selector :: para evitar la invocación directa del método:
 - Referencia a un método estático
ContainingClass::staticMethodName
 - Referencia a un constructor
ClassName::new
 - Referencia a un método de instancia de un objeto determinado
containingObject::instanceMethodName
 - Referencia a un método de instancia de un objeto arbitrario de un tipo particular (la instancia de la clase se pasa como primer argumento cuando se llama al método).
ContainingType::methodName

© JMA 2001. All rights reserved

java.util.function (v8)

<code>Consumer<T></code>	Representa una operación que acepta un único argumento de entrada y no devuelve ningún resultado.
<code>Supplier<T></code>	Representa un proveedor de resultados (sin argumentos).
<code>Function<T,R></code>	Representa una función que acepta un argumento y produce un resultado.
<code>Predicate<T></code>	Representa un predicado (función con valor booleano) de un argumento.
<code>UnaryOperator<T></code>	Representa una operación en un solo operando que produce un resultado del mismo tipo que su operando.
<code>BinaryOperator<T></code>	Representa una operación sobre dos operandos (argumentos) del mismo tipo, produciendo un resultado del mismo tipo que los operandos.
<code>BiConsumer<T,U></code>	Representa una operación que acepta dos argumentos de entrada y no devuelve ningún resultado.
<code>BiFunction<T,U,R></code>	Representa una función que acepta dos argumentos y produce un resultado.
<code>BiPredicate<T,U></code>	Representa un predicado (función con valor booleano) de dos argumentos.

Especializaciones:

`BooleanSupplier, DoubleBinaryOperator, DoubleConsumer, DoubleFunction<R>, DoublePredicate, DoubleSupplier, DoubleToIntFunction, DoubleToLongFunction, DoubleUnaryOperator, IntBinaryOperator, IntConsumer, IntFunction<R>, IntPredicate, IntSupplier, IntToDoubleFunction, IntToLongFunction, IntUnaryOperator, LongBinaryOperator, LongConsumer, LongFunction<R>, LongPredicate, LongSupplier, LongToDoubleFunction, LongToIntFunction, LongUnaryOperator, ObjDoubleConsumer<T>, ObjIntConsumer<T>, ObjLongConsumer<T>, ToDoubleBiFunction<T,U>, ToDoubleFunction<T>, ToIntBiFunction<T,U>, ToIntFunction<T>, ToLongBiFunction<T,U>, ToLongFunction<T>`

© JMA 2001. All rights reserved

Enumerados (v5)

- Tipo que restringe los valores permitidos a un conjunto de constantes predefinidas con nombres. Por convención, los nombres de valores deber ir en mayúsculas.
- Los enumerados son un tipo restringido de clase en Java que define un pequeño conjunto de instancias de clase con nombre (constantes), son finales y extienden implícitamente a `java.lang.Enum`, y pueden declarar atributos, métodos y constructores.
- Los nombres de valores (nombres de las instancias) pueden tener asociados valores primitivos (parámetros del constructor).

```
public enum Genero { DESCONOCIDO, MASCULINO, FEMENINO, NEUTRO }  
public enum Day {  
    SUNDAY(1), MONDAY(2), TUESDAY(3), WEDNESDAY(4),  
    THURSDAY(5), FRIDAY(6), SATURDAY(7);  
    private int numericValue;  
    Day(int value) { numericValue = value; }  
    public int getValue() { return numericValue; }  
    public static Day getEnum(int value) { // switch value to enum    }}
```

© JMA 2001. All rights reserved

Clases registro (v16)

- Las clases de registro, son un tipo especial de clase, que ayudan a modelar agregados de datos simples e inmutables con una sintaxis más concisa que las clases normales.
- Una declaración de registro especifica en un encabezado una descripción de su contenido; el constructor y los métodos descriptores de acceso, equals, hashCode y toString se crean automáticamente. Los campos de un registro son inmutables porque la clase está destinada a servir como un simple "soporte de datos".
`record Rectangle(double length, double width) { }`
- Una clase de registro es implícitamente estática final, por lo que no puedes extender explícitamente una clase de registro.
- La coincidencia de patrones permite desestructurar las instancias de la clase registro: (v21)
`if(obj instanceof Point(int x, int y)) return x + y;`

© JMA 2001. All rights reserved

Clases registro (v16)

- Una clase de registro declara automáticamente los siguientes miembros :
 - Para cada componente del encabezado, los siguientes dos miembros:
 - Un campo/atributo private final con el mismo nombre y tipo declarado que el componente de registro. Este campo a veces se denomina campo de componente.
 - Un método de acceso public con el mismo nombre y tipo del campo componente (getter pero sin prefijo get).
 - Un constructor canónico cuya firma es la misma que la del encabezado. Este constructor asigna cada argumento de la expresión new, que instancia la clase de registro, al campo de componente correspondiente. Acepta parámetros de tipo (genéricos)
 - Implementaciones de los métodos equals y hashCode, que especifican que dos clases de registros son iguales si son del mismo tipo y sus valores de campos de componentes son iguales.
 - Una implementación del método toString que incluye la representación en cadena de todos los componentes de la clase de registro, con sus nombres.
- Pueden implementar sus propios métodos.

© JMA 2001. All rights reserved

Clases e interfaces selladas (v17)

- Las clases e interfaces selladas restringen qué otras clases o interfaces pueden extenderlas o implementarlas.
`public sealed class Shape permits Circle, Square, Rectangle { ... }`
`public sealed interface Expr permits ConstantExpr, PlusExpr, TimesExpr, NegExpr { ... }`
- Las subclases permitidas tienen las siguientes restricciones:
 - Deben extender directamente la clase sellada.
 - Deben tener exactamente uno de los siguientes modificadores para describir cómo continúa el sellado iniciado por su superclase:
 - `final`: No se puede ampliar más
 - `sealed`: Solo se puede extender por sus subclases permitidas
 - `non-sealed`: Puede ampliarse con subclases desconocidas; una clase sellada no puede evitar que sus subclases permitidas hagan esto.
 - Deben estar en el mismo módulo que la clase sellada o en el mismo paquete. La clase sellada debe poder acceder a ellas en el momento de la compilación.

© JMA 2001. All rights reserved

Archivos fuente compactos (v25)

- Los archivos fuente compactos son archivos de código fuente con atributos y métodos que no están encerrados en una declaración de clase. El archivo fuente compacto declara implícitamente una clase con el nombre del archivo, cuyos miembros son los atributos y métodos no encerrados.
- Los archivos fuente compacto deben estar en el paquete por defecto e importan automáticamente las clases e interfaces públicas de nivel superior de los paquetes exportados por el módulo `java.base`.
- Los archivos fuente compacto mejoran el protocolo mediante el cual se inician los programas Java permitiendo métodos de instancia `main` que no son `static`, no necesitan ser `public` y no necesitan tener un parámetro `String[]`.

App.java

```
String greeting = "Hello, World!";
void main() {
    saluda(greeting);
}
void saluda(String msg) {
    IO.println(msg);
}
```

© JMA 2001. All rights reserved

TRATAMIENTO DE ERRORES Y LOGS

© JMA 2001. All rights reserved

Excepciones

- Las excepciones son situaciones anómalas que impiden la correcta ejecución del código y, en caso de no tratarlas, provocan la detención (ruptura) del programa.
- Dos tipos de tratamiento:
 - Pesimista: Antes de que se puedan producir se investigan todas las posibles causas de error.
 - Optimista: Después de que se produzcan se trata el error (el tratamiento de excepciones es costoso).
- Las excepciones derivan de la clase Throwable:
 - Error: Errores irreversibles de compilación, del sistema o de la JVM.
 - Exception: Excepciones recuperables:
 - No Comprobadas o unchecked (RuntimeException y herederas): muy frecuentes, normalmente por errores de programación detectables en pruebas, no necesitan declararse en la cláusula throws.
 - Comprobadas, checked o declarativas: El resto de las excepciones, Java obliga a interceptarlas con try catch o a propagarlas con la declaración throws, sino se producirá un error sintáctico

© JMA 2001. All rights reserved

Métodos de las Excepciones

- La causa de la excepción se identifica por la clase de la instancia generada en la situación anómala.
- Todas las excepciones derivan de la clase Throwable, por lo tanto cuentan con una serie de métodos comunes:
 - String getMessage()
 - Extrae el mensaje asociado con la excepción.
 - String toString()
 - Devuelve una cadena que describe la excepción.
 - void printStackTrace([PrintStream stream])
 - Imprime la traza de métodos donde se produjo la excepción.
- Todas las excepciones cuentan con un constructor que acepta una cadena con la descripción de la excepción.

© JMA 2001. All rights reserved

Tratamiento de las Excepciones

- try
 - Bloque de instrucciones vigiladas.
- catch(<TipoExcepción> <Instancia de la Excepción>)
 - Captura un tipo de excepción y realiza el tratamiento de la misma. En algunos casos pueden no tener tratamiento.
 - Pueden existir múltiples cláusulas catch aunque son opcionales.
- finally
 - Sentencias que se ejecutan siempre, aunque no se produzca una excepción.
 - Es opcional y única.
- throw <Instancia de la excepción>;
 - Lanza una excepción.
- throw new <TipoExcepción>("...Mensaje de la Excepción...");

© JMA 2001. All rights reserved

Captura de múltiples tipos de excepción (v7)

- Para tratamientos similares de diferentes tipos de excepciones:

```
catch (IOException ex) {  
    logger.log(ex);  
    throw ex;  
}  
catch (SQLException ex) {  
    logger.log(ex);  
    throw ex;  
}
```

- Se puede:

```
catch (IOException | SQLException ex) {  
    logger.log(ex);  
    throw ex; // excepción no tratada  
}
```

© JMA 2001. All rights reserved

Try-con-recursos (v7)

- En versiones anteriores:

```
try {  
    BufferedReader br = new BufferedReader(new FileReader(path));  
    try {  
        // ...  
    } catch (IOException e) { ... }  
    finally {  
        br.close();  
    }  
} catch (Exception e) {  
    // ...  
}
```

- Para cualquier objeto que implemente `java.lang.AutoCloseable`:
try (BufferedReader br = new BufferedReader(new FileReader(path))) {
 // ...
} catch (IOException e) { ... }

© JMA 2001. All rights reserved

Notificar Excepciones

- Es obligatorio tratar las excepciones comprobadas.
- No es necesario tratarlas en el momento de producirse, es posible relanzarlas para que sean tratadas a un nivel superior.

```
[...] <tipo> <nombreMétodo> ([<ListaDeParámetros>]) throws <ListaDeExcepciones> {  
    <Cuerpo del Método>  
}
```

 - La lista de excepciones separada por comas.
 - Las excepciones que no aparezcan en la lista debes ser tratadas dentro del método.
 - Las excepciones relanzadas deben ser tratadas por el método superior o, a su vez, ser relanzadas.
- En caso de relanzar una excepción capturada con catch con una nueva excepción, es conveniente que envuelva a la excepción original (cause) para no perder las trazabilidad en la pila.

```
catch (SQLException ex) {  
    throw new UnsupportedOperationException(ex);  
}
```

© JMA 2001. All rights reserved

Excepciones propias

- El tratamiento de excepciones requiere la creación de excepciones propias.
- Existen varias formas de clasificar los errores: por origen (componente, módulo, API, ...), por tipo (dispositivos, red, código, ...), ...
- La recomendación es definir las en función a como se capturan en los bloques catch para dar el tratamiento. La mayoría de los sistemas disponen de una jerarquía de clases para derivar las nuevas excepciones y utilizan el sufijo Exception en su nombre.
- Al usar APIs de terceros siempre es conveniente envolver sus excepciones en excepciones propias (patrón facade) y definir un controlador por encima para procesar las cancelaciones.
- Las excepciones propias se crean heredando de la clase Exception, o una de sus subclases, si son declarativas o de RuntimeException si no lo son. Por convenio se utiliza el sufijo Exception en el nombre.
- Es conveniente crearlas con los constructores habituales de las excepciones que invoquen al constructor de la excepción base.

```
public class <NombreExcepción> extends Exception {  
    public <NombreExcepción>() { super(); }
```

© JMA 2001. All rights reserved

Tratamiento estricto de nulos

- Null representa la ausencia de valor o valor vacío. Los nulos son una fuente inagotable de errores y excepciones (billion dollar mistake).
- Un argumento nunca debería ser nulo: si el parámetro es opcional debería ser tratado como tal (sobrecarga, valores por defecto, ...). Para no abusar de la `NullPointerException`, en caso de recibir un null no esperado, se debería generar una excepción específica (como `IllegalArgumentException`) o utilizar aserciones.
- Siguiendo la misma línea, si una función devuelve un valor, siempre debe devolverlo o, en caso de no ser posible, generar una excepción. Nunca debería devolver null, es lo mismo que no devolver valor, pero obligando a comprobar el valor o incurrir en una `NullPointerException` sin contexto.
- El argumento JVM `-XX:+ShowCodeDetailsInExceptionMessages` activa mejoras de diagnóstico y describe el origen del nulo (v14).

© JMA 2001. All rights reserved

Optional<T> (v8)

- Es un contenedor de objetos que pueden ser nulos (la alternativa más segura).
 - .isPresent(): indica si la referencia es distinta de null.
 - .isEmpty(): indica si la referencia es null. (v11)
 - .get(): devolver el valor o una `NoSuchElementException` si no hay valor.
 - .orElseThrow(() -> ex): alternativa al get() con excepción personalizada.
 - .orElse(): devolver un valor predeterminado si el valor no está presente.
 - .ifPresent(): ejecutar un bloque de código si el valor está presente.
 - ...
- Se crean con los métodos de clase:
 - .of(): Devuelve un Optional con el valor no nulo especificado.
 - .empty(): Devuelve un Optional vacío.
 - .ofNullable(): Devuelve un Optional (.of) si del valor especificado no es null o un Optional vacío (.empty) si es null.

© JMA 2001. All rights reserved

Aserciones

- Las aserciones, como invariantes, permiten comprobar en cualquier parte del código de un método que se cumplen los invariantes establecidos.
- Una aserción tiene la siguiente sintaxis:
assert condicion;
assert condición : mensaje;
- donde la condición es una expresión lógica que debe cumplirse en el momento en que se evalúa la aserción.
- Si la condición no se cumple, el programa termina con un AssertionError.
- Tras el operador dos puntos, puede ponerse un String con el mensaje que se devuelve como error.
- Las aserciones pueden ser desactivadas una vez terminada la fase de depuración y pruebas.
 - Run Configurations > VM arguments: -ea

© JMA 2001. All rights reserved

Aserciones

- Aserciones como precondiciones
 - Como convenio, es preferible que las precondiciones de un método público en Java se comprueben mediante una condición y lancen la excepción IllegalArgumentException, IndexOutOfBoundsException, NullPointerException o la excepción apropiada de acuerdo con el error encontrado.
 - Sin embargo, en un método no público, sí se puede sustituir esa comprobación por una aserción que asegure que el método se utiliza de forma consistente en la clase.
- Aserciones como postcondiciones
 - En este caso sí se recomienda el uso de aserciones para la comprobación de las postcondiciones de un método, pues se supone que el método se encuentra bien implementado.
 - En este caso, el uso de postcondiciones permite asegurar que se cumplen en todas las ejecuciones que se hacen del método.

© JMA 2001. All rights reserved

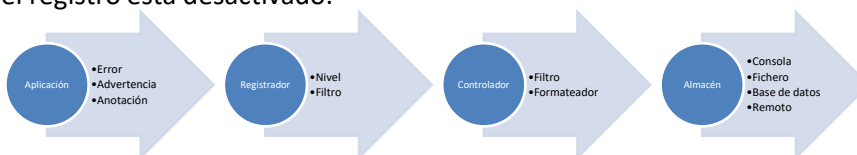
Registros de sucesos

- El registro es el proceso de escribir en un lugar central mensajes con sucesos, errores o eventos durante la ejecución de un programa. Este registro permite notificar y conservar mensajes de error y advertencia, así como mensajes de información (por ejemplo, estadísticas de tiempo de ejecución) para que se puedan recuperar y analizar posteriormente.
- El objeto que realiza el registro en las aplicaciones generalmente se llama Logger.
- Java define la API de Registro de Java. Esta API de registro permite configurar qué tipos de mensajes se escriben. Las clases individuales pueden usar este registrador para escribir mensajes en los archivos de registro configurados.
- El paquete `java.util.logging` proporciona las capacidades de registro a través de la clase `Logger`.

© JMA 2001. All rights reserved

Registro

- Las aplicaciones realizan llamadas de registro en objetos `Logger` (registradores). Estos objetos `Logger` asignan objetos `LogRecord` que se pasan a los objetos `Handler` (controladores) para su publicación.
- Tanto los registradores como los controladores pueden usar niveles de registro y (opcionalmente) filtros para decidir si están interesados en un registro de registro en particular .
- Cuando es necesario publicar un `LogRecord` externamente, un controlador puede (opcionalmente) usar un formateador para localizar y formatear el mensaje antes de publicarlo en un flujo de E / S.
- Las API está estructuradas de modo que las llamadas al `Logger` tengan un coste mínimo cuando el registro está desactivado.



© JMA 2001. All rights reserved

Niveles de registro

- Cada mensaje de registro tiene un nivel de registro asociado. El nivel ofrece una guía aproximada de la importancia y urgencia de un mensaje de registro. Los objetos de nivel de registro encapsulan un valor entero, y los valores más altos indican prioridades más altas.
- La clase Level define siete niveles de registro estándar:
 - SEVERE (más alta)
 - WARNING
 - INFO
 - CONFIG
 - FINE
 - FINER
 - FINEST (más leve)

© JMA 2001. All rights reserved

Controladores y Formateadores

- Java SE proporciona los siguientes controladores:
 - ConsoleHandler: un controlador simple para escribir registros formateados en System.err
 - StreamHandler: un controlador simple para escribir registros formateados en un OutputStream.
 - FileHandler: un controlador que escribe registros de registro formateados en un solo archivo o en un conjunto de archivos de registro rotativos.
 - SocketHandler: un controlador que escribe registros de registro formateados en puertos TCP remotos.
 - MemoryHandler: un controlador que almacena los registros de registro en la memoria.
- Java SE también incluye dos formateadores estándar:
 - SimpleFormatter : escribe breves resúmenes "legibles por humanos" de los registros.
 - XMLFormatter : escribe información detallada estructurada en XML.
- Se pueden desarrollar nuevos controladores y formateadores que requieran una funcionalidad específica con las abstracciones e interfaces del API.

© JMA 2001. All rights reserved

Configuración

- La configuración de registro se puede inicializar mediante un archivo de configuración de registro que se leerá al inicio (tiene el formato estándar `java.util.Properties`). Alternativamente, la configuración de registro se puede inicializar especificando una clase que se puede utilizar para leer las propiedades de inicialización de fuentes arbitrarias, como LDAP, JDBC, etc.
 - `java.util.logging.MemoryHandler.size=100`
- La configuración inicial puede:
 - Especificar niveles para registradores particulares. Estos niveles se aplican al registrador nombrado, o cualquier registrador debajo de él en la jerarquía de nombres, y se aplican en el orden en que están definidos en el archivo de configuración.
 - Contener propiedades arbitrarias para que las utilicen los controladores o los subsistemas que realizan el registro.
- La configuración predeterminada hace un uso limitado del espacio en disco, no inunda al usuario con información, pero se asegura de capturar siempre la información clave de fallas. Establece un solo controlador en el registrador raíz para enviar la salida a la consola.
- La configuración predeterminada se envía con JRE pero los ISV, los administradores del sistema y los usuarios finales pueden anularla.

© JMA 2001. All rights reserved

Realizar notificaciones

- Para crear un registrador asociado a una clase:
`private final static Logger LOGGER = Logger.getLogger( MyClass.class.getName());`
- Para establecer el nivel de registro:
`LOGGER.setLevel(Level.INFO);`
- Para escribir en el registro:
`LOGGER.severe("Es un error");`
`LOGGER.warning("Es un aviso");`
`LOGGER.info("Solo notifica");`
`LOGGER.finest("Carece de importancia");`
- Para configurar el registro:
`Logger.getLogger("").addHandler(new FileHandler("%t/my.log"));`
`FileHandler fileTxt = new FileHandler("Logging.txt");`
`fileTxt.setFormatter(new SimpleFormatter());`
`Logger.getLogger("").addHandler(fileTxt);`

© JMA 2001. All rights reserved

REFLEXIÓN

© JMA 2001. All rights reserved

Introducción

- La reflexión se basa en el concepto de que las clases (y el resto de tipos) son objetos y se puede obtener una instancia de la definición de tipo para interactuar con ella.
- La reflexión es comúnmente utilizada por programas que requieren la capacidad de examinar o modificar el comportamiento en tiempo de ejecución de las aplicaciones que se ejecutan en la máquina virtual Java. La reflexión es una técnica muy potente que permite realizar operaciones que de otro modo serían imposibles. La reflexión es una característica relativamente avanzada y solo debe ser utilizada cuando se tenga un conocimiento sólido de los fundamentos del lenguaje.
- La reflexión permite:
 - Codificar en tiempo de ejecución: guardar nombre en cadenas y utilizarlas para cargar dinámicamente clases por su nombre, instanciar las clases cargadas, invocar sus métodos por el nombre, ...
 - Explorar las clases para poder enumerar los miembros de las clases, extraer anotaciones, decidir que utilizar, ...
 - Poder examinar miembros privados de las clases para crear frameworks avanzados de pruebas, validaciones, serialización, mapeos, ...

© JMA 2001. All rights reserved

Inconvenientes

- La reflexión es poderosa, pero no debe usarse de forma indiscriminada. Es preferible evitar su uso si es posible realizar una operación sin usar la reflexión.
- Sobrecarga de rendimiento
Debido a que la reflexión involucra tipos que se resuelven dinámicamente, no se pueden realizar ciertas optimizaciones de máquinas virtuales Java. En consecuencia, las operaciones reflexivas tienen un rendimiento más lento que las alternativas no reflexivas y deben evitarse en secciones de código que se llaman con frecuencia en aplicaciones sensibles al rendimiento.
- Restricciones de seguridad
Reflection requiere un permiso en tiempo de ejecución que puede no estar presente cuando se ejecuta en contextos de seguridad restringido (Ej. Applet).
- Exposición de internos
Dado que la reflexión permite que el código realice operaciones que serían ilegales sin ella, como el acceso a miembros private, el uso de la reflexión puede provocar efectos secundarios inesperados y puede destruir la portabilidad. La reflexión rompe las abstracciones y, por lo tanto, puede cambiar el comportamiento con las actualizaciones de la plataforma.

© JMA 2001. All rights reserved

Objetos de clase

- Las instancias de tipo `java.lang.Class` encapsulan la definición de los tipos. No se pueden instanciar, es necesario utilizar uno de los siguientes mecanismos:
 - `Tipo.class`
 - `Class.forName(Cadena con el nombre calificado)`
 - `instancia.getClass()`
 - `envolvente.TYPE`
- Las instancias de `Class` disponen de métodos que devuelven clases:
 - `.getSuperclass()`: Devuelve la superclase para la clase dada.
 - `.getClasses()`: Devuelve todas las clases, interfaces y enumeraciones públicas que son miembros de la clase, incluidos los miembros heredados.
 - `.getDeclaredClasses()`: Devuelve todas las clases, interfaces y enumeraciones que se declaran explícitamente en la clase incluidos los privados.

© JMA 2001. All rights reserved

Obtención de Miembros

- Están disponibles métodos que devuelven todos los miembros de un tipo o que devuelven uno concreto.
- Así mismo, la versión por defecto solo obtiene los miembros públicos incluidos los heredados, la versión Declared solo obtiene los miembros declarado en la clase incluidos privados y protegidos.
 - `getFields()`, `getDeclaredFields()`, `getField()`, `getDeclaredField()`
 - `getMethods()`, `getDeclaredMethods()`, `getMethod()`, `getDeclaredMethod()`
 - `getConstructors()`, `getConstructor()`
 - `getAnnotations()`, `getDeclaredAnnotations()`, `getAnnotation()`, `getDeclaredAnnotation()`

© JMA 2001. All rights reserved

Acceder a Miembros

- Obtener y establecer valores de atributos:

```
Field cmd = instancia.getClass().getDeclaredField("atributo");
cmd.setAccessible(true);
String cad = (String)(cmd.get(instancia));
cmd.set(instancia, "nuevo valor");
```
- Obtener e invocar un método:

```
Method m = instancia.getClass().getDeclaredMethod("metodo");
// m.setAccessible(true);
Object o = m.invoke(instancia);
m.invoke(instancia, "parámetro");
```
- Crear nuevas instancias:

```
Class.newInstance() //solo puede invocar el constructor sin argumentos
Constructor ctor = Console.class.getDeclaredConstructor()[0];
ctor.newInstance()
```

© JMA 2001. All rights reserved

Restringir el accesos a miembros

- La invocación de `setAccessible(true)` indica que el objeto debe suprimir las comprobaciones del control de acceso del lenguaje Java cuando se utiliza la reflexión. Acceder a miembros privados fuera de la clase puede ser una amenaza para la seguridad de una aplicación Java.
- Se puede restringir usando Security Manager, aunque no se instala de forma predeterminada, por lo que todos los permisos están disponibles. Algunos lanzadores (como el entorno de subprogramas) instalarán un SecurityManager, que a su vez utiliza una política basada en archivos de políticas de forma predeterminada. Se puede cambiar en el archivo "java.security". Otros entornos, como un contenedor de servlets o un contenedor Java EE, también pueden instalar un SecurityManager.
- Se puede habilitar manualmente:
`System.setSecurityManager(new SecurityManager());`
- Una vez habilitado el SecurityManager, de forma predeterminada el código ya no puede llamar a "setAccessible".
- La seguridad se define en `${JRE_HOME}/conf/security/java.policy`. Para volver a habilitarlo:

```
grant {  
    permission java.lang.reflect.ReflectPermission "suppressAccessChecks";  
};
```

© JMA 2001. All rights reserved

Limitar la accesibilidad de los paquetes

- Los contenedores pueden ocultar el código de implementación agregando a la propiedad de seguridad "package.Access". Esta propiedad evita que las clases que no son de confianza de otros cargadores de clases se vinculen y utilicen la reflexión sobre la jerarquía de paquetes especificada.

```
private static final String PACKAGE_ACCESS_KEY = "package.access";  
static {  
    String packageAccess = java.security.Security.getProperty(  
        PACKAGE_ACCESS_KEY  
    );  
    java.security.Security.setProperty(PACKAGE_ACCESS_KEY, (  
        packageAccess == null || packageAccess.trim().isEmpty()) ? "" : (packageAccess + ",")  
        + "com.example.paquete.protegido"  
    );  
}
```

© JMA 2001. All rights reserved

ANOTACIONES

© JMA 2001. All rights reserved

Metadatos

- Las anotaciones son metadatos (o decoradores), con información adicional sobre el código que está escrito, destinado a las herramientas que manejan el código (que pueden utilizar esta información adicional si así lo desean) y sin significado en el propio código.
 - Aparecen en la versión 5.
 - Comienzan por @
 - Preceden inmediatamente al elemento que complementan (clase, interfaz, método, atributo, ...)
 - Pueden llevar parámetros
 - Pueden ser usadas por el propio código a través de la reflexión (reflection).
-

© JMA 2001. All rights reserved

Anotaciones predefinidas

- `java.lang`

- `@Deprecated` : Indica a los compiladores que adviertan cuando se utilizan que el elemento está en desuso o es obsoleto.
- `@Override` : Indica que el método está sobrescribiendo un método de la superclase y que debe existir en la misma.
- `@SuppressWarnings` : Indica a los compiladores que no muestren determinados avisos.
- `@SafeVarargs` : Indica a los compiladores que supriman las advertencias de comprobación de tipos en los argumentos variables con genéricos. (v9)
- `@FunctionalInterface` : Indica que debe cumplir con la especificación de interfaz funcional. (v8)

© JMA 2001. All rights reserved

Anotaciones predefinidas

- `java.lang.annotation`

- `@Documented` : Indica las anotaciones cuyo uso deben ser documentados por javadoc de forma predeterminada.
- `@Retention` : Indica cuánto tiempo se deben conservar las anotaciones con el tipo anotado: hasta que se compile (SOURCE), en el bytecode de clase (CLASS, por defecto) o en tiempo de ejecución (RUNTIME, para reflection)
- `@Target` : Indica el tipo de elemento en el que es aplicable la anotación: PACKAGE, TYPE, ANNOTATION_TYPE, CONSTRUCTOR, METHOD, PARAMETER, FIELD y LOCAL_VARIABLE. [v8: TYPE_PARAMETER (genéricos) y TYPE_USE (casting)].
- `@Inherited` : Indica que también se aplica a los herederos la anotación.
- `@Repeable` : Indica que se pueden adjuntar múltiples instancias de la anotación al destino de la anotación. (v8)

© JMA 2001. All rights reserved

Crear nuevas anotaciones

- Los tipos de anotación se definen con `@interface`. Los parámetros se definen de manera similar a los métodos de una interfaz regular:
 - Pueden ser valores únicos o matrices, de tipo `String`, `Class`, enumeraciones, tipos primitivos y tipos de anotaciones (anotaciones compuestas).
 - Son opcionales si tienen un valor por defecto.
 - Se asignan por su nombre (`nombre="valor"`), como valores únicos o lista de valores (`nombre={1,2,3}`). El primer parámetro se puede asignar sin nombre.

```
@Target({ElementType.FIELD, ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Inherited
public @interface MiAnotacion { int id(); String valor() default ""; }
```

- Para la comprobación de las anotaciones en tiempo de ejecución a través de la reflexión:

```
Annotation[] anotaciones = MyClass.class.getAnnotations();
int id = this.getClass().getAnnotation(MiAnotacion.class).id();
```

© JMA 2001. All rights reserved

PAQUETES Y MÓDULOS

© JMA 2001. All rights reserved

Paquetes

- Un paquete es una colección de clases e interfaces.
 - Agrupan una serie de clases relacionadas.
 - Resuelven los conflictos de nombre (nombre.paquete.NombreClase).
 - Controlan la accesibilidad.
- Definición de paquete:
 - Implícita: Todas la clases de un directorio.
 - Explícita: Utilizando en la primera línea no comentada del fichero de clase la cláusula:
 - `package <nombre_paquete>;`
- Se recomienda:
 - usar minúsculas en el nombre.
 - Comenzar con el dominio propio invertido: `com.miempresa.clasifica`

© JMA 2001. All rights reserved

Paquetes

- Importación de paquetes para calificar la referencia a tipos de otros paquetes:

```
import <paquete>.<clase>;
import <paquete>.*;
    • El * hace referencia a todas las clases del paquete.
import module java.base; (v25)

import java.lang.Math;
double r = Math.cos(Math.PI * theta);
```
- Importaciones estáticas: Permiten el acceso sin cualificar a miembros estáticos de otras clases como si fueran propios:

```
import static <paquete>.<clase>.<Método estático>;
import static <paquete>.<clase>.*;
    • El * hace referencia a todos los métodos de la clase.

import static java.lang.Math.PI;
import static java.lang.Math.*;
double r = cos(PI * theta);
```

© JMA 2001. All rights reserved

Administrar archivos

- Muchas implementaciones de la plataforma Java dependen de sistemas de archivos jerárquicos para administrar archivos fuente y de clase, aunque la especificación del lenguaje Java no lo requiere.
- El nombre calificado del miembro del paquete y el nombre de la ruta al archivo son paralelos (sustituyendo el punto por la barra apropiada). Un conjunto de directorios de puede comprimir en un fichero ZIP con extensión “.jar”.
- La ruta de clases se establece en la variable de sistema CLASSPATH y puede incluir varias rutas, separadas por un punto y coma (Windows) o dos puntos (UNIX). De forma predeterminada, el compilador y la JVM buscan las clases en el directorio actual, incluidos los contenidos de los ficheros JAR, y en la ruta de clases.

© JMA 2001. All rights reserved

Paquetes básicos

- java.lang
 - Objetos fundamentales del lenguaje, es el único que no necesita importación.
- java.util
 - Clases auxiliares que permiten el manejo de estructuras de datos.
- java.io
 - Clases de entrada/salida a ficheros
- java.applet
 - Clase base de los applets.
- java.awt, javax.swing
 - Contiene la mayoría de las clases para implementar el interface gráfico de usuario.

© JMA 2001. All rights reserved

Paquetes Avanzados

- **java.sql**
 - Proporciona las clases e interfaces para acceso a bases de datos con JDBC (Java Database Connectivity)
- **java.beans**
 - Soporte para el desarrollo de componentes
- **java.text**
 - Clases relacionadas con el formato de textos internacionales.
- **java.math**
 - Soporte avanzado de operaciones matemáticas en forma de métodos estadísticos.

© JMA 2001. All rights reserved

Paquetes de Red

- **java.net**
 - Soporte de red: TCPI/IP, HTTP y soporte Internet y WWW
- **java.rmi**
 - Soporte para objetos distribuidos (Invocación Remota de Métodos).
- **java.security**
 - Clases que permiten el establecimiento de comunicaciones seguras.
- **java.servlet**
 - Soporte para desarrollo de aplicaciones servidor.
- **java.util.mime**
 - Soporte para las extensiones de correo Internet multiproposito.

© JMA 2001. All rights reserved

Otras Ingenierías

- Java Foundation Classes (JFC) y Java 2D
 - Desarrollo de interfaces gráficos de usuarios (GUI), sustituye al AWT por componentes swing, permitiendo elegir el estilo, ampliando la funcionalidad y realizar operaciones de arrastrar y soltar. Incluye mejoras en el tratamiento de gráficos y textos en 2D.
- Java Media Frameworks (JMF)
 - Ingeniería que permite trabajar con elementos multimedia. Incorpora el Java 3D, que permite la manipulación de geometrías complejas en 3D.
- Java IDL
 - Permite la conectividad entre objetos distribuidos utilizando el estándar CORBA.
- Java Native Interface (JNI)
 - Es el interface de programación Java para ejecutar código nativo.

© JMA 2001. All rights reserved

Framework

- Es un conjunto de clases que definen un diseño abstracto para solucionar un conjunto de problemas relacionados.
- ¿Qué contiene un Framework?
 - Un conjunto de clases e interfaces
 - Modelo de uso de esas clases e interfaces
 - Modelo de funcionamiento del framework.

© JMA 2001. All rights reserved

Java Specification Request

- El Proceso de la Comunidad Java, o Java Community Process, establecido en 1998, es un proceso formalizado el cual permite a las partes interesadas a involucrarse en la definición de futuras versiones y características de la plataforma Java.
- El proceso JCP conlleva el uso de Java Specification Request (JSR), las cuales son documentos formales que describen las especificaciones y tecnologías propuestas para que sean añadidas a la plataforma Java. Las revisiones públicas formales de JSRs son controladas antes de que los JSR se conviertan en final y sean votados por el Comité Ejecutivo JCP.
- Un JSR final suministra una implementación de referencia la cual da una implementación libre de la tecnología en código fuente y un Kit de Compatibilidad de Tecnología para verificar la especificación de la API.

© JMA 2001. All rights reserved

Módulos (v9)

- Si un conjunto de paquetes es lo suficientemente cohesivo, entonces dichos paquetes pueden agruparse en un módulo.
- Un módulo categoriza algunos o todos sus paquetes como exportados, lo que significa que se puede acceder a sus tipos desde un código fuera del módulo.
- Un paquete no es exportado por un módulo, solo es accesible por el código dentro de su módulo.
- Si el código de un módulo desea acceder a los paquetes de otros módulos, entonces el módulo debe depender explícitamente de los otros módulos.
- Por lo tanto, un módulo controla cómo sus paquetes usan otros módulos (dependencias) y controla cómo otros módulos usan sus paquetes (exportaciones).

© JMA 2001. All rights reserved

Módulos (v9)

- Hay cuatro tipos de módulos en Java Platform Module System (JPMS):
 - Módulos del sistema: Estos son los módulos enumerados cuando ejecutamos el comando: `java --list-modules`. Incluyen los módulos Java SE (`java.`) y JDK (`jdk.`).
 - Módulos de aplicación: Estos módulos son los que normalmente queremos construir. Se nombran y definen en el archivo `module-info.java`, que se compila e incluye en el archivo JAR.
 - Módulos automáticos: El sistema de módulos crea por defecto un módulo automático para cada JAR sin descriptor de módulo.
 - Dado que no contiene información sobre la modularidad, exporta todos los paquetes como abiertos y que lean todos los demás módulos.
 - Módulo sin nombre: Cuando se carga una clase o JAR en la ruta de clases, pero no en la ruta del módulo, se agrega automáticamente al módulo sin nombre.
 - Es un único módulo general para mantener la compatibilidad con versiones anteriores sin módulos. Todas las clases dentro del módulo sin nombre pueden leer todos los demás módulos (con o sin nombre) sin ninguna declaración explícita de ningún tipo.
 - Los paquetes exportados por un módulo sin nombre solo pueden ser leídos por otro módulo sin nombre.
 - No es posible que un módulo con nombre pueda leer (requiera) el módulo sin nombre.

© JMA 2001. All rights reserved

Módulos (v9)

- Los objetivos principales de la modularidad de Java son: hacer que las implementaciones de la Plataforma sean más fácilmente escalables a dispositivos pequeños, mejorar la seguridad y el mantenimiento, permitir un mejor rendimiento de las aplicaciones y proporcionar a los desarrolladores mejores herramientas para la programación en general.
- La estructura modular del JDK a quedado como:
 - El módulo “`java.base`” contiene las API fundamentales de Java SE Platform, todos los módulos tienen una dependencia implícita y obligatoria a este módulo.
 - Los módulos estándar, cuyas especificaciones se rigen por el JCP, tienen nombres que comienzan con la cadena “`java.`”.
 - Todos los demás módulos son simplemente parte del JDK y tienen nombres que comienzan con la cadena “`jdk.`”.
- La estructura modular del JDK se puede visualizar como un grafo: cada módulo es un nodo y las flechas indican de quien dependen.

© JMA 2001. All rights reserved

The diagram illustrates the dependency structure of Java SE packages. The packages are arranged in a hierarchical manner, with 'java.se' at the top. It has numerous outgoing dependencies to other packages. For example, 'java.se' depends on 'java.xml.ws', 'java.xml.bind', 'java.activation', 'java.desktop', 'java.sql.rowset', 'java.security.jgss', 'java.naming', 'java.management', 'java.instrument', 'java.transaction', 'java.corba', 'java.se.ee', 'java.annotations.common', 'java.compiler', 'java.datatransfer', 'java.prefs', 'java.sql', 'java.xml.crypto', 'java.xml', 'java.logging', 'java.scripting', 'java.rmi', 'java.security.sasl', and 'java.base'. The graph shows a complex web of dependencies, with many packages having multiple incoming and outgoing arrows. The final destination for many of these dependencies is 'java.base' at the bottom.

Módulos (v9)

- Un módulo de la plataforma Java consta de:
 - Una colección de paquetes
 - Opcionalmente, archivos de recursos y otros archivos como bibliotecas nativas
 - Una lista de los paquetes accesibles en el módulo.
 - Una lista de todos los módulos de los que depende este módulo, en caso ser necesario.
 - Una lista de servicios que provee o consume, en caso ser necesario.
- Al crear un módulo de aplicación, es importante que su nombre sea único. Se espera que los nombres de los módulos sigan la convención del "nombre de dominio inverso", al igual que los nombres de los paquetes.

86

Módulos (v9)

- El archivo especial `module-info.java` es el descriptor del módulo y debe estar en el directorio raíz de los paquetes de clases de Java.
- Todos los módulos estándar de Java SE tienen una dependencia implícita y obligatoria de `java.base`. No es necesario definir la dependencia `java.base` explícitamente.
- No se permite tener dependencias circulares entre módulos. Si el módulo A requiere al módulo B, entonces el módulo B no puede requerir directa o indirectamente al módulo A.
- Hay dos tipos de módulos:
 - Abierto, con el modificador `open`, otorga acceso en tiempo de compilación a los tipos de los paquetes que se exportan explícitamente, pero otorga acceso mediante reflexión a los tipos de todos sus paquetes en tiempo de ejecución, como si todos los paquetes hubieran sido exportados.
 - Normal, sin el modificador `open`, otorga acceso en tiempo de compilación y tiempo de ejecución a los tipos en los paquetes que se exportan explícitamente.

© JMA 2001. All rights reserved

Módulos (v9)

- Las directivas de una declaración de módulo especifican las dependencias del módulo en otros módulos, los paquetes que pone a disposición de otros, los servicios que proporciona o que consume:
 - `requires`: indica un módulo del que depende. Modificadores:
 - `transitive`: propaga las dependencias transitivas del módulo requerido como dependencias implícitas del módulo actual.
 - `static`: es obligatoria en tiempo de compilación, pero opcional en tiempo de ejecución.
 - `exports`: otorga acceso externo a los tipos `public` y `protected` de un paquete en tiempo de compilación y de ejecución. Con el modificador `to` se limitan los módulos (calificados) a los que se otorga acceso (amigos).
 - `opens`: otorga acceso externo mediante reflexión a los tipos `public` y `protected` de un paquete en tiempo de ejecución, pero no en tiempo de compilación.
 - `provides`: especifica el tipo del servicio provisto y con la cláusula `with` especifica uno o más proveedores de servicios.
 - `uses`: especifica el tipo del servicio para descubrir proveedores.

© JMA 2001. All rights reserved

Módulos (v9)

```
module com.example.infraestructura.data {
    requires com.example.domain.entities;
    exports com.example.infraestructura.unitofwork;
    exports com.example.infraestructura.repositories to com.example.domain.services;
    opens com.example.infraestructura.validators;

    // Común para proveedores y consumidores de servicios.
    requires com.example.domain.contracts;
    // Exponer el proveedor de servicios
    provides com.example.domain.contracts.MyService with com.example.repositories.MyServiceImpl;
    // Exponer el consumo de servicios
    uses com.example.domain.contracts.MyService;
}
```

© JMA 2001. All rights reserved

Módulos (v9)

- Con Service Loader se puede tener una interfaz de proveedor de servicios (SPI), Servicio, y múltiples implementaciones de la SPI, Proveedores de servicios.
- Para evitar referencias circulares, un módulo de contratos exporta el o los interfaces de los servicios (aunque no recomendable, pueden ser clases abstractas o concretas).
- Los módulos que deseen proveer implementaciones del servicio deben:
 - Requerir el módulo de contratos.
 - Disponer de una clase publica que implemente el interfaz del servicio (proveedor).
 - Exportar el paquete del proveedor.
 - Exponer al proveedor como implementación del servicio.
- El modulo consumidor de los servicios debe:
 - Requerir el módulo de contratos.
 - Opcionalmente, requerir uno o varios módulos proveedores del servicio.
 - Declarar el uso de los servicios.
- Consumir los servicios:

```
ServiceLoader<MyService> services = ServiceLoader.load(MyService.class);
services.forEach(srv -> srv.action());
```

© JMA 2001. All rights reserved

Módulos (v9)

- Para enumerar todos los módulos JDK
 - `java --list-modules`
- Para enumerar todos los módulos de un determinado módulo:
 - `java --describe-module java.sql`
 - `jar --describe-module -f modulo.principal.jar`
- Para crear un módulo empaquetado:
 - `jar --create --file foo.jar --main-class com.foo.Main --module-version 1.0 -C foo/ classes resources`
- Para consultar las dependencias:
 - `jdeps --module-path . com.example.main.jar`
- Para ejecutar una aplicación modular:
 - `java --module-path ./ --module com.example.main/com.example.Principal`
- Para generar un JRE personalizado con lo mínimo imprescindible para poder ejecutar el módulo suministrado:
 - `jlink --module-path ./ --add-modules "com.example.main" --output build/`
 - `build/bin/java --list-modules`

© JMA 2001. All rights reserved

PAQUETE JAVA.LANG

© JMA 2001. All rights reserved

La clase Object

- Todos las clase en Java derivan de la clase Object:
 - Implícitamente: si no se define herencia, el sistema automáticamente le define como subclase de Object.
 - Explícitamente: La jerarquía de clases tiene como origen a la clase Object.
- Todas las clases heredan, por lo tanto, con los siguientes métodos:
 - clone() : Crea una copia superficial del objeto, los herederos deben autorizar la copia campo a campo implementando el interfaz Cloneable.
 - equals(Object obj) : Comparacion de dos objetos.
 - toString() : Devuelve una cadena con la representación del objeto.
 - getClass() : Devuelve objeto Class de la clase (final).
 - finalize() : Implica que todos los objetos tienen un destructor.
 - hashCode() : Devuelve un código hash para el objeto.
 - notify(), notifyAll() y wait() : Métodos relacionados con threads (final).

© JMA 2001. All rights reserved

Clases envolventes

- Clases diseñadas para ser un complemento de los tipos primitivos. Las clases son:
(Byte, Short, Integer, Long, Float, Double): Number, Character, Boolean
- Permiten implementar un mecanismo para pasar los tipos primitivos por referencia a los métodos.
- Amplían la funcionalidad de los tipos primitivos.
- El autoboxing/unboxing es una característica aparecida en JDK 1.5, que realiza automáticamente la conversión de tipo valor a clase y viceversa en función al tratamiento que se de al dato.
- La clase Math (final) encapsula las funciones matemáticas y cuenta con las constantes PI y el número E.

© JMA 2001. All rights reserved

java.math

- Soporte avanzado de operaciones matemáticas en forma de métodos estadísticos.
 - BigDecimal: Números decimales inmutables con signo de precisión arbitraria.
 - BigInteger: Enteros inmutables de precisión arbitraria.
 - MathContext: Objetos inmutables que encapsulan la configuración de contexto que describe ciertas reglas para operadores numéricos, como los implementados por la clase BigDecimal.
- Problemas IEEE 754

```
(new java.math.BigDecimal(value)).setScale(15,  
    java.math.RoundingMode.HALF_DOWN).doubleValue()
```

© JMA 2001. All rights reserved

Clases para Cadenas

- La clase String
 - Permite el manejo de cadenas de longitud fija.
 - Cuenta con métodos que permiten la comparación, manipulación y conversión de cadenas y subcadenas.
 - Aceptan el operador + como operador de concatenación.
- La clase StringBuilder
 - Permite el manejo de cadenas de longitud variable.
 - Cuenta con métodos que permiten la manipulación y cambio de tamaño de cadenas.
 - Aceptan el operador + como operador de concatenación.
- La clase StringBuffer
 - Versión síncrona de StringBuilder.

© JMA 2001. All rights reserved

La clase System

- Clase estática final que permite acceder a los recursos del sistema.
- Propiedades:
 - in : fichero de entrada estándar (InputStream).
 - out : fichero de salida estándar (PrintStream).
 - err : fichero de salida de error estándar (PrintStream).
- Métodos:
 - arraycopy(...) : Copia tablas elemento a elemento.
 - currentTimeMillis() : Devuelve la hora actual en milisegundos (1/1/1970).
 - exit(...) : Termina la ejecución de la aplicación.
 - gc(), runFinalization() y runFinalizersOnExit() : Gestionan la memoria.
 - setIn(), setOut() y setErr() : Cambian la entrada/salida estándar.
 - load() y loadLibrary() : Cargan métodos nativos.
 - getenv(), getProperties(), getProperty(), setProperty() : Permite la gestión de propiedades del sistema.

© JMA 2001. All rights reserved

IO (v25)

- La clase java.lang.IO proporciona métodos básicos de E/S orientados a líneas, que son alternativas más simples al método System.out.println y al java.util.Scanner:
 - println(Object): Escribe una representación de cadena del objeto especificado en la consola del sistema y luego vacía la consola.
 - print(Object): Escribe una representación de cadena del objeto especificado en la consola del sistema, finaliza la línea y luego vacía la consola.
 - readln(): Lee una sola línea de texto de la consola del sistema.
 - readln(String): Escribe un mensaje (prompt) como si llamara a print, y luego lee una sola línea de texto de la consola del sistema.

© JMA 2001. All rights reserved

La clase Runtime

- Clase estática final que permite acceder a la JVM.
 - Métodos:
 - `exec(...)` : Permite ejecutar comandos y aplicaciones del S.O.
 - `exit(...)` : Termina la ejecución y retorna al proceso padre.
 - `freeMemory()` y `totalMemory()` : Informan de la disponibilidad de memoria.
 - `getRuntime()` : Devuelve la instancia asociada a la aplicación.
-

© JMA 2001. All rights reserved

PAQUETE JAVA.UTIL

© JMA 2001. All rights reserved

Clases auxiliares

- Random
 - Generador de números pseudo-aleatorios.
- BitSet
 - Manejo de conjuntos de bits sin limitación de longitud.
- Locale
 - Representa las bases de internacionalización en programas Java.
- Date, DateFormat, Calendar, GregorianCalendar, TimeZone, SimpleTimeZone
 - Permiten manejar fechas y horas de distintos formatos y usos horarios.

© JMA 2001. All rights reserved

java.time (v8)

- Este paquete es una extensión de la versión Java 8 a las clases `java.util.Date` y `java.util.Calendar` que eran un poco limitadas para el manejo de fechas, horas y localización.
- Las clases definidas en este paquete representan los principales conceptos de fecha - hora, incluyendo instantes, fechas, horas, periodos, zonas de tiempo, etc. Están basados en el sistema de calendario ISO (aaaa-mm-ddThh:mm:ss) que es el calendario mundial de-facto y sigue las reglas del calendario Gregoriano.
- Cuenta con:
 - Enumerados de mes y de día de la semana
 - Las clases de fecha como el `java.time.LocalDate` manejan la fecha, pero, a diferencia del `java.util.Date`, es que es solo la fecha, y no hora.
 - La clase `java.time.LocalTime` es similar `java.util.Date` pero se centra únicamente en la hora.
 - La clase `java.time.LocalDateTime` manipula la fecha y la hora sin importar la zona horaria. Esta clase es usada para representar la fecha (año, mes, día) junto con la hora (hora, minuto, segundo, nanosegundo) y es, en efecto, la combinación de `LocalDate` y `LocalTime`.

© JMA 2001. All rights reserved

Clases para Cadenas

- La clase `StringTokenizer`
 - Ayuda a dividir un string en substrings o tokens, en base a otro string (normalmente un carácter) separador entre ellos denominado delimitador.
 - Implementa la interface `Enumeration`, por tanto, define las funciones `nextElement` y `hasMoreElements`.
- La clase `Scanner`
 - Es un iterador que permite analizar una fuente de texto y extraer los tipos primitivos y cadenas usando expresiones regulares.
 - Cuenta con los métodos `hasNextXXX()` y `nextXXX()`, donde XXX es el nombre del tipo primitivo.

© JMA 2001. All rights reserved

Clases para Cadenas

- La clase `Formatter`
 - Permite formatear cadenas al estilo de `printf`.
 - Similar a `String.format` pero permite globalización.

`%[argument_index$][flags][width][.precision]conversion`

 - `%b` Booleano
 - `%h` Hashcode
 - `%s` Cadena
 - `%c` Caracter unicode
 - `%d` Entero decimal
 - `%o` Entero octal
 - `%x` Entero hexadecimal
 - `%f` Real decimal
 - `%e` Real notación científica
 - `%g` Real notación científica o decimal
 - `%a` Real hexadecimal con mantisa y exponente
 - `%t` Fecha u hora

© JMA 2001. All rights reserved

Manipulaciones de tablas

- Las tablas, como objetos en Java, permiten:
 - clone: crear una copia superficial de la tabla.
 - length: informa de la dimensión de la tabla
- La clase `java.util.Arrays` proporciona algunas otras operaciones útiles:
 - `binarySearch`: Buscar en una matriz un valor específico para obtener el índice en el que se coloca.
 - `compare`, `compareUnsigned`, `equals`, `deepEquals`: Comparar dos matrices para determinar si son iguales o no.
 - `mismatch`: busca la primera discrepancia.
 - `fill`: Rellenar una matriz introduciendo un valor específico.
 - `copyOf`, `copyOfRange`: Devuelve una copia de la matriz.
 - `sort`, `parallelSort`: Ordenar secuencialmente o simultáneamente una matriz en orden ascendente.

© JMA 2001. All rights reserved

Paquete `java.util`

COLECCIONES

© JMA 2001. All rights reserved

Colecciones

- El Java da un amplio soporte para la creación y manejo de estructuras abstractas de datos.
- Una colección es una estructura que permite el almacenamiento de un conjunto de objetos.
- Colecciones vs Tablas:
 - Los elementos no tienen que ser del mismo tipo.
 - La capacidad es variable.
- Para implementar las colecciones aporta:
 - Interfaces.
 - Genéricos
 - Clases abstractas.
 - Clases ya implementadas.

© JMA 2001. All rights reserved

Interfaz Collection

- Define una serie de métodos que nos servirán para acceder a los elementos de cualquier colección de datos, sea del tipo que sea.
- Métodos:
 - `boolean add(Object o)`: Añade un elemento (objeto) a la colección.
 - `boolean remove(Object o)`: Elimina un determinado elemento (objeto) de la colección, devolviendo `true` si dicho elemento estaba contenido en la colección, y `false` en caso contrario.
 - `void clear()`: Elimina todos los elementos de la colección.
 - `boolean isEmpty()`: Indica si la colección está vacía (no tiene ningún elemento).
 - `int size()`: Nos devuelve el número de elementos que contiene la colección.
 - `boolean contains(Object o)`: Indica si la colección contiene el elemento (objeto) indicado.
 - `Iterator iterator()`: Proporciona un iterador para acceder a los elementos de la colección.
 - `Object [] toArray()`: Nos devuelve la colección de elementos como un array de objetos.

© JMA 2001. All rights reserved

Colecciones Genéricas

- Las versiones iniciales de las colecciones, en la mayoría de los casos, utilizaban como tipo base el Object dado que permiten almacenar referencias a cualquier clase.
- Podemos tener colecciones de tipos concretos de datos, lo que permite asegurar que los datos que se van a almacenar van a ser compatibles con un determinado tipo o tipos.
- Además, con esto nos ahorramos las conversiones cast al tipo que deseemos, puesto que la colección ya se asume que será de dicho tipo.
- A partir de JDK 1.5 deberemos utilizar genéricos siempre que sea posible.
- Si creamos una colección sin especificar el tipo de datos que contendrá normalmente obtendremos un warning.

© JMA 2001. All rights reserved

Inferencia de tipos genéricos (v7)

- Inferencia de tipos para la creación de instancia genérico:
 - Se puede sustituir los argumentos de tipo necesarios para invocar el constructor de una clase genérica con los paréntesis angulados de tipo vacíos (<>), informalmente llamados operador diamante, porque el compilador puede inferir los argumentos de tipo del contexto.
- La expresión:

```
Map<String, List<String>> myMap = new HashMap<String,  
List<String>>();
```
- Equivale a:

```
Map<String, List<String>> myMap = new HashMap<>();
```

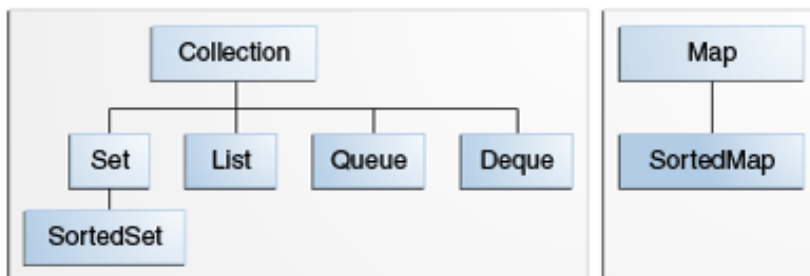
© JMA 2001. All rights reserved

Tipos de colecciones

- **Listas:** Una lista ordenada o secuencia. Normalmente permiten duplicados y tienen acceso aleatorio (es decir, puedes obtener elementos alojados en cualquier índice como si de un array se tratase).
- **Sets:** Conjuntos que no admiten dos elementos iguales. Es decir, colecciones que no admiten que un nuevo elemento B pueda añadirse a una colección que tenga un elemento A cuando `A.equals(B)`.
- **Pilas y Colas:** Colecciones ordenadas que permiten crear colas LIFO o FIFO. No permiten acceso aleatorio, solo pueden tomarse objetos de su principio, final o ambos, dependiendo de la implementación.
- **Maps:** Diccionario que asocian un valor a una clave. Parecido a la estructura de “array asociativo” que se encuentra en otros lenguajes. Un Map no puede tener dos claves iguales.

© JMA 2001. All rights reserved

Interfaces



© JMA 2001. All rights reserved

Implementaciones

- Las **implementaciones de propósito general** son las implementaciones más utilizadas, diseñadas para el uso diario.
- Las **implementaciones de propósito especial** están diseñadas para su uso en situaciones especiales y muestran características de rendimiento, restricciones de uso o comportamiento no estándar.
- Las **implementaciones concurrentes** están diseñadas para admitir una alta concurrencia, generalmente a expensas del rendimiento.
- Las **implementaciones de Wrapper** se utilizan en combinación con otros tipos de implementaciones, a menudo las de propósito general, para proporcionar una funcionalidad adicional o restringida.
- Las **implementaciones de conveniencia** son mini implementaciones, generalmente disponibles a través de métodos de fábrica estáticos, que brindan alternativas convenientes y eficientes a las implementaciones de propósito general para colecciones especiales (por ejemplo, conjuntos de singleton).
- Las **implementaciones abstractas** son implementaciones base que facilitan la construcción de implementaciones personalizadas.

© JMA 2001. All rights reserved

Listas de elementos

- Este tipo de colección se refiere a listas en las que los elementos de la colección tienen un orden (agregación), existe una secuencia de elementos en la que cada elemento estará en una determinada posición (índice) de la lista.
- Las listas vienen definidas en la interfaz **List**.
- Métodos:
 - void add(int indice, Object obj): Inserta un elemento (objeto) en la posición de la lista dada por el índice indicado.
 - Object get(int indice): Obtiene el elemento (objeto) de la posición de la lista dada por el índice indicado.
 - int indexOf(Object obj): Nos dice cual es el índice de dicho elemento (objeto) dentro de la lista. Nos devuelve -1 si el objeto no se encuentra en la lista.
 - Object remove(int indice): Elimina el elemento que se encuentre en la posición de la lista indicada mediante dicho índice, devolviéndonos el objeto eliminado.
 - Object set(int indice, Object obj): Establece el elemento de la lista en la posición dada por el índice al objeto indicado, sobrescribiendo el objeto que hubiera anteriormente en dicha posición. Nos devolverá el elemento que había previamente en dicha posición.

© JMA 2001. All rights reserved

Tipos de Lista

- **ArrayList:** Muy rápida accediendo a elementos, relativamente rápida agregando elementos si su capacidad inicial y de crecimiento están bien configuradas. Es la lista que deberías usar casi siempre.
- **LinkedList:** Una lista que también es una cola. Más rápida que ArrayList añadiendo elementos en su principio y eliminando elementos en general. Utilízala en lugar de ArrayList si realizas más operaciones de inserción (en posición 0) o de eliminación que de lectura. La diferencia de rendimiento es enorme.
- **Vector:** Vector es una colección deprecated (obsoleta), así que usadla únicamente si necesitáis un ArrayList concurrente. El rendimiento de Vector es superior al de Collections.synchronizedList(new ArrayList()).
- **CopyOnWriteArrayList:** Colección concurrente que es muy poco eficiente en operaciones de escritura, pero muy rápida en operaciones de lectura. Usala sobre Vector (o synced ArrayList) cuando el número de lecturas concurrentes sea mucho mayor al número de escrituras.

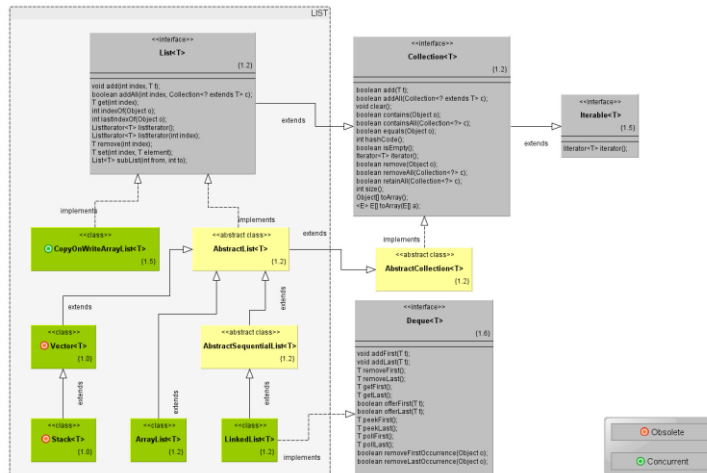
© JMA 2001. All rights reserved

Clase Vector

- Matriz de longitud variable de referencias a objetos.
- Métodos:
 - addElement(), insertElementAt(), setElementAt() :
 - Permiten introducir elementos en el vector.
 - removeElement(), removeAllElements(), removeElementAt():
 - Eliminan elementos del vector.
 - capacity(), ensureCapacity(), isEmpty(), setSize(), size():
 - Recuperan y manipulan el tamaño del vector.
 - contains(), elementAt(), indexOf(), firstElement(), lastElement():
 - Buscan y recuperan elementos.
 - elements():
 - Devuelve una enumeración para el recorrido del vector.

© JMA 2001. All rights reserved

Jerarquía de Listas



© JMA 2001. All rights reserved

Conjuntos

- Los conjuntos son grupos de elementos en los que no encontramos ningún elemento repetido.
- Consideramos que un elemento está repetido si tenemos dos objetos o1 y o2 iguales, comparándolos mediante el operador o1.equals(o2).
- De esta forma, si el objeto a insertar en el conjunto estuviese repetido, no nos dejará insertarlo.
- El método add devolverá un valor booleano, true si el elemento a añadir no estaba en el conjunto y ha sido añadido, o false si el elemento ya se encontraba dentro del conjunto.
- Un conjunto podrá contener como máximo un elemento null.
- Los conjuntos se definen en la interfaz Set

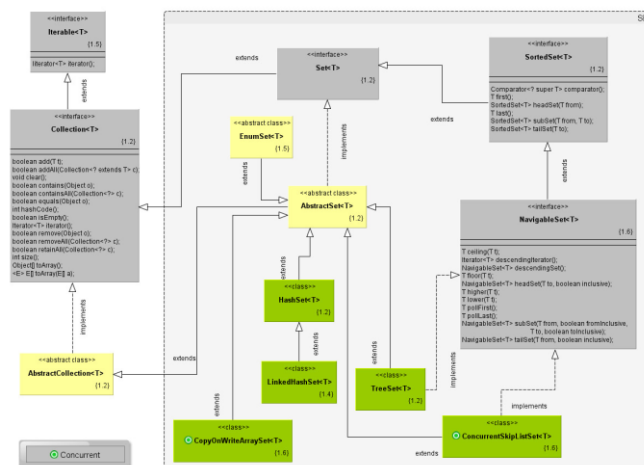
© JMA 2001. All rights reserved

Tipos de Set

- **HashSet**: La implementación más equilibrada de la interfaz Set. Es rápida y no permite duplicados, pero no tiene ningún tipo de ordenación. Utilízala si necesitas un control de duplicados pero ningún tipo de ordenación o acceso aleatorio.
- **LinkedHashSet**: Un HashSet que incluye ordenación de elementos por orden de entrada, una velocidad de iteración mucho mayor y un rendimiento mucho peor a la hora de añadir elementos. Utilízala si necesitas un Set ordenado por orden de inserción o si vas a usar un Set que vaya a realizar solamente operaciones de iteración.
- **TreeSet**: Un set que incluye una implementación de un árbol binario de búsqueda equilibrado. Este Set puede ser ordenado, pero su rendimiento es mucho peor en cualquier operación (menos iteración) respecto aun HashSet. Utilízalo solo si necesitas un Set con un criterio de ordenación específico y ten cuidado con las inserciones.
- **EnumSet**: La mejor implementación de Set para tipos enumerados (Enum).
- **CopyOnWriteArraySet**: Set concurrente que tiene un gran rendimiento de lectura, pero pésimo de escritura, eliminado y búsquedas. Úsalo solo en Sets concurrentes con pocas de estas operaciones.
- **ConcurrentSkipListSet**: Set concurrente y ordenable. Utilízalo solo cuando requieras un Set ordenable (como TreeSet) en entornos de concurrencia. En Sets de tamaños muy grandes su rendimiento empeora notablemente.

© JMA 2001. All rights reserved

Jerarquía de Conjuntos



© JMA 2001. All rights reserved

Pilas y Colas

- Las colas son estructuras que ofrecen un gran rendimiento al obtener elementos de su principio o de su final, representando colas LIFO / FIFO, aunque también veremos colas ordenadas en función de otros criterios.
- Deberás usar una cola cuando vayas a recuperar siempre el primer o último elemento de una serie. Se usan para implementar las mencionadas colas LIFO / FIFO, así como colas de prioridades (como puede ser un sistema de tareas o de procesos).
- Cabe destacar que hay dos tipos de colas, Queues y Deques. Las primeras solo proporcionan métodos para acceder al último elemento de la cola, mientras que las Deques permiten acceder a cualquiera de los dos extremos.

© JMA 2001. All rights reserved

Tipos de colas

- **ArrayDeque:** Una implementación de Deque de rendimiento excepcional. Implementa tanto cola LIFO como FIFO al ser una Deque y es la cola que deberías usar si quieres implementar una de estas dos estructuras.
- **ArrayBlockingQueue:** Cola limitada (FIFO) con bloqueo, una vez creada su capacidad no se puede cambiar. Los intentos put de colocar un elemento en una cola llena bloquean la operación y los intentos take en una cola vacía se bloquearán de manera similar.
- **DelayQueue:** Cola con bloqueo de elementos demorados, en la que un elemento solo se puede retirar una vez ha expirado un timeout.
- **LinkedBlockingDeque:** Una Deque concurrente que has de usar cuando quieras usar un ArrayDeque en entornos multihilo.
- **LinkedList:** LinkedList, anteriormente mencionada en la sección de listas, también es una Deque, sin embargo, su rendimiento es muy inferior al de ArrayDeque. No deberías usar LinkedList cuando quieras usar una cola.
- **PriorityQueue:** Una cola que se ordena mediante un Comparator, permitiendo crear una Cola donde el primer elemento no dependerá de su tiempo de inserción, sino de cualquier otro factor (tamaño, prioridad, etc). Deberemos usarlo cuando necesitemos este comparador (ArrayDeque no lo permite).
- **PriorityBlockingQueue:** La versión concurrente de PriorityQueue.

© JMA 2001. All rights reserved

Interface Queue

- La interface Queue permite implementar colas de tipo FIFO (primero en entrar, primero en salir).
- Métodos:
 - `add()`: Añade un elemento a la cola
 - `poll()`: Devuelve el primer elemento de la cola y lo elimina de ella
 - `peek()`: Devuelve el primer elemento de la cola sin sacarlo
 - `size()`: Número de elementos de la cola
 - `iterator()`: Devuelve un elemento Iterator para poder recorrer la cola.
 - `isEmpty()`: Indica si la cola está vacía o no.
 - `remove()`: Elimina el primer elemento de la Cola
- Implementaciones:
 - `java.util.LinkedList`
 - `java.util.PriorityQueue`

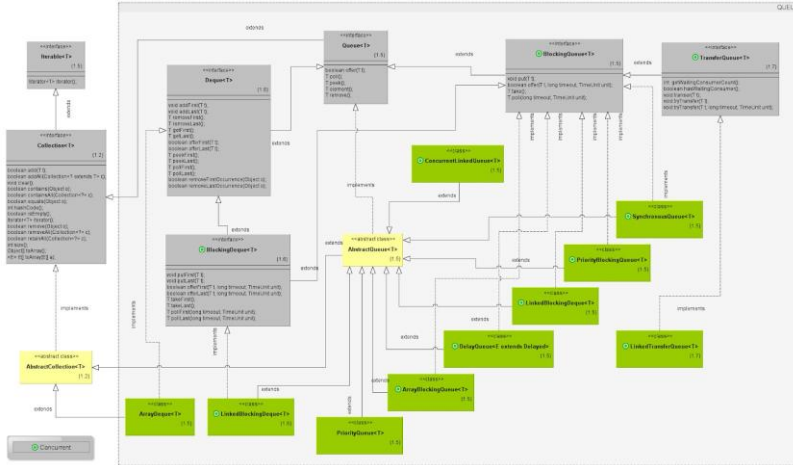
© JMA 2001. All rights reserved

Clase Stack

- Subclase de Vector que implementa una pila de tipo LIFO (último en entrar, primero en salir).
- Métodos:
 - `empty()`: Indica que la pila está vacía.
 - `peek()`: Devuelve el elemento superior de la pila.
 - `pop()`: Devuelve el elemento superior de la pila y lo elimina.
 - `push()`: Añade un elemento.
 - `search()`: Busca un elemento

© JMA 2001. All rights reserved

Jerarquía de Colas



© JMA 2001. All rights reserved

Mapas

- Aunque muchas veces se hable de los mapas como una colección, en realidad no lo son, ya que no heredan de la interfaz Collection. Los mapas se definen en la interfaz Map.
- Un mapa es un diccionario que relaciona una clave con un valor. Contendrá un conjunto de claves y a cada clave se le asociará un determinado valor.
- En versiones anteriores este mapeado entre claves y valores lo hacía la clase Dictionary, que ha quedado obsoleta. Tanto la clave como el valor puede ser cualquier objeto.
- Métodos:
 - Object get(Object clave): Nos devuelve el valor asociado a la clave indicada
 - Object put(Object clave, Object valor): Inserta una nueva clave con el valor especificado. Nos devuelve el valor que tenía antes dicha clave, o null si la clave no estaba en la tabla todavía.
 - Object remove(Object clave): Elimina una clave, devolviendonos el valor que tenía dicha clave.
 - Set keySet(): Nos devuelve el conjunto de claves registradas
 - int size(): Nos devuelve el número de parejas (clave,valor) registradas.

© JMA 2001. All rights reserved

Tipos de Map

- **HashMap:** La implementación más genérica de Map. Un array clave→valor que no garantiza el orden de las claves (de la misma forma que un HashSet). Si necesitas un Map no-concurrente que no requiera ordenación de claves, este es el tuyo. Si os fijáis en el código de HashSet, veréis que curiosamente utiliza un HashMap internamente.
- **LinkedHashMap:** Implementación de map que garantiza orden de claves por su momento de inserción; es decir, las claves que primero se creen serán las primeras. También puede configurarse para que la orden de claves sea por tiempo de acceso (las claves que sean accedidas precederán a las que no son usadas). Itera más rápido que un HashMap, pero inserta y elimina mucho peor. Utilízalo cuando necesites un HashMap ordenado por orden de inserción de clave.
- **TreeMap:** Un Map que permite que sus claves puedan ser ordenadas, de la misma forma que TreeSet. Úsalo en lugar de un HashMap solo si necesitas esta ordenación, ya que su rendimiento es mucho peor que el de HashMap en casi todas las operaciones (excepto iteración).

© JMA 2001. All rights reserved

Tipos de Map

- **EnumMap:** Un Map de alto rendimiento cuyas claves son Enumeraciones (Enum). Muy similar a EnumSet. Usadlo si vais a usar Enums como claves.
- **WeakHashMap:** Un Map que solo guarda referencias blandas de sus claves y valores. Las referencias blandas hacen que cualquier clave o valor sea elegible por el recolector de basura si no hay ninguna otra referencia al mismo desde fuera del WeakHashMap. Usa este Map si quieres usar esta característica, ya que el resto de operaciones tienen un rendimiento pésimo. Comúnmente usado para crear registros que vayan borrando propiedades a medida que el sistema no las vaya necesitando y vaya borrando sus referencias.
- **HashTable:** Map deprecated y concurrente. Básicamente, es un HashMap concurrente que no debes usar nunca. En su lugar, utiliza ConcurrentHashMap.
- **ConcurrentHashMap:** Un Map concurrente que no permite valores nulos. Sustitución básica de HashTable. Usala si necesitas un HashMap concurrente.

© JMA 2001. All rights reserved

Clase abstracta Dictionary

- Clase abstracta que permite el almacenamiento de conjuntos de pares clave/valor.
- Métodos:
 - put(): Permiten introducir un elemento.
 - get(): Recupera un valor por su clave.
 - remove(): Elimina un elemento.
 - isEmpty(), size(): Recuperan el tamaño del vector.
 - elements(): Devuelve una enumeración de los valores.
 - keys(): Devuelve una enumeración de las claves.

© JMA 2001. All rights reserved

Clase Hashtable

- Subclase instanciable de la clase Dictionary.
- Implementa el mecanismo de hashing (valor único) de claves para el almacenamiento y recuperación de elementos.
- La clase de los objetos utilizados como clave deben sobrescribir los métodos hashCode() y equals(). La clase String implementa los dos métodos.
- Métodos adicionales:
 - clear(): Borra toda la tabla.
 - contains(), containsKey(): Busca si contiene el valor o la clave.
 - rehash(): Recalcula la tabla y sus claves.

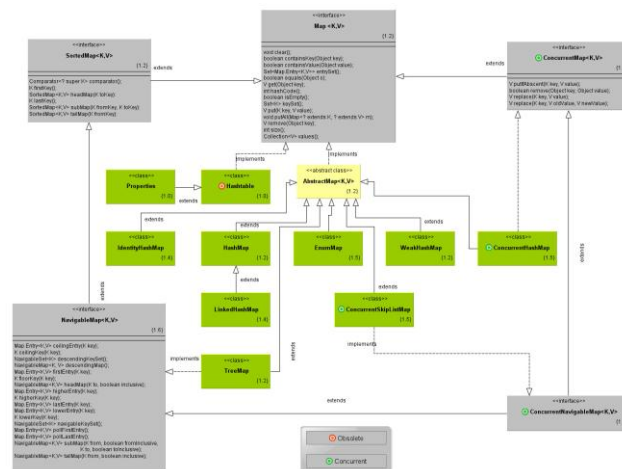
© JMA 2001. All rights reserved

Clase Properties

- Subclase persistente de Hashtable, donde las claves y los valores son cadenas.
- Métodos adicionales:
 - getProperty(): Devuelve el valor de una propiedad.
 - list(): Envía una lista de las propiedades y sus valores
 - load(): Recupera la tabla.
 - save(): Guarda la tabla.
 - propertyNames(): Devuelve una enumeración de las claves.

© JMA 2001. All rights reserved

Jerarquía de Mapas



© JMA 2001. All rights reserved

Wrappers

- La clase Collections aporta una serie métodos para cambiar ciertas propiedades de las listas. Los wrappers son objetos que ‘envuelven’ al objeto de nuestra colección, pudiendo de esta forma hacer que la colección esté sincronizada o que pase a ser de solo lectura.
- Podemos conseguir que las operaciones se ejecuten de forma sincronizada envolviendo nuestro objeto de la colección con un wrapper, que será un objeto que utilice internamente nuestra colección encargándose de realizar la sincronización cuando llamemos a sus métodos. Para obtener estos wrappers utilizaremos los siguientes métodos estáticos de Collections:
 - Collection synchronizedCollection(Collection c), List synchronizedList(List l), Set synchronizedSet(Set s), Map synchronizedMap(Map m), SortedSet synchronizedSortedSet(SortedSet ss), SortedMap synchronizedSortedMap(SortedMap sm)
- Para obtener versiones de sólo lectura de nuestras colecciones:
 - Collection unmodifiableCollection(Collection c), List unmodifiableList(List l), Set unmodifiableSet(Set s), Map unmodifiableMap(Map m), SortedSet unmodifiableSortedSet(SortedSet ss), SortedMap unmodifiableSortedMap(SortedMap sm)

© JMA 2001. All rights reserved

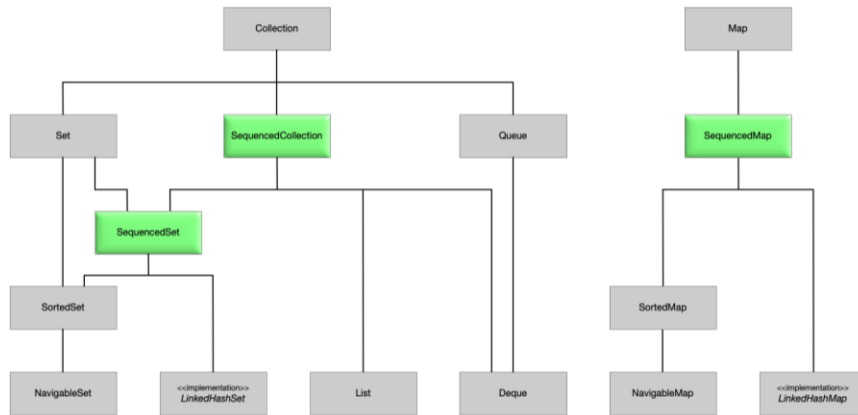
Sequenced Collections (v21)

- Las colecciones secuenciales, inyecta nuevas interfaces en la jerarquía existente, ofreciendo un mecanismo para acceder, añadir o borrar el primer y último elemento de una colección utilizando métodos predeterminados integrados. Además, proporciona soporte para obtener una vista invertida de la colección.

```
interface SequencedCollection<E> extends Collection<E> {  
    // new method  
    SequencedCollection<E> reversed();  
    // methods promoted from Deque  
    void addFirst(E);  
    void addLast(E);  
    E getFirst();  
    E getLast();  
    E removeFirst();  
    E removeLast();  
}
```

© JMA 2001. All rights reserved

Sequenced Collections (v21)



© JMA 2001. All rights reserved

Tipos primitivos en las colecciones

- Los tipos primitivos del Java son: boolean, int, long, float, double, byte, short, char.
- Cuando trabajamos con colecciones los elementos que contienen éstas son siempre objetos, por lo que en un principio no podríamos utilizar tipos valor.
- Para hacer esto posible tenemos una serie de clases envoltentes que se encargarán de envolver a estos tipos primitivos, permitiéndonos tratarlos como objetos y por lo tanto utilizarlos como elementos de colecciones. Las clases envoltentes tienen nombres similares al del tipo primitivos que encapsulan, empezando con mayúscula: Boolean, Integer, Long, Float, Double, Byte, Short, Character.
- El autoboxing/unboxing es una característica aparecida en JDK 1.5, que realiza automáticamente la conversión de tipo valor a clase y viceversa en función al tratamiento que se de al dato.

© JMA 2001. All rights reserved

Métodos factoría para colecciones (v9)

- Se incorporan a las colecciones métodos de clase con argumentos variables que simplifican su creación con una lista inmutable de valores:

```
// Java 8
List<String> list = Collections.unmodifiableList(Arrays.asList("a", "b", "c"));
Set<String> set = Collections.unmodifiableSet(new HashSet<>(Arrays.asList("a", "b", "c")));
Map<String, Integer> map = new HashMap<>();
map.put("a", 1);
```

```
// Java 9
List<String> list = List.of("a", "b", "c");
Set<String> set = Set.of("a", "b", "c");
Map<String, Integer> map = Map.of("a", 1);
```

© JMA 2001. All rights reserved

Interfaces Enumeration e Iterator

- Enumeration: Define los métodos que permiten recorrer los elementos de un conjunto de objetos.
 - boolean hasMoreElements()
 - Indica si quedan elementos por recorrer.
 - Object nextElement()
 - Devuelve el siguiente elemento del conjunto.
- Iterator<E>: Define los métodos que permiten recorrer los elementos de una colección o un array. A diferencia de las enumeraciones, los iteradores permiten al llamador eliminar elementos de la colección subyacente durante la iteración.
 - boolean hasNext()
 - Indica si quedan elementos en la iteración.
 - E next()
 - Devuelve el siguiente elemento de la iteración.

© JMA 2001. All rights reserved

Recorrer las colecciones

- Originalmente, mediante la interfaz Enumeration:

```
while (enum.hasMoreElements()) {  
    Object item = enum.nextElement();  
    // Hacer algo con el item leido  
}
```

- Posteriormente, mediante la interfaz Iterator:

```
while (iter.hasNext()) {  
    Object item = iter.next();  
    if(condicion_borrado(item))  
        iter.remove();  
}
```

- A partir de JDK 1.5:

```
ArrayList<String> a = new ArrayList<String>();  
for(String cadena: a)  
    System.out.println (cadena);
```

© JMA 2001. All rights reserved

Comparación de objetos

- Comparar objetos es fundamental para hacer ciertas operaciones y manipulaciones en estructuras de datos.
- Disponemos de diferentes mecanismos:
 - Sobrecarga de equals
 - Implementación de Comparable
 - Implementación de Comparator

© JMA 2001. All rights reserved

Sobre escritura de equals

- Todas las clases heredan de la clase Object el método equals(Object o) que es sobre escribible (también conviene sobrecargar el método hashCode):

```
@Override
public boolean equals(Object o) {
    // return true o false, según un criterio
}
```
- Cuando se sobrecarga el método equals se deben cumplir las siguientes propiedades:
 - Reflexividad: x.equals(x) devuelve siempre verdadero, si no es nulo.
 - Simetría: para cualquier par de instancias no nulas, x.equals(y) devuelve verdadero si y sólo si y.equals(x) también devuelve verdadero.
 - Transitividad: si x.equals(y)==true y y.equals(z)==true, entonces x.equals(z) también será verdadero, para cualesquiera instancias no nulas.
 - Consistencia: múltiples llamadas al método con las mismas instancias devuelven el mismo resultado.
 - Comparación con null falsa: x.equals(null) devuelve falso.

© JMA 2001. All rights reserved

Sobre escritura de hashCode

- hashCode devuelve un valor de resumen hash para el objeto. Este método es utilizado en tablas hash como las proporcionadas por HashMap.

```
@Override
public int hashCode(){
    final int prime = 31;
    int result = 1;
    result = prime * result + ((nombre == null) ? 0 : nombre.hashCode());
    result = prime * result + ((apellidos == null) ? 0 : apellidos.hashCode());
    return result;
}
```
- Cuando se sobrecarga el método hashCode se deben cumplir las siguientes normas:
 - Siempre que se invoca en el mismo objeto más de una vez durante la ejecución de una aplicación Java, el método hashCode debe devolver el mismo número entero, siempre que no se modifique la información utilizada en las comparaciones equals del objeto. Este número entero no necesita permanecer consistente de una ejecución de una aplicación a otra ejecución de la misma aplicación.
 - Si dos objetos son iguales, según el método equals, sus hashCode deben ser iguales.
 - No es obligatorio que dos objetos distintos generen hashCode distintos, pero en caso de serlo pueden mejorar el rendimiento de las tablas hash.

© JMA 2001. All rights reserved

Implementación de Comparable

- Los algoritmos, como `Collections.sort()`, requieren que los objetos implementen la interfaz `Comparable` para que tengan un método `compareTo()` que devuelva un número negativo, positivo o cero, según si un objeto es menor, mayor o igual que el otro (0 debería equivaler a equals).
- Por supuesto, no todos los objetos se pueden comparar en términos de mayor o menor. Así que, el hecho de que una clase implemente `Comparable`, nos indica que una colección de dichos objetos tiene un “orden natural”.

```
public class Persona implements Comparable<Persona> {  
    public int id;  
    public String apellido;  
    ...  
    @Override  
    public int compareTo(Persona p) {  
        return this.id - p.id;  
    }  
    ...  
}
```

© JMA 2001. All rights reserved

Implementación de Comparator

- En muchas colecciones la ordenación podría realizarse por diferentes criterios, por lo que se debe utilizar un comparador externo.
- Para ello tenemos que implementar la interfaz `Comparator` que nos obliga a implementar el método `compare`. No debemos hacerlo dentro de la propia clase cuyas instancias vamos a comparar, es necesario crear una clase adicional que puede ser anónima:

```
public class ComparaPersonaPorNombre implements Comparator<Persona>{  
    public int compare(Persona p1, Persona p2) {  
        return p1.nombre.compareToIgnoreCase(p2.nombre);  
    }  
}
```

- Para ordenar la colección:
`Collections.sort(personas); //Comparable.compareTo`
`Collections.sort(personas, new ComparaPersonaPorNombre()); //Comparator.compare`

© JMA 2001. All rights reserved

Collection Stream (v8)

- En la versión 8 se agrego la implementación del interfaz `Stream<T>` a los array, enumeraciones y colecciones de la biblioteca estándar.
- Permite acceder a sus elementos de una nueva forma distinta a los `Iterator`, recorrerlos como una secuencia de elementos con soporte para operaciones de agregación secuenciales y paralelas, así como el uso de expresiones `lambda`.
- Un flujo consiste una fuente (una colección), varias operaciones intermedias (de filtrado o transformación) y una operación final que produce un resultado (suma, cuenta...).

```
int sum = widgets.stream()
    .filter(w -> w.getColor() == RED)
    .mapToInt(w -> w.getWeight())
    .sum();
```

© JMA 2001. All rights reserved

Collection Stream (v8)

- Características:
 - Sin almacenamiento. Un flujo no es una estructura de datos que almacena elementos, es un cursor o intermediario con una colección, una matriz, un canal de E/S, ...
 - Inmutable. El origen no es modificado.
 - Consumible. Los elementos de una corriente solo se visitan una vez durante la vida de una corriente.
 - Sólo se pueden «consumir» elementos que estén en el stream. Al terminar el recorrido, el stream desaparece.
 - Una cadena de operaciones sólo puede ocurrir una vez en un stream determinado.
 - Puede ser en serie (por defecto) o en paralelo
- Las estructuras que soportan esta nueva API se encuentran en el paquete `java.util.stream`.
- Se han modificado los paquetes de colecciones, IO, ... para que incorporen los métodos `stream()` para poder usarlos con la nueva API.

© JMA 2001. All rights reserved

Collection Stream (v8)

- Java Stream nos permite realizar operaciones de tipo filtro/mapeo/reducción sobre colecciones de datos.
- Modelo filtro/mapeo/reducción:
 - Filtro: se seleccionan los datos que se van a procesar
 - Mapeo: se convierten a otro tipo de dato
 - Reducción: y al final se obtiene el resultado deseado.
- El filtrado y el mapeo son operaciones intermedias con estado o sin estado: si crean o no nuevos valores.
- Las operaciones con estado pueden necesitar procesar toda la entrada antes de producir un resultado.

© JMA 2001. All rights reserved

Collection Stream (v8)

- Para crear un Stream con sus métodos estáticos:
 - `iterate()`: Devuelve un Stream producido por una iteración
 - `generate()`: Devuelve un Stream producido por un Supplier.
 - `concat()`: Crea un Stream secuencial con los elementos de los dos Stream.
 - `of()`: Devuelve un Stream con los elementos especificados.
 - `empty()`: Devuelve un Stream vacío .
 - `ofNullable()`: Devuelve un Stream que contiene un solo elemento, si no es nulo o un Stream vacío si es nulo.
- Se pueden obtener de:
 - Collection, Arrays, BufferedReader, Files, Random y otras muchas clases.

© JMA 2001. All rights reserved

Collection Stream (v8)

- Para el filtro/mapeo:
 - `distinct()`: Devuelve un Stream con los elementos distintos elementos.
 - `filter()`: Devuelve un Stream con los elementos que coinciden con el predicado.
 - `dropWhile()`: Devuelve un Stream sin los elementos que coinciden con el predicado.
 - `takeWhile()`: Devuelve un Stream con los elementos que coinciden con el predicado hasta encontrar el primero que no coincide.
 - `map()`: Devuelve un Stream con los elementos transformados por la función dada. [`mapToDouble()` , `mapToInt()` `mapToLong()`]
 - `flatMap()`: Devuelve un Stream que consta de los resultados de reemplazar cada elemento de este flujo con el contenido de un Stream mapeado producido al aplicar la función de mapeo proporcionada a cada elemento. [`flatMapToDouble()`, `flatMapToInt()`, `flatMapToLong()`]
 - `peek()`: Devuelve el Stream después de aplicar la acción proporcionada en cada elemento a medida que se consumen elementos del flujo.
 - `sorted()`: Devuelve un Stream con los elementos ordenados.
 - `skip()`: Devuelve un Stream después de descartar los primeros n elementos.
 - `limit()`: Devuelve un Stream con un número máximo de elementos.
 - `gather()`: Devuelve el Stream resultante de aplicar la operación personalizada dada (Gatherer API) (v24)

© JMA 2001. All rights reserved

Collection Stream (v8)

- Para reducir:
 - `allMatch()`: Devuelve true si todos los elementos la secuencia cumplen con el predicado.
 - `anyMatch()`: Devuelve true si algún elemento de la secuencia cumple con el predicado.
 - `findFirst()`: Devuelve un Optional con el primer elemento del flujo.
 - `findAny()`: Devuelve un Optional vacío si el flujo está vacío o con cualquier elemento si no.
 - `reduce()`: Reduce la secuencia a un valor calculado con la función de acumulación asociativa proporcionada y devuelve un Optional.
 - `count()`: Devuelve el recuento de elementos de esta secuencia.
 - `min()`, `max()`: Devuelve un Optional con el elemento mínimo o máximo de la secuencia según Comparador proporcionado.
 - `forEach()`: Realiza una acción para cada elemento de esta secuencia.
 - `collect()`: Realiza una operación de reducción mutable en los elementos de este flujo usando Collectors (Collector API). [~~Collectors.toList()~~, `Collectors.toCollection(TreeSet::new)`]
 - `toArray()`: Devuelve una matriz con los elementos de la secuencia.
 - `toList()`: Devuelve una lista con los elementos de la secuencia. (v16)

© JMA 2001. All rights reserved

Collection Stream (v8)

```
int sum = widgets.stream()
    .filter(b -> b.getColor() == RED)
    .map(b -> b.getWeight())
    .reduce(0, (acumulado, item) -> acumulado + item);
List<String> list = people.stream()
    .sorted(Person::getId)
    .skip(page * rows)
    .limit(rows)
    .collect(Collectors.toList());
Map<Department, Integer> totalByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment,
        Collectors.summingInt(Employee::getSalary)));
people.stream().map(Person::getName).forEach(System.out::println);
```

© JMA 2001. All rights reserved

Collection Stream (v8)

- Los streams actúan en modo perezoso: las operaciones solo se realizan cuando se llama a la operación final (reducción).
- Son mas eficientes, en algunos casos no necesitando procesar todos los elementos del stream para devolver el resultado final.
- Permiten dividir las operaciones en varias instrucciones, aplicando condicionales, devolviendo como valor de retorno y postergando la ejecución.

```
Stream<Person> query = lista.stream();
if(femaleOnly)
    query = query.filter(e -> e.getGender() == Person.Sex.FEMALE);
if(paged)
    query = query.sorted().skip(page * rows).limit(rows);
// ...
query.forEach(e -> e.setSalar(e.getSalar() * delta));
```

© JMA 2001. All rights reserved

Parallel Stream (v8)

- Las operaciones en un flujo paralelo se procesan utilizando varios subprocesos. No es necesario realizar pasos adicionales para procesar flujos paralelos.
- La mayoría de los métodos de la API Streams producen flujos secuenciales de forma predeterminada. Para crear un flujo paralelo se usa el método `parallelStream()` de la interfaz `Collection` (`List`, `Set`) o el método `parallel()` de una secuencia secuencial para en una paralela. El método `sequential()` convierte una secuencia paralela en una secuencial.

```
List<Integer> listOfIntegers = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9);
System.out.println("Sequential Stream: ");
listOfIntegers.stream().forEach(e -> System.out.print(e + " "));
System.out.println("\nParallel Stream: ");
listOfIntegers.stream().parallel().forEach(e -> System.out.print(e + " "));
```

© JMA 2001. All rights reserved

PROGRAMACIÓN MULTITHILO

© JMA 2001. All rights reserved

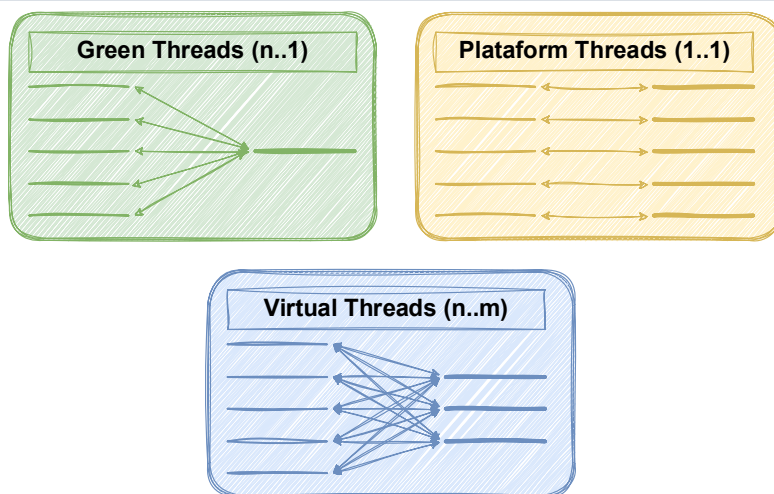
Evolución de la concurrencia en Java (v21)



- Las implementaciones iniciales de JVM (1.0 hasta 1.3) basaban su ejecución en los llamados Green Threads. Estos threads y su ejecución eran manejados íntegramente por la JVM. El hilo principal del proceso (Platform Thread) era único y era el que se usaba para la gestión de todos los hilos Java.
- La introducción de la programación con Futures sobre un pool de ExecuteService fue un cambio grande en el enfoque de programación en modo tradicional. Estos Thread Pools eran reservas de hilos de plataforma que eran administrados y mantenidos para poder dar cabida a la ejecución de los Java Threads.
- El gestor ForkJoinPool introdujo el concepto de work-stealing. ForkJoinPool está pensado para optimizar el paralelismo en un enfoque de tareas Fork-and-Join. Es decir, dividir una tarea en tareas más pequeñas que puedan correr de manera concurrente. ForkJoinPool sigue siendo un gestor ThreadPool de hilos plataforma, pero su gestión de los hilos le permite compartir los hilos del pool entre las distintas subtareas.
- El Project Loom (Virtual Threads) nace con la intención de reemplazar al sistema operativo como gestor del ciclo de vida de los threads, ocupándose ahora la JVM de la gestión de memoria del stack del hilo y la orquestación.

© JMA 2001. All rights reserved

Hilos virtuales (v21)



© JMA 2001. All rights reserved

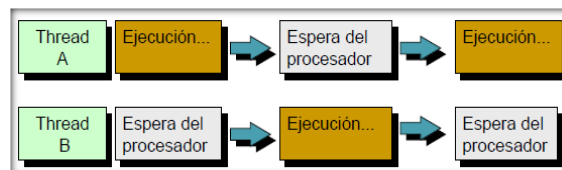
Introducción

- Multitarea, posibilidad de que un ordenador realice varias tareas a la vez.
- La Multitarea real se consigue cuando disponemos de múltiples CPU, sino será “emulada”.
- Esta emulación se consigue repartiendo tiempo y recursos entre distintos procesos.
- La palabra thread (hilo) hace referencia a un flujo de control dentro de un programa.
- Dentro de un programa pueden ejecutarse varios hilos a la vez.
- Los hilos comparten:
 - Los datos del programa (además pueden tener datos propios)
 - El entorno de ejecución
- Un Hilo puede ser el recolector de basura de Java que elimina los datos no deseados mientras el programa continúa con su ejecución normal. El uso de hilos es muy variado
 - Animación
 - Creación de servidores
 - Tareas de segundo plano
 - Programación paralela,...

© JMA 2001. All rights reserved

Introducción

- La máquina virtual Java implementa un entorno que permite a varias secuencias de instrucciones ejecutarse al mismo tiempo.
- En Java, los threads no se ejecutan necesariamente al mismo tiempo.
- En realidad, el interprete de Java puede ejecutar los threads por rodajas de tiempo (restricción impuesta en los sistemas monoprocesador).
- Este tipo de ejecución está basada en las posibilidades del planificador del Sistema Operativo.



© JMA 2001. All rights reserved

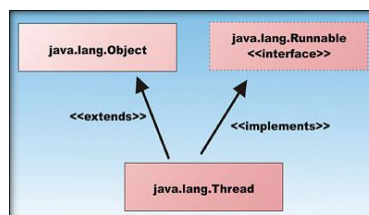
Introducción

- Cuando se ejecuta un programa Java, se lanza automáticamente el hilo PRINCIPAL.
- Es el hilo desde el cual se lanzan los demás hilos y debe de ser el último en terminar → Final del Programa.
- El hilo principal puede ser controlado a través de un Objeto de tipo Thread de la siguiente forma:
 - `Thread t = Thread.currentThread();`
- A partir de este momento este hilo se puede gestionar igual que los demás.

© JMA 2001. All rights reserved

Clase Thread y la interfaz Runnable

- La interfaz `java.lang.Runnable` permite definir las operaciones que realiza cada thread.
- Esta interfaz se define con un solo método público llamado `run` que puede contener cualquier código y que será el código que se ejecutará cuando se lance el thread.
- Para que una clase realice tareas concurrentes, basta con implementar `Runnable` y programar el método `run`.



© JMA 2001. All rights reserved

Clase Thread y la interfaz Runnable

- La clase Thread , implementa la interface Runnable, por lo que es posible crea objetos cuyo código se ejecute en un hilo aparte.
- Esta clase permite:
 - Iniciar
 - Controlar
 - Detener hilos de programa.
- Un nuevo thread se crea con un nuevo objeto de la clase `java.lang.Thread` o cualquier otra clase que implemente la interface Runnable.

```
hilo = new Thread(objetoRunnable);  
hilo.start(); //Se ejecuta el método run del objeto Runnable
```

© JMA 2001. All rights reserved

Creación con Runnable

- La interface Runnable proporciona un método alternativo a la herencia de la clase Thread, para los casos en los que no es posible hacer que nuestra clase extienda la clase Thread.
- Esto ocurre cuando nuestra clase, que deseamos correr en un thread independiente, deba extender alguna otra clase.
- Dado que no existe herencia múltiple, nuestra clase no puede extender a la vez la clase Thread y otra más.
- En este caso nuestra clase debe implantar la interface Runnable, variando ligeramente la forma en que se crean e inician los nuevos threads.

© JMA 2001. All rights reserved

Creación con Runnable

- Para poder ejecutar los threads necesitamos realizar los siguientes pasos:
 - Crear una clase que implemente Runnable
 - Definir el método run con las acciones que tomará el hilo de programa

```
class Movimiento implements Runnable{  
    public void run() {  
        while (true) {  
            //código
```

- Crear un objeto Thread tomando como parámetro un objeto de la clase anterior

```
Movimiento a1 = new Movimiento();  
Thread thread1 = new Thread(a1,"Nombre");
```

- Ejecutar el método start del Thread para ejecutar el método run

```
thread1.start();
```

© JMA 2001. All rights reserved

Creación con Runnable

```
public class HiloRunnable implements Runnable {  
    private int hijo = 0;  
    private int iter = 0;  
    private int sleep = 0;  
    private Thread t;  
  
    public HiloRunnable(int hijo, int iter, int sleep) {  
        super();  
        this.hijo = hijo;  
        this.iter = iter;  
        this.sleep = sleep;  
    }  
  
    public HiloRunnable(int hijo) {  
        this(hijo, 5, 500);  
    }  
}
```

→

© JMA 2001. All rights reserved

Creación con Runnable

←

```
@Override
public void run() {
    try {
        for (int i = 0; i < iter; i++) {
            System.out.printf("Hilo: %s Paso: %d de %d\n", Thread.currentThread().getName(), i+1, iter);
            Thread.sleep(sleep);
        }
    } catch (Exception e) {
        System.out.printf("Error en el hijo %d\n", hijo);
    }
    System.out.printf("Termina Hilo: %d\n", hijo);
}
public void lanzar() {
    t = new Thread(this, "Hijo " + hijo);
    System.out.println("Lanzo " + t.getName());
    t.start();
}
}
```

© JMA 2001. All rights reserved

Pasos de creación mediante herencia

- Es la opción más utilizada, pues como la clase Thread implementa la interfaz Runnable, ya cuenta con el método run.
- Pasos:
 - Crear una clase basada en Thread
 - Definir el método run y en él las acciones que tomará el hilo de programa

```
class Movimiento extend Thread{
    public void run() {
        while (true) {
            //código
        }
    }
}
```
 - Crear un objeto de la clase

```
Movimiento a1 = new Movimiento();
```
 - Ejecutar el método start

```
a1.start();
```

© JMA 2001. All rights reserved

Creación mediante herencia

```
public class HiloHerencia extends Thread {
    private int iter = 0; private int sleep = 0;
    public HiloHerencia(String name) {
        this(name, 5, 500);
    }
    public HiloHerencia(String name, int iter, int sleep) {
        super(name);
        this.iter = iter; this.sleep = sleep;
    }
    @Override
    public void run() {
        try {
            for (int i = 0; i < iter; i++) {
                System.out.printf("Hilo: %s Paso: %d de %d\n", getName(), i+1, iter);
                Thread.sleep(sleep);
            }
        } catch (Exception e) { System.out.printf("Error en el hijo %s\n", getName()); }
        System.out.printf("Termina Hilo: %s\n", getName());
    }
}
```

© JMA 2001. All rights reserved

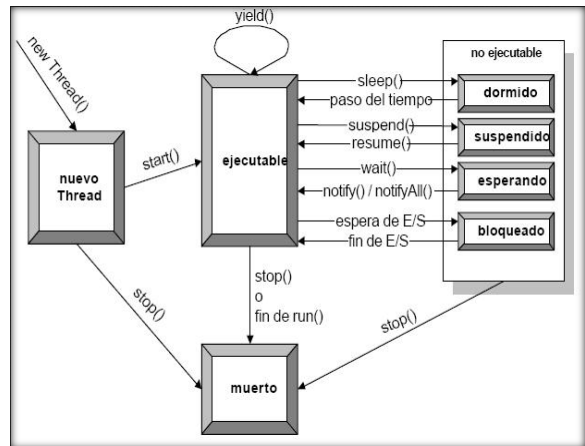
Ejecución simultanea

```
public static void main(String[] args) {
    HiloRunnable h = new HiloRunnable(1);
    HiloHerencia h2 = new HiloHerencia("Hijo 99", 50, 99);
    try {
        h.lanzar();
        h2.start();
        for (int i = 5; i > 0; i--) {
            System.out.println("Hilo Padre: " + i);
            Thread.sleep(1000);
        }
        System.out.println("Termina el padre");
    } catch (Exception e) {
        System.out.println("Error en el padre");
    }
}
```

© JMA 2001. All rights reserved

Ciclo de Vida

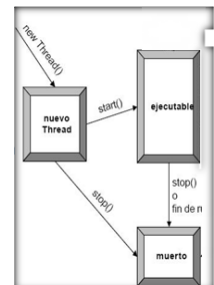
- Se entiende por “ciclo de vida” los diferentes estados en los que un hilo puede pasar desde que se inicia hasta que finaliza.



© JMA 2001. All rights reserved

Ciclo de Vida. Estados

- Estado Nuevo:**
 - Es el estado en el que se encuentra un thread en cuanto se crea.
 - En ese estado el thread no se está ejecutando, sólo se ha ejecutado el código del constructor del Thread.
- Ejecutable**
 - Ocorre cuando se llama al método start.
 - No tiene por qué estar ejecutándose el thread, el thread está listo para ser ejecutado; se halla en la cola del planificador del sistema operativo.
- Muerto**
 - Significa que el método finalizó. Esto puede ocurrir si:
 - El flujo de programa salió del método run
 - Por una excepción no capturada
 - Se invocó el método stop().
 - Se puede comprobar si un thread no está muerto con el método isAlive



© JMA 2001. All rights reserved

Ciclo de Vida. Estados

- **No Ejecutándose:**

- El thread espera un evento que le permita volver a la cola del planificador.
- Esto puede ocurrir si:
 - Se llamó al método `sleep`
 - Se llamó al método `wait`
 - Se llamó al método `suspend`
 - Se llamó a una operación de entrada o salida.
 - Se intento bloquear otro thread que ya estaba bloqueado

- **Dormido**

- Se llamó al método `sleep`

- **Bloqueado**

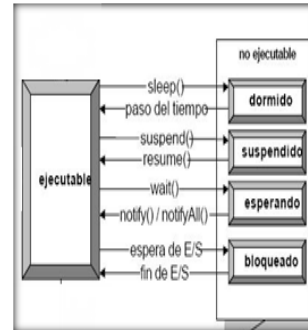
- Se llamó a una operación de entrada o salida.

- **Esperando**

- Se llamó al método `wait`

- **Suspendido**

- Se intento bloquear otro thread que ya estaba bloqueado



© JMA 2001. All rights reserved

Ciclo de Vida. Estados

- **Reanudación:**

- El thread vuelve al estado de Listo a Ejecutarse cuando ocurre alguna circunstancia:

- **Dormido**

- Se pasaron los milisegundos programados por el método `sleep`

- **Bloqueado**

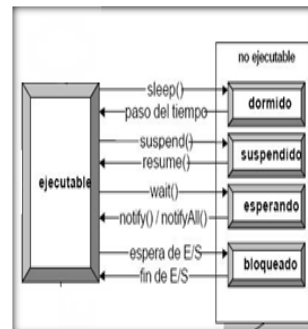
- Se terminó la operación de entrada o salida que había bloqueado al thread
- Se liberó al thread de un bloqueo.

- **Esperando**

- Se llamó a `notify` tras haber usado `wait`

- **Suspendido**

- Se ha invocado al método `resume`.



© JMA 2001. All rights reserved

Estructura

- Creación:

`Hilo miHilo = new Hilo( );`

- Inicio:

`miHilo.start( );` Implícitamente se invoca el método `run()`.

- Muerte:

- El hilo finaliza cuando finaliza el método `run( )`

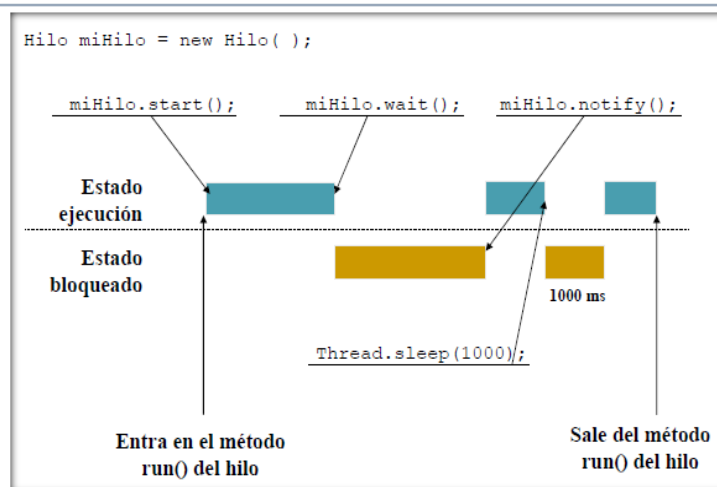
- Bloqueo de un thread

Desbloqueo del thread cuando:

- `Thread.sleep(10000);` ← Período de tiempo terminado
- `miHilo.wait();` ← `miHilo.notify()` o `miHilo.notifyAll()`
- Espera de recurso(s) ← Recurso(s) liberado(s)

© JMA 2001. All rights reserved

Estructura



© JMA 2001. All rights reserved

Control de los Threads

- **sleep**
 - Utilizado para que un thread se detenga durante un tiempo sin consumir CPU (objeto.sleep(mseg) o Thread.sleep(mseg))
 - Tras consumir finalizar el tiempo indicado, continuará la ejecución.
 - En programación multitarea, dormir un Thread es obligatorio para que los otros se pueden ejecutar.
- **interrupt**
 - Los Threads no tienen ningún método de interrumpir un Hilo.
 - Usando interrupt indicamos al Thread que debe de finalizar su proceso cuando pueda.
 - Si el hilo no se está ejecutando, no se atiende la petición hasta que se ejecute

© JMA 2001. All rights reserved

Control de los Threads

```
public class SleepMessages {  
  
    public static void main(String args[]) throws InterruptedException {  
        String importantInfo[] = { "Mares eat oats", "Does eat oats", "Little lambs eat ivy",  
            "A kid will eat ivy too"    };  
  
        for (int i = 0; i < importantInfo.length; i++) {  
            //Pause for 4 seconds  
            Thread.sleep(4000);  
            //Print a message  
            System.out.println(importantInfo[i]);  
        }  
    }  
}
```

© JMA 2001. All rights reserved

Control de los Threads

- wait (tiempo máximo)
 - Establecen un mecanismo seguro para detener y recuperar la ejecución del thread.
 - Este método bloquea al thread actual en un entorno sincronizado para que el resto pueda ejecutarse.
 - Esto se puede hacer definiendo el método con este modificador o definiendo un bloque.

```
public synchronized void metodo()  
{  
    ...  
}
```

```
public void metodo()  
{  
    synchronized (this)  
    {  
        ...  
    }  
}
```

© JMA 2001. All rights reserved

Control de los Threads

- wait (tiempo maximo)
 - Este método puede provocar una excepción del tipo InterruptedException si el thread es interrumpido durante la espera.
 - Por eso los métodos synchronized deben propagar esta excepción a el thread que lo llamó mediante la palabra throws

```
public class Banco...{  
    ...  
    public synchronized void sacarDinero(int Cantidad) throws InterruptedException{  
        while (saldo<0){  
            wait(); //El thread se queda a la espera de que otro thread cambien  
            // el saldo  
        }  
    }  
}
```

- Se trata mas adelante en Sincronismo.

© JMA 2001. All rights reserved

Control de los Threads

```
public void detener () {
    hiloDetenido = true;
}

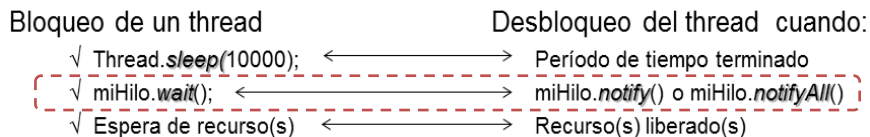
public synchronized void reanudar() {
    hiloDetenido = false;
    this.notify();
}

public void run() {
    while(!fin) {
        txtCrono.setText(String.valueOf(++contador*1000));
        try {
            Thread.sleep(1000);
            synchronized(this) {
                while(hiloDetenido);
                this.wait();
            }
        } catch (InterruptedException e){}
    }
}
```

© JMA 2001. All rights reserved

Control de los Threads

- notify y notifyAll
 - Los métodos bloqueados con wait se quedan en espera y no salen hasta que sean liberados por otros threads o finalice el tiempo de espera.
 - Se puede liberar un thread mediante el método notify para liberar al thread bloqueado o el método notifyAll para liberar a todos los threads bloqueados.



- Se trata mas adelante en Sincronismo.

© JMA 2001. All rights reserved

Control de los Threads

- `join`
 - Espera a que finalice el hilo sobre el que se llama.
 - Se utiliza habitualmente en el hilo padre para esperar a sus hijos.
- `Hilo.isAlive`
 - Devuelve false si el hilo al que se refiere aún no se ha arrancado o está muerto.
- `suspend`
 - Suspende indefinidamente un hilo.
 - No es recomendado su uso → `WAIT`
- `resume`
 - Activación de nuevo de un hilo suspendido
 - No es recomendado su uso → `NOTIFY`

© JMA 2001. All rights reserved

Prioridades

- Cada hilo se crea con una prioridad, idéntica al proceso/hilo principal que lo lanza (habitualmente 5).
- Los hilos con mayor prioridad se ejecutan más rápidamente que los de menos prioridad.
- Las prioridades se establecen de 1 a 10 a través del método `setPriority(int)`.
- También existen constantes definidas en la clase `Thread` que definen la prioridad máxima, mínima y normal:
 - `Thread.MIN_PRIORITY` (1)
 - `Thread.NORMAL_PRIORITY` (5)
 - `Thread.MAX_PRIORITY` (10)
- El método de clase `yield` hace que el intérprete cambie de contexto entre el hilo actual y el siguiente hilo ejecutable disponible, lo que es una manera de asegurar que los hilos de menor prioridad no sufran inanición.

© JMA 2001. All rights reserved

La interfaz Callable

- La interfaz Runnable es una interfaz funcional y tiene un único método run() que no acepta ningún parámetro y no devuelve ningún valor. Esto es adecuado para situaciones en las que no buscamos un resultado de la ejecución del hilo. Dado que la firma del método no tiene la cláusula "throws" especificada, no hay forma de propagar más excepciones comprobadas.
- La interfaz Callable es una interfaz funcional genérica que contiene un único método call(), que devuelve un valor genérico.

```
public class MiCallable implements Callable<Integer> {  
    // ...  
    public Integer call() throws InvalidParamaterException {  
        if(number < 0)  
            throw new InvalidParamaterException("Number should be positive");  
        // ...  
        return number;  
    }  
}
```

© JMA 2001. All rights reserved

Ejecutar un Callable

- Las tareas Runnable se pueden ejecutar usando las clases Thread o ExecutorService, mientras que las Callable solo se pueden ejecutar usando la última.

```
try {  
    ExecutorService servicio= Executors.newFixedThreadPool(1);  
    Future<Integer> resultado= servicio.submit(new MiCallable());  
    if(resultado.isDone())  
        System.out.println(resultado.get());  
} catch (InterruptedException e) {  
    e.printStackTrace();  
} catch (ExecutionException e) {  
    e.getCause().printStackTrace();  
} finally {  
    servicio.shutdown();  
}
```

© JMA 2001. All rights reserved

Future<V>

- Usado ExecutorService se crear un pool de Thread con el número de hilos indicados.
- Para enviar la tarea al pool se utiliza el método submit.
- Al invocar el método submit recibiremos de forma automática una instancia de tipo Future.
- Los métodos isDone() e isCancelled() de Future informan del estado de la tarea y con cancel() se cancela.
- El método get() de Future permite recuperar el valor resultado del call() de la tarea, esperando si es necesario.
- En caso de producirse una excepción en la ejecución del método call(), la invocación a get() lanza una ExecutionException, que envuelve la excepción original. Para acceder a la excepción original, ExecutionException dispone del método getCause().

© JMA 2001. All rights reserved

ExecutorService

- El ExecutorService nos simplifica las tareas asíncronas proveyendo un pool de hilos que son manejados el, abstrayéndonos del trabajo de crearlos y asignarles tareas a estos hilos.
- Se puede instanciar:

```
ExecutorService servicio = new ThreadPoolExecutor(10, 10, 0L, TimeUnit.MILLISECONDS,
    new LinkedBlockingQueue<Runnable>());
ExecutorService servicio = Executors.newFixedThreadPool(10);
ExecutorService servicio = Executors.newSingleThreadExecutor();
```
- Por defecto ExecutorService se queda 'escuchando' nuevas tareas, para que deje de hacerlo cuando termine la última tarea o inmediatamente indicando las tareas pendientes:

```
servicio.shutdown();
List<Runnable> runnableList = servicio.shutdownNow();
```

© JMA 2001. All rights reserved

ExecutorService

- `ExecutorService.execute` es un método void que ejecuta cualquier implementación de `Runnable`, por lo que no puedes conocer el resultado :
`servicio.execute(() -> doExecWork("hi 1"));`
- Para ejecutar un `Callable` que devuelva un resultado se utiliza `ExecutorService.submit`:
`Future<String> future = servicio.submit(() -> doFuncWork("hi with future 1"));`
- `ExecutorService.invokeAny` recibe una lista de `Callable`, ejecuta todas las tareas devolviendo el resultado de una que se ha completado con éxito (es decir, sin lanzar una excepción), si alguna lo hace.
`Collection<Callable<String>> callables = new ArrayList<>();`
`callables.add(() -> doLongWork("hi! 1"));`
`callables.add(() -> doLongWork("hi! 2"));`
`String oneResult = servicio.invokeAny(callables);`
- `ExecutorService.invokeAll` recibe una lista de `Callable`, ejecuta todas las tareas devolviendo una lista de futuros cuando todos se completa.
`List<Future<String>> result = servicios.invokeAll(callables);`

© JMA 2001. All rights reserved

ForkJoinPool (v7)

- `ForkJoinPool` es un tipo de `ExecutorService` que está optimizado para algoritmos de divide y vencerás, tareas que se pueden dividir recursivamente en sub-tareas más pequeñas. Su principal característica es el "robo de trabajo" (work-stealing), donde los hilos que terminan sus tareas buscan y ejecutan tareas de la cola de otros hilos para mantener la CPU ocupada, mejorando el rendimiento en sistemas multinúcleo. Se utiliza a menudo para operaciones paralelas en streams y en problemas que se pueden dividir, como la suma recursiva de números o la multiplicación de matrices.
- `ForkJoinPool` se aplica a tareas que se pueden dividir: Si una tarea puede ser recursivamente dividida en sub-tareas que se pueden ejecutar de forma independiente y luego unir sus resultados, `ForkJoinPool` es la opción ideal.
- `ForkJoinPool` puede mejorar el rendimiento: Su objetivo es aprovechar toda la potencia de procesamiento de los múltiples núcleos del sistema para acelerar la ejecución de cálculos intensivos.
- Frente al `ExecutorService`, `ForkJoinPool` es mejor para tareas que son un problema complejo que se puede dividir en sub-problemas más pequeños y que requieren que los resultados de las sub-tareas se unan al final.

© JMA 2001. All rights reserved

FutureTask

- Future es una interface, por lo que no podemos instanciarla directamente. Una clase que implementa Future es FutureTask. Esta clase permite añadirle un Runnable o un Callable y luego lanzarla en un hilo.

```
FutureTask<String> future = new FutureTask<>(() ->{
    try {
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
}, "Done!");
ForkJoinPool.commonPool().submit(future);
while(!future.isDone()){
    Thread.sleep(100);
}
System.out.println("Future Task Runnable devuelve "+future.get());
```

© JMA 2001. All rights reserved

CompletableFuture (v8)

- CompletableFuture es una extensión de Future que ofrece más funcionalidad. Entre otras cosas, gestiona sus propios hilos y permite encadenar y tratar en conjunto varios Future. Permite:
 - Marcar el proceso como terminado y devolver el resultado: completed, completedFuture, completeExceptionally
 - Añadir un callback para que nos avise cuando el resultado esté disponible: handle, thenAccept, exceptionally, whenComplete.
 - Encadenar varios procesos de forma que unos esperen por los resultados del anterior, formando una cadena: thenApply()
 - Tiene métodos que permiten tratar varios CompletableFuture como si fueran uno solo.

© JMA 2001. All rights reserved

CompletableFuture (v8)

```
CompletableFuture.supplyAsync(() -> {
    System.out.println("CompletableFuture: generando resultado ");
    task(1000);
    return "Done!";
}).thenApply(value -> {
    System.out.println("CompletableFuture: pasando a minúsculas ");
    task(1000);
    return value.toLowerCase();
}).thenApply(value -> {
    System.out.println("CompletableFuture: pasando a bytes ");
    task(1000);
    return value.getBytes();
}).thenAccept(value -> { System.out.println(Arrays.toString(value)); })
.exceptionally(e -> { System.err.println("Algo ha ido mal. Excepción: " + e.getMessage()); return null; });
```

```
void task(int seconds) {
    try {
        Thread.sleep(seconds);
    } catch (InterruptedException e) {
        throw new RuntimeException(e);
    }
}
```

© JMA 2001. All rights reserved

Hilos virtuales (v21)

- El número de hilos disponibles en el sistema es limitado, son costosos de crear y de gestionar y son un potencial cuello de botella.
- Para solucionar este problema se han intentado diversas cosas como el uso de extensiones o frameworks reactivos, pero estas técnicas son complejas y además difíciles de mantener.
- En Java 21 se introdujeron los hilos virtuales. Los hilos virtuales son hilos ligeros que reducen drásticamente el esfuerzo necesario para escribir, mantener y supervisar aplicaciones concurrentes de alto rendimiento. Gracias a ellos, es posible ejecutar millones de hilos virtuales utilizando por debajo tan solo unos pocos hilos del sistema operativo, y apenas es necesario modificar el código Java multihilo pre existente, basta con cambiar el FixedThreadPool por el nuevo VirtualThreadPerTaskExecutor.

```
try (var executor = Executors.newFixedThreadPool(100)) {
    try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
```

© JMA 2001. All rights reserved

Hilos virtuales (v21)

- También se pueden crear sin un ejecutor y usarlos de manera individual utilizando los métodos `Thread.startVirtualThread(x)` o `Thread.ofVirtual().start(x)`, a los que se pasa un `Runnable`. `Thread.currentThread().isVirtual()` informa si se está ejecutando dentro de un hilo virtual.
- Los hilos virtuales ofrecen grandes ventajas y cambian el panorama de programación en Java, ya que:
 - Son muy ligeros y rápidos de crear (1 μ s, un microsegundo, la millonésima parte de un segundo: en 1s se podrían crear 1 millón)
 - Requieren mucha menos memoria: unas 1.000 veces menos que un hilo nativo (en pilas de llamada planas). 1 KB de los hilos virtuales frente a 1MB aproximadamente para cada hilo nativo.
 - Los bloqueos se efectúan en el espacio de usuario y no en el kernel del S.O., por lo que los cambios de contexto son muy rápidos.
 - Apenas es necesario cambiar el código respecto al uso de hilos nativos para poder ejecutarlas.

© JMA 2001. All rights reserved

Hilos virtuales (v21)

```
public class Tarea implements Callable<Integer> {
    private final int number;
    public Tarea(int number) {
        this.number = number;
    }
    @Override
    public Integer call() {
        var name = Thread.currentThread().isVirtual() ? "Virtual" : Thread.currentThread().getName();
        System.out.println("Hilo %s - Tarea %d esperando...".formatted(name, number));
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            System.out.println("Hilo %s - Tarea %d cancelada.".formatted(name, number));
            return -1;
        }
        System.out.println("Hilo %s - Tarea %d terminada.".formatted(name, number));
        return ThreadLocalRandom.current().nextInt(100);
    }
}
```

© JMA 2001. All rights reserved

Hilos virtuales (v21)

```
List<Tarea> tasks = new ArrayList<>();
for (int i = 0; i < 1_000; i++) { tasks.add(new Tarea(i)); }
long time = System.currentTimeMillis(), sum = 0;

// try (var executor = Executors.newFixedThreadPool(100)) {
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
    List<Future<Integer>> futures = executor.invokeAll(tasks);
    for (Future<Integer> future : futures) {
        sum += future.get();
    }
    time = System.currentTimeMillis() - time;
    System.out.println("sum = " + sum + "; time = " + time + " ms");
}
```

© JMA 2001. All rights reserved

HTTPCLIENT (v11)

© JMA 2001. All rights reserved

HttpClient

- El HttpClient se introdujo como un módulo incubador en Java 9 y de forma definitiva en Java 11 como parte de JEP 321 .
- Su propósito es enviar solicitudes HTTP y recibir sus respuestas de una manera moderna, eficiente y flexible.
- HttpClient reemplaza la clase heredada HttpURLConnection presente en el JDK desde las primeras versiones de Java.
- Algunas de sus características incluyen:
 - Soporte para HTTP/1.1, HTTP/2 y Web Socket.
 - Soporte para modelos de programación síncronos y asíncronos.
 - Manejo de cuerpos de solicitud y respuesta como flujos reactivos.
 - Soporte para cookies.

© JMA 2001. All rights reserved

Clases

Clase	Descripción	Patrón de Creación
HttpClient	Contenedor de configuración. Es el objeto que envía las solicitudes. Puede ser reutilizado para múltiples peticiones.	HttpClient.newBuilder().build()
HttpRequest	Representa la solicitud a enviar (URL, método, encabezados, cuerpo). Es inmutable.	HttpRequest.newBuilder().build()
HttpResponse<T>	Representa la respuesta recibida (código de estado, encabezados, cuerpo). El tipo T depende de cómo se maneje el cuerpo.	Devuelto por los métodos send o sendAsync.
BodyPublishers	Transforma en un formato de stream los datos (String, File, InputStream, ByteArray, ...) a enviar con POST o PUT.	

© JMA 2001. All rights reserved

Flujo de la petición

- Usar el `HttpClient` sigue un flujo de tres pasos, utilizando intensivamente el patrón Builder:
 1. Crear y configurar un **HttpClient** (el remitente).
 2. Crear y configurar un **HttpRequest** (el mensaje).
 3. Enviar la solicitud y manejar el **HttpResponse** (la recepción).

```
HttpClient cliente = HttpClient.newHttpClient(); // 1. Crear el cliente
HttpRequest solicitud = HttpRequest.newBuilder() // 2. Crear la solicitud (GET por defecto)
    .uri(URI.create("https://jsonplaceholder.typicode.com/posts/1"))
    .header("Accept", "application/json") // Opcional: añadir encabezado
    .build();
try { // 3. Enviar la solicitud de forma síncrona
    HttpResponse<String> respuesta = cliente.send(solicitud, HttpResponse.BodyHandlers.ofString());
    System.out.println("Código de estado: " + respuesta.statusCode());
    System.out.println("Cuerpo de la respuesta:\n" + respuesta.body());
} catch (Exception e) {
    e.printStackTrace();
}
```

© JMA 2001. All rights reserved

Configuración

- Aunque `HttpClient.newHttpClient()` es suficiente para la mayoría de los casos, se pueden usar configuraciones avanzadas:
 - `.version()`: Elige la versión HTTP (HTTP_1_1 o HTTP_2).
 - `.followRedirects()`: Define la política de redirección.
 - `.connectTimeout()`: Establece el tiempo de espera para establecer la conexión.
 - `.proxy()`: Configura un proxy.
- El `HttpRequest.newBuilder()` permite especificar (carece de PATCH):

	Método del Builder	Ejemplo
URL	<code>.uri()</code>	<code>.uri(URI.create("https://api.ejemplo.com"))</code>
GET	<code>.GET()</code> (por defecto)	<code>.GET()</code>
POST/PUT	<code>.POST()</code> <code>.PUT()</code> y <code>.bodyPublisher()</code>	<code>.POST(BodyPublishers.ofString("{\"key\":\"value\"}))</code>
DELETE	<code>.DELETE()</code>	<code>.DELETE()</code>
HEAD	<code>.header()</code>	<code>.header("Content-Type", "application/json")</code>
Tiempo de espera	<code>.timeout()</code>	<code>.timeout(Duration.ofSeconds(5))</code>

© JMA 2001. All rights reserved

Configuración

- El `HttpResponse.BodyHandlers`, usado en `send()` o `sendAsync()`, define el tipo de retorno para el cuerpo de la respuesta (el `T` del `HttpResponse<T>`).
 - `.ofString()`: `String`, para lectura de cuerpos de texto (JSON, HTML)
 - `.ofByteArray()`: `byte[]`, para datos binarios (imágenes, archivos pequeños)
 - `.ofFile(Path)`: `Path`, para guardar el cuerpo directamente en un archivo
 - `.discarding()`: `void`, cuando solo interesa el `http status` y no el cuerpo.
- `BodyPublishers` es la clase auxiliar, utilizada con `POST` o `PUT`, que toma una fuente de datos (`String`, `File`, `InputStream`, `ByteArray`, ...) y la convierte en el flujo de datos (stream) que `HttpClient` puede "publicar" o transmitir eficientemente a través de la red.
- Uno de los grandes beneficios es la capacidad asíncrona, que no bloquea el hilo principal y utiliza `CompletableFuture`. Esto es ideal para aplicaciones de alto rendimiento (servidores, microservicios). Se usa el método `client.sendAsync()`.

```
CompletableFuture<HttpResponse<String>> futuroRespuesta = cliente.sendAsync(solicitud,
    HttpResponse.BodyHandlers.ofString()
);
```

© JMA 2001. All rights reserved

Ejemplo completo

```
HttpClient client = HttpClient.newBuilder()
    .version(Version.HTTP_2)
    .followRedirects(Redirect.NORMAL)
    .proxy(ProxySelector.of(new InetSocketAddress("proxy.corp", 8080)))
    .build();

HttpRequest request = HttpRequest.newBuilder()
    .uri(new URI("https://picsum.photos/v2/list?limit=10"))
    .GET()
    .header("Accept", "application/json")
    .timeout(Duration.ofSeconds(10))
    .build();

client.sendAsync(request, BodyHandlers.ofString())
    .thenApply(HttpResponse::body)
    .thenAccept(System.out::println)
    .join();
```

© JMA 2001. All rights reserved

REACTIVE STREAMS (v9)

© JMA 2001. All rights reserved

El problema

- La asincronía es necesaria para permitir el uso paralelo de los recursos informáticos y su optimización.
- El manejo de flujos de datos, especialmente datos "en vivo" cuyo volumen no está predeterminado, requiere un cuidado especial en un sistema asincrónico. El problema más importante es que el consumo de recursos debe controlarse de manera que una fuente de datos rápida no sature al destinatario de la transmisión.
- En la programación asíncrona tradicional (push) es frecuente que haya un componente que produce datos (un Publicador o Publisher) mucho más rápido de lo que un componente consumidor (Subscriber) puede procesarlos. Esto provoca lo que se conoce como "Resource Exhaustion": el sistema consumidor se ve obligado a crear buffers de memoria sin límite para cachear los datos aun no procesados, lo que inevitablemente lleva a errores de "Out of Memory" y a la inestabilidad del sistema si la situación se mantiene mucho tiempo.

© JMA 2001. All rights reserved

La solución

- Reactive Streams soluciona este problema invirtiendo el control. En lugar de un modelo puramente push (en que el productor empuja datos al consumidor), se utiliza un modelo híbrido pull/push mediante la Contrapresión sin bloqueo:
 - El Consumidor (Subscriber) no espera pasivamente, sino que solicita explícitamente la cantidad de elementos que está listo para procesar (un mecanismo pull).
 - El Productor (Publisher) solo debe enviar (hacer push) como máximo de la cantidad de elementos que le han sido solicitados.
- Esto permite al consumidor controlar dinámicamente la velocidad del flujo de datos, asegurando que nunca se vea sobrecargado.
- El objetivo principal de Reactive Streams es gobernar el intercambio de flujos de datos a través de un límite asíncrono (pasar elementos a otro subproceso o grupo) al tiempo que garantiza que el lado receptor no se vea obligado a almacenar en búfer cantidades arbitrarias de datos. La contrapresión es una parte integral de este modelo para permitir que las colas que median entre hilos estén delimitadas. Los beneficios del procesamiento asíncrono se anularían si la comunicación de la contrapresión fuera síncrona, se debe exigir un comportamiento totalmente asíncrono y sin bloqueo en todos los aspectos.

© JMA 2001. All rights reserved

Flujos reactivos

- Reactive Streams es una iniciativa para proporcionar un estándar para el procesamiento de flujo asíncrono con contrapresión sin bloqueo. Esto abarca esfuerzos dirigidos a entornos de ejecución (JVM y JavaScript), así como a protocolos de red.
- Dicha iniciativa publicó en Julio de 2015 la especificación de Reactive Streams 1.0.0 for the JVM con la participación de equipos de, entre otros, Netflix, Red Hat, Pivotal, Oracle y Typesafe.
- Java 9 (Septiembre de 2017) introdujo la clase `java.util.concurrent.Flow` en un intento de estandarizar de forma nativa la implementación de Reactive Streams. Las interfaces disponibles en `java.util.concurrent.Flow` eran semánticamente equivalentes a sus respectivas contrapartes de Reactive Streams, pero no coincidentes, lo que supuso la migración de las bibliotecas existentes para adoptar los nuevos tipos en el JDK.

© JMA 2001. All rights reserved

Reactive Streams

- Los Reactive Streams se asemejan al patrón Observer, con un modelo de publicadores y suscriptores. Los publicadores generan una secuencia de elementos que envían a los suscriptores, pero teniendo además estos últimos la capacidad de limitar el número de elementos que quieren recibir.
- Reactive Streams es un estándar y una especificación de bibliotecas para:
 - procesar un número potencialmente ilimitado de elementos,
 - en secuencia,
 - pasando los elementos de forma asincrónica entre componentes,
 - con contrapresión no bloqueante obligatoria.

© JMA 2001. All rights reserved

Componentes API

- La API consta de los siguientes componentes que deben proporcionar las implementaciones de Reactive Stream:
 - Publicador
 - Suscriptor
 - Procesador
 - Suscripción
- Un Publicador es un proveedor de un número potencialmente ilimitado de elementos sucesivos, publicándolos de acuerdo con la demanda recibida de sus suscriptores.
- El Publicador admite las suscripciones de Suscriptores que reciben un objeto Suscripción con el que demandar elementos o cancelar la demanda.
- El Suscriptor habilita los mecanismos para que el Publicador le pueda indicar: el siguiente elemento (cuando se genere), el error (si hay un fallo) o que se ha completado (cuando ya no va a haber más elementos disponibles).
- La secuencia es:
 - `onSubscribe onNext* (onError | onComplete)?`

© JMA 2001. All rights reserved

Interfaces Flow

- Java ha añadido interfaces con el mínimo número de métodos necesarios para implementar Reactive Streams, pero no implementaciones de dichas interfaces. Con la creación de estas interfaces dentro del propio JDK se unifican por parte de Java las distintas propuestas existentes actualmente para la implementación de Reactive Streams.
- Las 4 interfaces son:
 - `Flow.Publisher<T>`: Productor de artículos (y eventos de control) recibidos por los suscriptores.
 - `Flow.Subscriber<T>`: Receptor de artículos y mensajes.
 - `Flow.Processor<S, P>`: Componente que actúa tanto como suscriptor y publicador.
 - `Flow.Subscription`: Objeto de comunicación del suscriptor con el publicador.
- Establecen mecanismos de comunicación de una única dirección, mediante procedimientos con un parámetro como máximo, sin acoplamiento.

© JMA 2001. All rights reserved

Flow.Publisher<T>

- La interface funcional `Flow.Publisher<T>` representa a los publicadores.
`@FunctionalInterface`

```
public static interface Publisher<T> {  
    public void subscribe(Subscriber<? super T> subscriber);  
}
```
- El método `subscribe` permite que los suscriptores se suscriban a un publicador.
- El método recibe una instancia de `Subscriber` y, a través de los métodos de la instancia, el publicador se comunicará con el suscriptor.
- El publicador debe elevar una excepción de tipo `NullPointerException` si la instancia de `Subscriber` pasada como parámetro es nula.

© JMA 2001. All rights reserved

Flow.Subscriber<T>

- La interface Flow.Subscriber<T> representa los suscriptores. Es utilizada por los publicadores para notificar eventos a los suscriptores.

```
public static interface Subscriber<T> {  
    public void onSubscribe(Subscription subscription);  
    public void onNext(T item);  
    public void onError(Throwable throwable);  
    public void onComplete();  
}
```

- onSubscribe:** una vez aceptada la suscripción (suscribe), se notifica al suscriptor la instancia Subscription a través de la podrá que controlar la suscripción.
- onNext:** pasa al suscriptor los elementos generados por la suscripción.
- onComplete:** se invoca cuando la suscripción termina, ya sea porque se han consumido el número de elementos solicitado por el suscriptor o el publicador no va a producir nuevos valores, después de lo cual no se invocará ningún método del Suscriptor.
- onError:** se invoca tras un error irrecuperable encontrado por un publicador o una suscripción, después del cual el publicador no invoca ningún otro método de Suscriptor. La excepción se pasa como argumento por lo que no puede ser interceptada.
- De forma general no esta definido el comportamiento si se produce un error y se eleva una excepción durante la invocación de cualquiera de estos métodos por parte del suscriptor.

© JMA 2001. All rights reserved

Flow.Subscription

- La interface Flow.Subscription representa las suscripciones. Los generan los publicadores para que los suscriptores puedan controlar la suscripción.

```
public static interface Subscription {  
    public void request(long n);  
    public void cancel();  
}
```

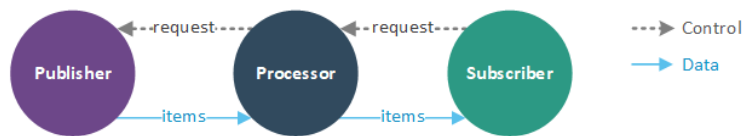
- request:** indica al publicador que está en disposición de recibir los siguientes n elementos de la suscripción, además de los ya solicitados (se añaden a los pendientes). Si es menor o igual a cero, se llamará al método onError con una IllegalArgumentException. Si se solicita Long.MAX_VALUE se interpretará que el suscriptor quiere recibir todos los elementos generados por la suscripción sin ningún tipo de restricción en cuanto a su volumen, cuando n tiende a infinito.
- cancel:** hace que el suscriptor (eventualmente) deje de recibir mensajes. Debido a la naturaleza asíncrona de todo el proceso, se pueden recibir mensajes onNext adicionales pendientes después de invocar la cancelación, pero nunca onComplete o onError.

© JMA 2001. All rights reserved

Flow.Processor<S, P>

- La interface Flow.Processor<S, P> representa un componente que se comporta tanto como un publicador como un suscriptor.

```
public static interface Processor<S, P> extends Subscriber<S>, Publisher<P> {  
    }  
}
```
- El propósito principal de esta interface es permitir la implementación por componentes intermedios que se dediquen a realizar algún tipo de transformación sobre los datos recibidos de un publicador para publicarlos para otros componentes una vez transformados.
- Siendo S el tipo de origen, al que se suscribe (suscriptor), y P el tipo de destino, el que publica (publicador).



© JMA 2001. All rights reserved

Reglas para los Publisher

1. Un publicador DEBE enviar al suscriptor un número total de elementos igual o menor que el número total de elementos solicitados por dicho suscriptor.
2. Un publicador PODRÍA enviar menos elementos que los solicitados por un suscriptor, por ejemplo cuando ya no sea capaz de producir más elementos a partir de su fuente origen de información.
3. Un publicador DEBE invocar los métodos `onSubscribe`, `onNext`, `onError` y `onComplete` de un suscriptor de forma thread-safe y utilizando algún mecanismo de sincronización en caso de utilizar múltiples threads.
4. Un publicador DEBE comunicar sus errores invocando el método `onError` de sus suscriptores.
5. Un publicador DEBE comunicar que ya no puede generar más elementos invocando el método `onComplete` de sus suscriptores.
6. Un publicador que invoque los métodos `onError` u `onComplete` de un suscriptor DEBE considerar cancelada dicha suscripción.

© JMA 2001. All rights reserved

Reglas para los Publisher

7. Un publicador NO DEBERÍA invocar ningún otro método de un suscriptor después de haber invocado sus métodos `onError` u `onComplete`.
8. Un publicador DEBE dejar de invocar eventualmente métodos de un suscriptor si su suscripción ha sido cancelada. No obstante, debido a la naturaleza asíncrona del proceso, y la demora entre la petición de cancelación y su cancelación efectiva, podría llegar a invocar alguno de sus métodos.
9. Un publicador DEBE, dentro de la implementación de su método `subscribe`, invocar el método `onSubscribe` del suscriptor pasado la suscripción como parámetro antes de invocar cualquier otro de método de dicho suscriptor. El método debe terminar normalmente, excepto si el suscriptor pasado como parámetro es nulo, en cuyo caso debe elevar una excepción de tipo `NullPointerException`. Cualquier otro tipo de error debe realizarse invocando el método `onError` del suscriptor.
10. Un publicador NO DEBE permitir la suscripción de un mismo suscriptor más de una vez.
11. Un publicador PUEDE decidir el número de suscriptores distintos que soporta al mismo tiempo y si la implementación de las suscripciones la realiza de forma unicast (transmisión de uno a uno) o multicast (transmisión de uno a muchos).

© JMA 2001. All rights reserved

Reglas para los Subscriber

1. Un suscriptor DEBE llamar al método `request` de `Subscription` para empezar a recibir elementos generados por la suscripción. Es decir, es responsabilidad del suscriptor indicar al publicador cuando quiere empezar a recibir y cuantos elementos está preparado para recibir.
2. Un suscriptor DEBERÍA procesar un elemento de forma asíncrona si estima que su tiempo de respuesta puede afectar la calidad de servicio del publicador.
3. Un suscriptor NO DEBE llamar a ningún método de `Subscription` ni `Publisher` dentro de sus métodos `onComplete` y `onError` para evitar ciclos y condiciones de carrera.
4. Un suscriptor DEBE considerar cancelada una suscripción cuando el publicador invoca su método `onComplete` u `onError`.
5. Un suscriptor DEBE llamar al método `cancel` de `Subscription` cuando un publicador invoca su método `onSubscribe` teniendo el suscriptor ya una suscripción activa.
6. Un suscriptor DEBE llamar al método `cancel` de `Subscription` cuando la suscripción ya no sea necesaria para permitir liberar los recursos asociados a dicha suscripción.
7. Un suscriptor DEBE asegurar que todas las llamadas a los métodos de `Subscription` se realizan desde un mismo thread, o utilizando algún método de sincronización en caso de que la suscripción se esté usando concurrentemente desde distintos threads.

© JMA 2001. All rights reserved

Reglas para los Subscriber

8. Un suscriptor DEBE estar preparado para recibir elementos a través de su método `onNext`, aún después de haber llamado al método `cancel` de `Subscription`, debido a la naturaleza asíncrona del proceso y la demora que se produce entre que el suscriptor solicita la cancelación y el publicador la cancela realmente.
9. Un suscriptor DEBE estar preparado para recibir la finalización de la suscripción a través de su método `onComplete`, incluso antes de haber llamado al método `request` de `Subscription`, debido a que el publicador tiene potestad para dar por terminada una suscripción.
10. Un suscriptor DEBE estar preparado para recibir la finalización de la suscripción a través de su método `onError`, incluso antes de haber llamado al método `request` de `Subscription`, debido a errores en el publicador.
11. Un suscriptor DEBE asegurar que el procesamiento asíncrono de sus métodos `onSubscribe`, `onNext`, `onComplete` y `onError` se realiza de manera `thread-safe`.
12. Un mismo suscriptor DEBE suscribirse a un mismo publicador una única vez.
13. Un suscriptor DEBE terminar las invocaciones a sus métodos `onSubscribe`, `onNext`, `onComplete` y `onError` sin elevar ninguna excepción. Excepto cuando algún parámetro recibido sea nulo, en cuyo caso deberá elevar una excepción de tipo `NullPointerException`. Cualquier otro error debe resolverse cancelando la suscripción. El publicador no puede tratar los errores del suscriptor, por lo que dará por cancelada la suscripción en caso de producirse alguno.

© JMA 2001. All rights reserved

Reglas para los Subscription

1. Una suscripción es el único componente a través del que se establece la comunicación entre un suscriptor y un publicador. Un objeto `Subscription` concreto pertenece de forma exclusiva a un `Subscriber` concreto con un `Publisher` específico. Sólo el suscriptor DEBE llamar a los métodos `request` y `cancel` de `Subscription`.
2. Una suscripción DEBE permitir que se llame a su método `request` desde los métodos `onSubscribe` y `onNext` de un suscriptor de manera síncrona. Es decir, la implementación del método `request` debe ser reentrante.
3. Una suscripción DEBE establecer un número máximo de llamadas recursivas entre los métodos `request` de `Subscriber` y `onNext` de `Subscription`. La recomendación a este respecto es limitar a uno el número de llamadas recursivas evitando la secuencia `request -> onNext -> request -> onNext -> ...`
4. Una suscripción DEBERÍA implementar su método `request` de forma que retorne lo antes posible para no afectar la calidad de servicio del suscriptor que lo invoque.
5. Una suscripción DEBE implementar su método `cancel` de forma idempotente, `thread-safe` y retornar lo antes posible para no afectar la calidad del servicio del suscriptor que lo invoque.
6. Una suscripción NO DEBE realizar ninguna acción cuando se llame a su método `request` una vez que la suscripción haya sido cancelada.
7. Una suscripción NO DEBE realizar ninguna acción cuando se llame a su método `cancel` una vez que la suscripción haya sido cancelada.
8. Una suscripción DEBE almacenar la suma de todas las cantidades de elementos solicitadas por todas las llamadas realizadas por el suscriptor a su método `request`.

© JMA 2001. All rights reserved

Reglas para los Subscription

9. Una suscripción DEBE invocar al método `onError` del suscriptor con una instancia de una excepción `IllegalArgumentException` si se solicita una cantidad de elementos igual o menor que cero en alguna llamada realizada por el suscriptor a su método `request`.
10. Una suscripción PODRÍA implementar en su método `request` invocaciones al método `onNext` del suscriptor de manera síncrona, siempre y cuando la suscripción no haya sido ya cancelada.
11. Una suscripción PODRÍA implementar en su método `request` invocaciones al método `onComplete` del suscriptor de manera síncrona, siempre y cuando la suscripción no haya sido ya cancelada.
12. Una suscripción DEBE requerir al Publisher que deje de invocar los métodos del Subscriber cuando el suscriptor llame a su método `cancel`, siempre y cuando la suscripción no haya sido ya cancelada.
13. Una suscripción DEBE requerir del Publisher que libere cualquier referencia al Subscriber cuando el suscriptor llame a su método `cancel`, siempre y cuando la suscripción no haya sido cancelada.
14. Una suscripción PODRÍA causar que el Publisher pase a un estado de finalizado cuando el suscriptor llame a su método `cancel`, siempre y cuando la suscripción no haya sido ya cancelada.
15. Una suscripción NO DEBE elevar excepciones en la implementación de su método `cancel`.
16. Una suscripción NO DEBE elevar excepciones en la implementación de su método `request`.
17. Una suscripción DEBE permitir peticiones de una cantidad de elementos potencialmente infinita. Una petición de $2^{63}-1$ (`java.lang.Long.MAX_VALUE`) o más elementos se considera como una petición de una cantidad infinita de elementos.

© JMA 2001. All rights reserved

Reglas para los Processor

1. Un procesador DEBE cumplir tanto los requerimientos establecidos para los publicadores como para los suscriptores.
2. Un procesador PODRÍA elegir recuperarse de una invocación a su método `onError` por parte de un publicador, pero en ese caso DEBE considerar la suscripción cancelada, en caso contrario DEBE propagar el error inmediatamente invocando el método `onError` de sus suscriptores.

© JMA 2001. All rights reserved

SubmissionPublisher<T>

- SubmissionPublisher es una implementación de Flow.Publisher que emite elementos (no nulos) enviados de forma asíncrona a los suscriptores actuales hasta que se cierra. Cada suscriptor actual recibe los elementos recién enviados en el mismo orden a menos que se encuentren caídos o en excepciones. Su permite que los generadores de elementos actúen como publicadores de flujos reactivos que se basan en el manejo de caídas y/o de bloqueo del control del flujo.
- Un SubmissionPublisher utiliza el Executor suministrado en su constructor para la entrega a los suscriptores. La mejor elección de Ejecutor depende del uso esperado.
- El almacenamiento en búfer permite a los productores y consumidores operar a diferentes velocidades. A cada suscriptor se le asigna un búfer independiente. Los búferes se crean con el primer uso y se expanden según sea necesario hasta el máximo dado.
- Los métodos de publicación admiten diferentes políticas sobre qué hacer cuando los búferes están saturados:
 - submit: Publica el elemento dado para cada suscriptor actual invocando de forma asíncrona su método onNext, bloqueando ininterrumpidamente los recursos para cualquier suscriptor mientras no están disponibles.
 - offer: A diferencia del submit, solo mantiene el bloqueo hasta el tiempo de espera especificado o hasta que se interrumpe el hilo, en cuyo caso la invocación del controlador de caída suministrado debe indicar si se reintenta.

© JMA 2001. All rights reserved

SubmissionPublisher<T>

- Si algún método del suscriptor genera una excepción, su suscripción se cancela. Si se proporciona un controlador de errores como un argumento del constructor, se invoca antes de la cancelación en caso de una excepción en el método onNext, pero excepciones en los métodos onSubscribe, onError y onComplete no se registran ni se maneja antes de la cancelación. Si el Executor suministrado arroja RejectedExecutionException (o cualquier otra RuntimeException o Error) al intentar ejecutar una tarea, o un controlador de caída arroja una excepción al procesar un elemento eliminado, la excepción se vuelve a lanzar. En estos casos, no todos los suscriptores habrán recibido el artículo publicado. Por lo general, es una buena práctica en estos casos invocar a closeExceptionally que, a menos que ya esté cerrado, ejecuta los onError de los suscriptores actuales con el error dado y no permite los intentos posteriores de publicación.
- El método consume(Consumer) simplifica el soporte para el caso común en el que la única acción de un suscriptor es solicitar y procesar todos los elementos utilizando una función proporcionada.
- Esta clase también puede servir como una base conveniente para las subclases que generan elementos y utilizar los métodos de esta clase para publicarlos.

© JMA 2001. All rights reserved

PrintSubscriber

```
public class PrintSubscriber implements Flow.Subscriber<Integer> {
    private Flow.Subscription subscription;
    private boolean continuar = true;
    private boolean isCancel = false;

    @Override
    public void onSubscribe(Subscription subscription) {
        this.subscription = subscription;
        subscription.request(1);
    }

    @Override
    public void onNext(Integer item) {
        System.out.println("PrintSubscriber: " + item);
        Sleeper.sleep(1000);
        if(continuar && !isCancel) subscription.request(1);
    }

    @Override
    public void onError(Throwable error) {
        error.printStackTrace();
    }

    @Override
    public void onComplete() {
        System.out.println("PrintSubscriber onComplete");
    }

    public void cancel() {
        isCancel = true;
        subscription.cancel();
    }

    public void pause() {
        continuar = false;
    }

    public void continua() {
        if(isCancel) throw new IllegalStateException("Is cancel");
        continuar = true;
        subscription.request(1);
    }
}
```

© JMA 2001. All rights reserved

CacheSubscriber

```
public class CacheSubscriber implements Flow.Subscriber<Integer> {
    private Flow.Subscription subscription;
    private List<Integer> cache = new ArrayList<Integer>();
    private final int MAX_CACHE;

    public CacheSubscriber(int maxCache) {
        MAX_CACHE = maxCache;
    }

    @Override
    public void onSubscribe(Subscription subscription) {
        this.subscription = subscription;
        subscription.request(MAX_CACHE);
    }

    @Override
    public void onNext(Integer item) {
        cache.add(item);
        if(cache.size() == MAX_CACHE) {
            pinta();
            cache.clear();
            subscription.request(MAX_CACHE);
        }
    }

    @Override
    public void onError(Throwable error) {
        pinta();
        error.printStackTrace();
    }

    @Override
    public void onComplete() {
        pinta();
        System.out.println("CacheSubscriber onComplete");
    }

    public void pinta() {
        System.out.println(">>>> Elementos de la cache");
        cache.forEach(System.out::println);
        System.out.println("<<<< Elementos de la cache");
    }
}
```

© JMA 2001. All rights reserved

GenericProcessor

```
public static class GenericProcessor<S, P> extends SubmissionPublisher<P> implements Flow.Processor<S, P> {
    private Flow.Subscription subscription;
    private Function<S, P> function;

    public GenericProcessor(Function<S, P> function) {
        super();
        this.function = function;
    }
    @Override
    public void onSubscribe(Flow.Subscription subscription) {
        this.subscription = subscription;
        subscription.request(Long.MAX_VALUE);
    }
    @Override
    public void onNext(S item) { submit(function.apply(item)); }
    @Override
    public void onError(Throwable error) { error.printStackTrace(); }
    @Override
    public void onComplete() {
        System.out.println("GenericProcessor completed");
        close();
    }
}
```

© JMA 2001. All rights reserved

SubmissionPublisher

```
public static void ejecutar() throws Exception {
    SubmissionPublisher<Integer> publisher = new SubmissionPublisher<>();
    Flow.Processor<Integer, Integer> processor = new GenericProcessor<>(item -> item * 2);
    Flow.Subscriber<Integer> display = new PrintSubscriber();
    Flow.Subscriber<Integer> cache = new CacheSubscriber(10);

    publisher.subscribe(cache);
    publisher.subscribe(processor);
    processor.subscribe(display);

    IntStream.range(0, 10).forEach(item -> {
        System.out.println("Generated item: " + item);
        publisher.submit(item);
        Sleeper.sleep(500);
    });

    publisher.close();
    System.out.println("Publisher completed and closed");
    Sleeper.sleep(6000);
}
```

```
private static class Sleeper {
    private static void sleep(int time) {
        try {
            Thread.sleep(time);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

© JMA 2001. All rights reserved

Publisher personalizado

```
public class NotificationService extends
SubmissionPublisher<NotificationService.Notification> {
    public static enum Types { ERROR, WARN, INFO, LOG }
    public static record Notification(String message, Types type) {
        public Notification(String message) { this(message, Types.ERROR); }
        public Notification(String message, Types type) {
            if (message == null || message.isBlank())
                throw new IllegalArgumentException("El mensaje no puede ser
                nulo o estar en blanco");
            if (type == null)
                throw new IllegalArgumentException("El tipo es obligatorio");
            this.message = message;
            this.type = type;
        }
    }
    private List<Notification> list = new ArrayList<Notification>();
    private Consumer<Notification> output;
    public void setOutput(Consumer<Notification> output) { this.output =
        output; }
    public List<Notification> getNotifications() { return
        Collections.unmodifiableList(list); }

    public void add(String message, Types type) {
        if (message == null || message.isBlank())
            throw new IllegalArgumentException();
        var msg = new Notification(message, type);
        list.add(msg);
        if (output != null)
            output.accept(msg);
        this.submit(msg);
    }
    public void add(String message) { add(message, Types.ERROR); }
    public void remove(int index) {
        if (index < 0 || index >= list.size())
            throw new IndexOutOfBoundsException();
        list.remove(index);
    }
    public void clear() { list.clear(); }
    public boolean hasNotifications() { return list.size() > 0; }
    public boolean hasErrors() {
        return list.stream().anyMatch(item -> item.type() == Types.ERROR);
    }
}
```

© JMA 2001. All rights reserved

Executor personalizado

```
static class PeriodicPublisher<T> extends SubmissionPublisher<T> {
    final ScheduledFuture<?> periodicTask;
    final ScheduledExecutorService scheduler;
    PeriodicPublisher(Executor executor, int maxBufferCapacity, Supplier<? extends T> supplier, long period,
        TimeUnit unit) {
        super(executor, maxBufferCapacity);
        scheduler = new ScheduledThreadPoolExecutor(1);
        periodicTask = scheduler.scheduleAtFixedRate(() -> submit(supplier.get()), 0, period, unit);
    }
    public void close() {
        periodicTask.cancel(false);
        scheduler.shutdown();
        super.close();
    }
}

private static int value = 0;
public static void ejecutar() throws Exception {
    ExecutorService servicio = Executors.newFixedThreadPool(10);
    SubmissionPublisher<Integer> publisher = new PeriodicPublisher<>(servicio, 10, () -> value++, 1, TimeUnit.SECONDS);
    publisher.subscribe(new PrintSubscriber());
    servicio.awaitTermination(10, TimeUnit.SECONDS);
    publisher.close(); System.out.println("Publisher completed and closed");
}
```

© JMA 2001. All rights reserved

Implementaciones

- Reactive Streams es solo la especificación. `java.util.concurrent` solo suministra los interfaces `Flow` y una implementación básica de un publicador a través de `SubmissionPublisher`.
- Las librerías que implementan esta especificación y proporcionan una rica API de operadores (`map`, `filter`, `merge`, etc.) son:
 - RxJava: Una de las implementaciones originales del concepto reactivo, que también cuenta con la versión RxJS (JavaScript).
 - Reactor: La base de Spring WebFlux.
 - Akka Streams: Utilizada en el ecosistema de Scala/Akka.
 - Reactive Rabbit
 - Ratpack
 - Slick
 - Vert.x 3.0